



## End-of-Studies Project

Submitted on 21 September 2026  
in partial fulfillment of the requirements  
for the Professional Master's degree in Data Science

Presented and completed by :

**Adem MRABET**

---

## DAM AI Assistant

Grounded question answering over the African Development Bank's authority matrix

---

Prepared within :



|                     |                                      |                            |
|---------------------|--------------------------------------|----------------------------|
| Dr. ....            | Professor at Polytech Intl           | President                  |
| Dr. ....            | Professor at Polytech Intl           | Reviewer                   |
| M. Yasser AHMAD     | Operations Manager – AfDB            | Professional supervisor    |
| M. Surya JOYMUNGUL  | Chief Regional IT Coordinator – AfDB | Co-professional supervisor |
| Dr. Marouene CHAIEB | Professor at Polytech Intl           | Academic Supervisor        |

**Professional Supervisor**

Mr. Yasser AHMAD

---

Signature

**Academic Supervisor**

Dr. Marouene CHAIEB

---

Signature

# Dedication

*I dedicate this work:*

**To my father,** for the years you carried things you never mentioned. You spent your life putting us ahead of yourself, clearing our paths even when your own was not clear, and you taught us kindness without ever making it a lesson. Nothing in these pages would exist without that.

**To my mother,** who has never needed words to say it. Your love has always arrived as things quietly done: worries carried alone so they would not reach us, and a hundred small comforts arranged before any of us noticed they were missing. I hope this makes you proud enough to feel it was worth something.

**To my brother,** who has spent his life pretending to be harder than he is. Four years younger, and somehow you have been the one who shows up.

**To Nadine Sliti,** who showed me what it looks like when love is not exhausting. I kept the doubt and the exhaustion from you as best I could, and you knew anyway, every time, and sat with me in it without being asked. You held the standard when I wanted to lower it. This work has my name on it and your fingerprint all through it.

**And to all my friends.** To Ayoub and Alaa, who walked the same road at the same time and still set their own worries aside whenever I needed advice. To Fayrouz, who was already there before anyone thought to ask, and to Nour El Houda, who could lift the mood of a room simply by walking into it. To Brahim and to Issam, for being constant. Thank you, all of you, for making a long year lighter than it had any right to be.

Adem

# Acknowledgments

---

I would like to express my sincere gratitude to Dr. Marouene CHAIEB, my academic supervisor at Polytech Intl, for his guidance throughout this project. His questions consistently pushed the work toward being defensible rather than merely finished, and several of the design decisions recorded in this report exist because he declined to accept the easier answer.

I am equally grateful to Mr. Yasser AHMAD, Operations Manager, and Mr. Surya JOYMUNGUL, Chief Regional IT Coordinator, my professional supervisors at the African Development Bank. They gave me access to the institutional context that gives this work its purpose, and the time to understand the Delegation of Authority Matrix as a governance instrument rather than as a document to be parsed. The distinction shaped the entire project.

My thanks go to the teaching staff of Polytech Intl, whose instruction over the course of the Master's programme in Data Science provided the foundation this project was built on.

Finally, I thank the members of the jury for the time and attention they have given to reading and evaluating this work.

# Abstract

---

The Delegation of Authority Matrix (DAM) of the African Development Bank specifies, for every operational activity, which roles may approve, review, verify, or initiate it, and which must be informed of it. Consulted as a page-by-page PDF, it is easy to misread: authority is encoded in seventeen level-qualified codes across five obligations, and near-identical activities can carry different approvers depending on a monetary threshold. A misreading is therefore a governance failure, not a mere inconvenience, since an approval beyond a role's delegated authority can look entirely legitimate to everyone involved.

This work delivers an agent that answers authority questions over the DAM without inventing anything. A deterministic, eight-stage pipeline extracts the matrix from the source PDF into a knowledge graph of 327 activity nodes, 1,776 individual responsibilities, 131 distinct roles, and 2,108 edges, with an unresolved-role rate of 0.6%. Questions are resolved against that graph, by identifier or by TF-IDF lexical search with deterministic typo correction, and answers are assembled from graph records rather than generated: a language model only rephrases an already-assembled answer, and a verification step discards any rephrasing that loses a fact. The agent is delivered as an authenticated FastAPI/React platform (conversation persistence, voice input, attachment Q&A, real-time sharing, analytics dashboard), deployed and publicly reachable.

**Keywords:** Knowledge Graph; Grounded Generation; Retrieval-Augmented Generation; Delegation of Authority; Governance and Compliance; Document Information Extraction; TF-IDF; Question Answering; Hallucination Mitigation; African Development Bank.

---

# Résumé

---

La Matrice de Délégation d'Autorité (DAM) de la Banque africaine de développement précise, pour chaque activité opérationnelle, les rôles habilités à l'approuver, la réviser, la vérifier ou l'initier, ainsi que ceux devant en être informés. Consultée sous forme de PDF, page par page, elle se prête à l'erreur : l'autorité est codée sur dix-sept niveaux couvrant cinq obligations distinctes, et des activités presque identiques peuvent relever d'approbateurs différents selon un seuil monétaire. Une erreur de lecture relève donc de la gouvernance, non d'un simple désagrément, une approbation hors délégation pouvant paraître parfaitement légitime à l'ensemble des intervenants.

Ce travail livre un agent répondant aux questions d'autorité sur la DAM sans rien inventer. Une chaîne déterministe en huit étapes extrait la matrice en un graphe de 327 nœuds d'activité, 1 776 responsabilités, 131 rôles et 2 108 arêtes, avec un taux de rôles non résolus de 0,6 %. Les questions sont résolues sur ce graphe, par identifiant ou recherche lexicale TF-IDF avec correction orthographique déterministe, et les réponses sont assemblées à partir du graphe plutôt que générées : le modèle de langue reformule seulement une réponse déjà assemblée, et une vérification écarte toute reformulation ayant omis un fait. L'agent est livré comme plateforme authentifiée FastAPI/React (persistance des conversations, saisie vocale, pièces jointes, partage en temps réel, tableau de bord analytique), déployée et accessible publiquement.

**Mots-clés :** Graphe de connaissances ; Génération ancrée ; Génération augmentée par récupération ; Délégation d'autorité ; Gouvernance et conformité ; Extraction documentaire ; TF-IDF ; Question-réponse ; Réduction des hallucinations ; Banque Africaine de Développement.

# CONTENTS

|                                                                           |            |
|---------------------------------------------------------------------------|------------|
| <b>Abstract</b>                                                           | <b>iii</b> |
| <b>List of Figures</b>                                                    | <b>x</b>   |
| <b>List of Tables</b>                                                     | <b>xii</b> |
| <b>List of Acronyms</b>                                                   | <b>xiv</b> |
| <b>General Introduction</b>                                               | <b>1</b>   |
| <b>1 General Context</b>                                                  | <b>3</b>   |
| Introduction . . . . .                                                    | 3          |
| 1.1 General Framework of the Project . . . . .                            | 3          |
| 1.2 Host Organisation . . . . .                                           | 4          |
| 1.2.1 Presentation of Polytech Intl . . . . .                             | 4          |
| 1.2.2 Project Context and the Client Organisation . . . . .               | 5          |
| 1.3 Study of the Existing System and Critical Analysis . . . . .          | 5          |
| 1.3.1 Context of Delegation of Authority in Development Banking . . . . . | 5          |
| 1.3.1.1 Definition and Stakes of the DAM . . . . .                        | 6          |
| 1.3.1.2 Governance and Compliance Requirements . . . . .                  | 7          |
| 1.3.1.3 Challenges in Consulting the DAM in Practice . . . . .            | 8          |
| 1.3.2 Study of Existing Systems . . . . .                                 | 9          |
| 1.3.2.1 Manual Consultation and Document Search . . . . .                 | 9          |
| 1.3.2.2 Generic Document Question-Answering Assistants . . . . .          | 9          |
| 1.3.2.3 Critical Analysis: Limitations of Current Approaches . . . . .    | 11         |
| 1.4 Functional Needs Analysis . . . . .                                   | 12         |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 1.4.1    | Functional requirements . . . . .                                     | 12        |
| 1.4.2    | Non-Functional Requirements . . . . .                                 | 14        |
| 1.4.3    | Project Constraints . . . . .                                         | 15        |
| 1.5      | Proposed Solution . . . . .                                           | 16        |
| 1.5.1    | General Presentation of the Solution . . . . .                        | 16        |
| 1.5.2    | Global System Architecture . . . . .                                  | 16        |
| 1.5.3    | Selected Technologies . . . . .                                       | 18        |
|          | Conclusion . . . . .                                                  | 19        |
| <b>2</b> | <b>Data Understanding: Analysis of the DAM Source Document</b>        | <b>20</b> |
|          | Introduction . . . . .                                                | 20        |
| 2.1      | Source Document Presentation . . . . .                                | 21        |
| 2.1.1    | Source and Context . . . . .                                          | 21        |
| 2.1.2    | Hierarchical Structure of the Document . . . . .                      | 21        |
| 2.1.3    | Table Layout and the Authority Notation . . . . .                     | 21        |
| 2.2      | Exploratory Analysis of the Document . . . . .                        | 22        |
| 2.2.1    | Global Overview and Node Distribution . . . . .                       | 22        |
| 2.2.2    | Authority Code Distribution . . . . .                                 | 23        |
| 2.2.3    | Role and Actor Analysis . . . . .                                     | 23        |
| 2.2.4    | Threshold Variants and Monetary Bands . . . . .                       | 23        |
| 2.2.5    | Cross-References and Redirects . . . . .                              | 24        |
| 2.2.6    | Consolidated View . . . . .                                           | 24        |
| 2.3      | Extraction Quality Assessment . . . . .                               | 24        |
| 2.3.1    | Text Layer Inspection . . . . .                                       | 24        |
| 2.3.2    | Rotated Column Headers . . . . .                                      | 25        |
| 2.3.3    | Fused Footnote Digits and Authority Codes . . . . .                   | 25        |
| 2.3.4    | Cross-Row Text Bleeding and Title Corruption . . . . .                | 26        |
| 2.3.5    | Split Rows in the Non-Sovereign Operations Chapter . . . . .          | 26        |
| 2.3.6    | Consolidated Data Issues Summary . . . . .                            | 27        |
|          | Conclusion . . . . .                                                  | 28        |
| <b>3</b> | <b>Data Engineering Pipeline and Knowledge Graph Construction</b>     | <b>29</b> |
|          | Introduction . . . . .                                                | 29        |
| 3.1      | Processing Environment . . . . .                                      | 29        |
| 3.1.1    | Motivation for a Deterministic, Rule-Based Pipeline . . . . .         | 30        |
| 3.1.2    | Why Not a Distributed Processing Framework . . . . .                  | 30        |
| 3.1.3    | Libraries Selected for Extraction, Validation and Modelling . . . . . | 30        |
| 3.2      | Pipeline Design . . . . .                                             | 31        |
| 3.2.1    | Global Architecture . . . . .                                         | 31        |
| 3.2.2    | Data Flow and Modular Organisation . . . . .                          | 33        |
| 3.3      | Data Ingestion . . . . .                                              | 33        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 3.4      | Document Processing and Normalisation . . . . .                | 33        |
| 3.4.1    | Row Reconstruction . . . . .                                   | 35        |
| 3.4.2    | Task Block Detection . . . . .                                 | 35        |
| 3.4.3    | Title and Reference Cleaning . . . . .                         | 35        |
| 3.4.4    | Geometric Attribution of Authority Codes . . . . .             | 35        |
| 3.4.5    | Role Attribution . . . . .                                     | 35        |
| 3.4.6    | Geometric Tolerances . . . . .                                 | 36        |
| 3.5      | Knowledge Graph Modelling . . . . .                            | 37        |
| 3.5.1    | Why a Graph . . . . .                                          | 38        |
| 3.5.2    | Node and Edge Schema . . . . .                                 | 38        |
| 3.5.3    | Graph Statistics . . . . .                                     | 39        |
| 3.6      | Pipeline Evaluation . . . . .                                  | 39        |
| 3.6.1    | Execution and Caching . . . . .                                | 40        |
| 3.6.2    | Data Quality Outcomes . . . . .                                | 40        |
| 3.6.3    | Observed Limitations . . . . .                                 | 40        |
|          | Conclusion . . . . .                                           | 41        |
| <b>4</b> | <b>Retrieval, Grounded Generation and Agent Design</b>         | <b>42</b> |
|          | Introduction . . . . .                                         | 42        |
| 4.1      | Retrieval Methodology . . . . .                                | 42        |
| 4.1.1    | Identifier Fast Path . . . . .                                 | 43        |
| 4.1.2    | Identifier Grammar and the Invalid-Identifier Branch . . . . . | 43        |
| 4.1.3    | Free-Text Resolution . . . . .                                 | 44        |
| 4.1.4    | Index Construction . . . . .                                   | 44        |
| 4.1.5    | Why Lexical Retrieval Rather Than Embeddings . . . . .         | 45        |
| 4.1.6    | Typo Tolerance . . . . .                                       | 46        |
| 4.1.7    | Correction Algorithm and Parameters . . . . .                  | 46        |
| 4.2      | Evaluation Protocol and Confidence Semantics . . . . .         | 47        |
| 4.2.1    | Resolution Thresholds . . . . .                                | 48        |
| 4.3      | Deterministic Answer Construction . . . . .                    | 49        |
| 4.3.1    | Intent Detection . . . . .                                     | 50        |
| 4.3.2    | Mandatory Obligation Surfacing . . . . .                       | 50        |
| 4.3.3    | Cross-Reference Following . . . . .                            | 50        |
| 4.3.4    | Honest Refusal . . . . .                                       | 50        |
| 4.4      | Auxiliary Query Types . . . . .                                | 51        |
| 4.5      | Hybrid Language Model Layer . . . . .                          | 51        |
| 4.5.1    | Model Configuration . . . . .                                  | 52        |
| 4.5.2    | Language Detection . . . . .                                   | 52        |
| 4.6      | Grounded Generation and the Verification Check . . . . .       | 53        |
| 4.6.1    | Contract Given to the Model . . . . .                          | 53        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 4.6.2    | Verification Step . . . . .                            | 53        |
| 4.6.3    | Answer Length and the Fallback Rate . . . . .          | 55        |
| 4.7      | Conversational Capabilities . . . . .                  | 55        |
| 4.7.1    | Context Carry-Over . . . . .                           | 55        |
| 4.7.2    | Tone Adaptation . . . . .                              | 56        |
| 4.7.3    | Multilingual Questions . . . . .                       | 56        |
| 4.8      | Worked Examples . . . . .                              | 56        |
|          | Conclusion . . . . .                                   | 57        |
| <b>5</b> | <b>Implementation Results and Delivered Platform</b>   | <b>59</b> |
|          | Introduction . . . . .                                 | 59        |
| 5.1      | Application Architecture . . . . .                     | 60        |
| 5.1.1    | Client–Server Structure . . . . .                      | 60        |
| 5.1.2    | Two Stores, Deliberately Separated . . . . .           | 61        |
| 5.1.3    | Choosing the Persistence Layer . . . . .               | 61        |
| 5.1.4    | Identifiers and Dialect Portability . . . . .          | 62        |
| 5.1.5    | Accounts Table . . . . .                               | 62        |
| 5.1.6    | Conversations Table . . . . .                          | 63        |
| 5.1.7    | Sharing Tables . . . . .                               | 65        |
| 5.1.8    | Referential Integrity and Deletion . . . . .           | 66        |
| 5.2      | Access Control . . . . .                               | 67        |
| 5.2.1    | Why Authentication Was Necessary . . . . .             | 67        |
| 5.2.2    | Password Storage . . . . .                             | 68        |
| 5.2.3    | Token Design . . . . .                                 | 68        |
| 5.2.4    | Federated Sign-In . . . . .                            | 69        |
| 5.2.5    | Resolving a Federated Identity to an Account . . . . . | 70        |
| 5.2.6    | Roles and Authorisation . . . . .                      | 71        |
| 5.2.7    | Hardening the Sign-In Path . . . . .                   | 71        |
| 5.3      | Conversation Interface . . . . .                       | 72        |
| 5.3.1    | Asking a Question . . . . .                            | 72        |
| 5.3.2    | Voice Input . . . . .                                  | 73        |
| 5.3.3    | Attachment Question Answering . . . . .                | 74        |
| 5.3.4    | Interface Localisation . . . . .                       | 75        |
| 5.4      | Conversation Persistence and Sharing . . . . .         | 75        |
| 5.4.1    | Conversations as Owned Objects . . . . .               | 75        |
| 5.4.2    | Granting Access to a Colleague . . . . .               | 75        |
| 5.4.3    | Link and QR-Code Sharing . . . . .                     | 76        |
| 5.4.4    | Real-Time Propagation . . . . .                        | 76        |
| 5.5      | Analytics Dashboard . . . . .                          | 77        |
| 5.5.1    | Purpose . . . . .                                      | 78        |

|       |                                                          |           |
|-------|----------------------------------------------------------|-----------|
| 5.5.2 | Computation Model . . . . .                              | 78        |
| 5.5.3 | How Indicators Were Chosen . . . . .                     | 79        |
| 5.5.4 | Correctness Indicators . . . . .                         | 80        |
| 5.5.5 | Structural Indicators . . . . .                          | 81        |
| 5.5.6 | An Indicator That Changed the Design . . . . .           | 81        |
| 5.5.7 | Interactive Cross-Filtering . . . . .                    | 82        |
| 5.5.8 | Governance Indicators Derivable from the Graph . . . . . | 82        |
| 5.6   | REST Interface . . . . .                                 | 84        |
| 5.7   | Testing and Validation . . . . .                         | 85        |
| 5.8   | Deployment . . . . .                                     | 86        |
|       | Conclusion . . . . .                                     | 87        |
|       | <b>General Conclusion</b>                                | <b>88</b> |
|       | <b>Bibliography</b>                                      | <b>91</b> |
|       | <b>Webography</b>                                        | <b>93</b> |

# LIST OF FIGURES

|     |                                                                                                                                                                                                                                                |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Example of a DAM page, showing the two stacked tables and the footnote band at the foot of the page. . . . .                                                                                                                                   | 6  |
| 1.2 | Authority-code legend, in which five codes expand into seventeen level-qualified notations. . . . .                                                                                                                                            | 7  |
| 1.3 | ChatPDF’s answers to two DAM authority questions, with page citations it generated automatically. The two exchanges are stacked rather than placed side by side so that the wording of each answer stays legible at normal page width. . . . . | 10 |
| 1.4 | Global architecture of the proposed system, from source document to user-facing answer. . . . .                                                                                                                                                | 17 |
| 2.1 | Column headers as printed, rotated through ninety degrees, beside the text an extractor returns for the same region. The header row is not recoverable as a table header. . . . .                                                              | 25 |
| 2.2 | An activity title as printed, and the corrupted string produced when text from an adjacent row bleeds into it during extraction. . . . .                                                                                                       | 26 |
| 2.3 | An authority code printed beside a superscript footnote marker, and the fused token produced when the size difference between them is disregarded. . . . .                                                                                     | 27 |
| 3.1 | Eight-stage data engineering pipeline, from raw PDF extraction to a validated node set. . . . .                                                                                                                                                | 34 |
| 3.2 | A DAM table region annotated with the geometric quantities used during extraction: rotated header clusters, the horizontal alignment band of one column, and a full-size authority code beside a reduced-size footnote digit. . . . .          | 38 |

|      |                                                                                                                                                                                                                                        |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.3  | Excerpt of the knowledge graph around activity 3.111, showing the containment edge from its parent process and the responsibility edges connecting it to its approving and informed roles. . . . .                                     | 39 |
| 4.1  | Resolution decision flow, from raw query to one of four outcomes: identifier resolution, invalid-identifier refusal, clarification, or a scored free-text match. . . . .                                                               | 44 |
| 4.2  | Top-ranked similarity score across a batch of free-text queries, with the answer floor, clarification ceiling, and near-tie suppression level marked. . . . .                                                                          | 49 |
| 4.3  | Grounded generation flow. The model may only rephrase facts that have already been verified, and any rephrasing that drops or alters a fact is discarded in favour of the deterministic template answer. . . . .                       | 54 |
| 5.1  | Accounts table in the deployed instance. Each account carries a bcrypt digest beginning \$2b\$12\$, and the federated identity columns are null for accounts registered with a password. . . . .                                       | 64 |
| 5.2  | Conversations table in the deployed instance. Each row carries a whole thread: the owning account, the title derived from the first question, and the JSON message column holding every exchange with its provenance metadata. . . . . | 64 |
| 5.3  | Sharing grants held in <code>conversation_shares</code> , showing one live grant. The row names the conversation, the account granted access, and the account that granted it. . . . .                                                 | 66 |
| 5.4  | Sign-in screen, offering federated sign-in through Google or Microsoft alongside the email-and-password form. . . . .                                                                                                                  | 67 |
| 5.5  | Resolving a federated identity to a local account. The provider-and-subject pair is checked first and email is consulted only as a fallback for linking, never for authentication. . . . .                                             | 70 |
| 5.6  | Conversation interface. The first answer refuses an under-specified question and offers candidates rather than guessing; the second resolves by identifier at confidence 1.0 and carries its provenance beneath the answer. . . . .    | 73 |
| 5.7  | Landing page, which states the grounding guarantee before any question is asked. . . . .                                                                                                                                               | 73 |
| 5.8  | Sharing dialogue. A conversation may be shared with a named colleague or through an expiring link rendered as a QR code, with the expiry date stated and revocation available. . . . .                                                 | 76 |
| 5.9  | Real-time propagation of a shared conversation. The Server-Sent Events stream delivers a new message within about a second; the thirty-second poll is a fallback that bounds staleness if the stream drops. . . . .                    | 77 |
| 5.10 | Analytics dashboard, showing headline indicators, the distribution of node types, and the roles ranked by the number of responsibilities they carry. . . . .                                                                           | 78 |

# LIST OF TABLES

|     |                                                                                 |    |
|-----|---------------------------------------------------------------------------------|----|
| 1.1 | Framework of the project. . . . .                                               | 4  |
| 1.2 | Client organisation at a glance. . . . .                                        | 5  |
| 1.3 | Two near-identical activities with different approvers. . . . .                 | 8  |
| 1.4 | Existing approaches assessed against the needs of this project. . . . .         | 11 |
| 1.5 | Functional requirements. . . . .                                                | 12 |
| 1.6 | Non-functional requirements. . . . .                                            | 14 |
| 1.7 | Project constraints. . . . .                                                    | 15 |
| 1.8 | Selected technologies and their role in the system. . . . .                     | 18 |
| 2.1 | Distribution of node types across the processed subset. . . . .                 | 22 |
| 2.2 | Consolidated summary of extraction issues found in the source document. . . . . | 27 |
| 3.1 | Stages of the data engineering pipeline, with the motivation for each. . . . .  | 31 |
| 3.2 | Data quality indicators after node construction. . . . .                        | 36 |
| 3.3 | Geometric tolerances governing extraction, in PDF user-space units. . . . .     | 36 |
| 3.4 | Edge types in the knowledge graph. . . . .                                      | 39 |
| 4.1 | Configuration of the TF-IDF retrieval index. . . . .                            | 45 |
| 4.2 | Parameters governing deterministic typo correction. . . . .                     | 46 |
| 4.3 | Resolution methods and their interpretation. . . . .                            | 47 |
| 4.4 | Thresholds governing the outcome of a free-text resolution. . . . .             | 48 |
| 4.5 | Language model providers and operating modes. . . . .                           | 51 |
| 4.6 | Models used by task, with the mode that selects each. . . . .                   | 52 |
| 5.1 | Accounts table users. . . . .                                                   | 62 |
| 5.2 | Conversations table conversations. . . . .                                      | 65 |

|     |                                                                                                                              |    |
|-----|------------------------------------------------------------------------------------------------------------------------------|----|
| 5.3 | Two sharing tables. . . . .                                                                                                  | 66 |
| 5.4 | Claims carried by the access token. . . . .                                                                                  | 68 |
| 5.5 | Authentication threats and the corresponding mitigation. . . . .                                                             | 71 |
| 5.6 | Indicators computed by the delivered analytics dashboard. . . . .                                                            | 79 |
| 5.7 | Governance indicators derivable from the knowledge graph, specified as the de-<br>signed extension to the dashboard. . . . . | 82 |
| 5.8 | REST API endpoints exposed by the backend. . . . .                                                                           | 84 |
| 5.9 | Breakdown of the automated test suite. . . . .                                                                               | 85 |

# LIST OF ACRONYMS

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <b>AfDB</b>     | African Development Bank Group                  |
| <b>AI</b>       | Artificial Intelligence                         |
| <b>API</b>      | Application Programming Interface               |
| <b>CRISP-DM</b> | Cross-Industry Standard Process for Data Mining |
| <b>CSV</b>      | Comma-Separated Values                          |
| <b>DAM</b>      | Delegation of Authority Matrix                  |
| <b>DDG</b>      | Deputy Director-General                         |
| <b>HTML</b>     | HyperText Markup Language                       |
| <b>JSON</b>     | JavaScript Object Notation                      |
| <b>JWT</b>      | JSON Web Token                                  |
| <b>KPI</b>      | Key Performance Indicator                       |
| <b>LLM</b>      | Large Language Model                            |
| <b>NSO</b>      | Non-Sovereign Operations                        |
| <b>PDF</b>      | Portable Document Format                        |
| <b>QR</b>       | Quick Response (code)                           |
| <b>RAG</b>      | Retrieval-Augmented Generation                  |
| <b>RDGN</b>     | Director General, North Africa Region (AfDB)    |
| <b>REST</b>     | Representational State Transfer                 |
| <b>S3</b>       | Simple Storage Service                          |
| <b>TF-IDF</b>   | Term Frequency – Inverse Document Frequency     |
| <b>URL</b>      | Uniform Resource Locator                        |
| <b>UTC</b>      | Coordinated Universal Time                      |
| <b>UUID</b>     | Universally Unique Identifier                   |

# GENERAL INTRODUCTION

Sooner or later, every large institution has to write down who is allowed to decide what. In a development bank, that question has teeth. An approval signed by the wrong person is a governance failure, and auditors find those.

The African Development Bank Group keeps its answer in a document called the Delegation of Authority Matrix, the DAM, which goes through the Bank's activities one by one and records who starts each task, who has to check it, who reviews it, who approves it, and who simply has to be told it happened.

The document is accurate. Getting at what it says is the problem. The full DAM runs past two hundred and fifty pages. Nearly all of it sits inside wide tables, and because those tables carry more columns than a page can hold, the role names at the top are printed sideways. Authority is written in a shorthand of single letters, and a small digit beside a letter can change what it means. So a staff member who wants to know whether they can sign off on a mission program has to find the right row, in the right table, in the right chapter, follow a rotated header down to work out which column belongs to which role, and then decode the character where the two meet. People do manage this. They manage it slowly and not always correctly.

None of the bank's priorities get delivered unless decisions are made correctly by people entitled to make them. Take RDGN, the regional office covering Tunisia, Algeria, Morocco, Mauritania, and Libya. Staff there spend their days on exactly the activities the Matrix governs: Organizing missions, dealing with co-financiers, and committing funds against country programs. Each needs an answer to the same four questions. who starts it, who checks it, who approves it, who needs to know. Delegation of authority is not paperwork sitting alongside the real work. It is how the real work gets done, and anything that makes it harder to consult slows the Bank down.

The obvious thing is to try is a chatbot. It does not work here, and the reason is worth being pre-

cise aboput. Language models write fluently whether or not they are right, and fluent text about who controls financial approval is dangerous when it is wrong. A model that invents a plausible-sounding role, or quietly leaves out one of the three people who should have been informed, hands you something that reads exactly like correct answer. For a compliance document, that is worse than no answer.

So the system described here is built around that problem rather than hoping to avoid it. The DAM is parsed into a structured knowledge base. That knowledge base becomes a graph of activities and the roles responsible for them, and questions are answered by pulling real records out of the graph. There is a language model involved, but it never decides what the facts are, It is handed facts that have already been retrieved and asked to put them into a sentence, and before anyone sees that sentence it is checked, word by word, against the facts it came from. If a role name. If a role name is missing or changed, the sentence is thrown away and a plain one is shown instead.

The report runs to five chapters:

**Chapter 1** sets the scene: the host instituion, the client, how the Matrix is consulted today, where that breaks down, what existing tools offer, and the requirements and architecture that follow from all of it

**Chapter 2** looks hard at the source document. there is no tidy dataset here, only a PDF whose structure lives in its layout, so this chapter works through how that layout carries meaning and where automated extraction goes wrong on it.

**Chapter 3** is the pipeline that turns the document into data, stage by stage, along with the checks applied at each one and the knowledge graph built on the output

**Chapter 4** covers the agent: how questions finds the right activity, how an answer is assembled, what language model allowed to do, and the check that keeps it honest

**Chapter 5** presents what was actually delivered, including the features added in the project's second phase, the analytics dashboard, the testing behind it, and how it is deployed. The general conclusion then takes stock of what worked, what did not, and what should be tackled next

# CHAPTER

# 1

## GENERAL CONTEXT

### Introduction

This Chapter sets out where the project came from and what it was asked to solve. It introduces the institution that hosted the work and the client whose document sits at the center of it, then looks closely at how the document is used today. That second part carries more weight than it might appear: The requirements at the end of this chapter are not invented, they are what is left once you understand exactly how the current process fails. The chapter closes with the tools that already exist for this kind of problem, why none of them fit, and the solution proposed in response.

### 1.1 General Framework of the Project

This was a Final Year Project (FYP) inside the master's degree program in Data Sciences at Polytech Intl, carried out over a six months internship. Roughly four of those months went into building the system. The rest went on framing the subject, learning the source material well enough to be useful, and writing the report.

What kept me on this was how ordinary the document looks and how badly it breaks automated tools. A person reads a page of the DAM without difficulty. Every library I pointed at the same page came back with something plausible and wrong, and finding out why meant learning to read the file the way the printer wrote it rather than the way a reader sees it.

Table 1.1 summarises the arrangement.

Table 1.1: Framework of the project.

| Item                     | Detail                                         |
|--------------------------|------------------------------------------------|
| Project type             | End-of-studies project                         |
| Host institution         | Polytech Intl, Tunis                           |
| Programme                | Professional Master's in Data Science          |
| Academic year            | 2025 – 2026                                    |
| Duration                 | Six months                                     |
| Client organisation      | African Development Bank Group                 |
| Academic supervisor      | Dr. Marouene Chaieb                            |
| Professional supervisors | Mr. Yasser Ahmad, Mr. Surya Joymungul          |
| Source document          | Delegation of Authority Matrix, over 250 pages |
| Scope processed          | 79-page subset covering three chapters         |

## 1.2 Host Organisation

Two institutions were involved in this project, with different parts to play. Polytech Intl provided the academic framework: the supervision, and the requirements the work had to satisfy as a degree project. The African Development Bank provided the document at the centre of the work, along with the operational judgement of whether an answer the system produced would be trusted by someone who had to act on it. A university could not have supplied the second of those, and a bank could not have supplied the first. The subsections below introduce each in turn.

### 1.2.1 Presentation of Polytech Intl

Polytech Intl [15], the École Polytechnique Internationale Privée de Tunis, is a private higher education institution founded in 2013 and based at Rue du Lac d'Annecy in Les Berges du Lac 1, Tunis. It was set up largely by institutional figures from the professional world, principally from banking, insurance and industry, which is visible in how the school approaches training: the intent is that graduates leave employable, and ideally capable of building something of their own.

Teaching covers engineering and related disciplines. Programmes run across computer science, with tracks in big data, business intelligence, software engineering, IT-finance and embedded systems, alongside mechatronics, industrial engineering, financial engineering, telecommunications and architecture. Students are also offered international mobility, with internships and teaching modules arranged in partnership with schools in Europe, Asia and North America. The school is a member of the Agence Universitaire de la Francophonie.

The Data Sciences Master's spans statistics and machine learning, data engineering, and the software side of turning an analysis into something that runs and keeps running. A final project is meant to show all three. That expectation is part of why this one was pushed through to a deployed application instead of stopping at a clean dataset.

## 1.2.2 Project Context and the Client Organisation

Polytech Intl hosted and supervised the project. The subject came from elsewhere. The document at the centre of it belongs to the African Development Bank Group [16], which acted as the client. There is a certain symmetry in that: a school founded in part by people from banking, supervising a project built for a bank.

The African Development Bank Group was founded in 1964 [16] and has its headquarters in Abidjan, Côte d'Ivoire. Eighty-one countries are members, 54 of them African and 27 from outside the continent, and the Bank exists to support economic development and social progress across its regional member countries. Table 1.2 sets out the essentials.

Table 1.2: Client organisation at a glance.

| Item                      | Detail                                                                    |
|---------------------------|---------------------------------------------------------------------------|
| Institution               | African Development Bank Group                                            |
| Founded                   | 1964                                                                      |
| Headquarters              | Abidjan, Côte d'Ivoire                                                    |
| Membership                | 81 countries: 54 regional, 27 non-regional                                |
| Purpose                   | Economic development and social progress across regional member countries |
| Group entities            | African Development Bank, African Development Fund, Nigeria Trust Fund    |
| Regional office concerned | RDGN: Tunisia, Algeria, Morocco, Mauritania, Libya                        |
| Regional hub              | Tunis                                                                     |

Working from a real institution's document changed how the project had to be done. The material could not be tidied up to suit the parser. Correctness meant whatever the printed pages say, not a score that happened to be easy to compute. And every claim in this report about what the system pulls out of the document can be checked against the page it came from. That constraint runs through the whole of Chapter 2.

## 1.3 Study of the Existing System and Critical Analysis

The point of this section is to stress that one cannot judge the adequacy of the tools we have until we know what adequacy means in this instance. Section 1.3.1 details the requirements the document places on its users, while Section 1.3.2 compares the resources currently available against those requirements. This suggests there is an argument in this section, not just content.

### 1.3.1 Context of Delegation of Authority in Development Banking

Before the document's shortcomings can be assessed, it has to be described on its own terms: what it is for, how its notation works, and what happens when that notation is misread.

### 1.3.1.1 Definition and Stakes of the DAM

The Delegation of Authority Matrix (DAM) is an internal Bank document [14] that sets out who is responsible for taking specific actions on identified tasks. The Task Managers of the African Development Bank depend on it to delegate the authority of the President down to the appropriate level for a decision to be made. Without it, every decision would require the President's approval.

A DAM page consists of two tables that fill the page, as shown in Figure 1.1. The bottom of each page is reserved for footnotes. A brief reminder of what the letters in the authority codes mean is printed in the upper right, but it only goes so far: it explains the letters and not the numbers that qualify them. The main table carries the process, the sub-task, and the roles with their assigned authority codes.

**CSP-RISP 1.100: PROGRAMMATIC ENGAGEMENT WITH COUNTRIES & RECS**

| DAM Code No. |                                                                                                               | CSP-RISP 1.110                                                         |   | SNVP/RDVP/AHVP/PEVP/PIVP/FIVP/ECVP |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            | Authority Codes            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
|--------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|---|------------------------------------|-----------------------------------------------|-------------------------------------------------|------------------------------|---------------------------------------------|-----------------|------------------------------------|------------------------------------|-----|----------------------------|----------------------------|-------------------------------------------------------|--------------------|------|-----------------|--------|------|--|---------------------------------|------------------------------|------------------------------|
| Process:     |                                                                                                               | PROGRAMMING FOR COUNTRY & REGIONAL INTEGRATION OPERATIONS              |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            | I                          | Initiate (or Launch) / Originate (or Prepare)         |                    |      |                 |        |      |  |                                 |                              |                              |
|              |                                                                                                               |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            | C                          | Check and Verify / Check and Clear                    |                    |      |                 |        |      |  |                                 |                              |                              |
|              |                                                                                                               |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            | R                          | Review and Recommend / Review and Clear               |                    |      |                 |        |      |  |                                 |                              |                              |
|              |                                                                                                               |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            | A                          | Endorse (A <sub>1</sub> ) / Approve (A <sub>2</sub> ) |                    |      |                 |        |      |  |                                 |                              |                              |
| Date:        |                                                                                                               | December 2022                                                          |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            | (i)                        | To Be Informed                                        |                    |      |                 |        |      |  |                                 |                              |                              |
| Activity No. |                                                                                                               |                                                                        |   | Country Economist <sup>1</sup>     | Regional Integration Coordinator <sup>2</sup> | Country / Regional Program Officer <sup>3</sup> | CSP / RISP Team <sup>4</sup> | Regional Portfolio & Implementation Manager | Sector Managers | Lead Economist (ECCE) <sup>5</sup> | Country Manager / DDG <sup>6</sup> | RDG | Director RDRI <sup>7</sup> | Director ECCE <sup>8</sup> | TQAC <sup>9</sup>                                     | PSRC <sup>10</sup> | RDVP | ECVP            | OPSCOM | PRST |  | CODE                            | BOARD                        |                              |
| 1.110        | Country Strategy Papers (CSP), Regional Integration Strategy Papers (RISP), and other programming documents * |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            |                            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
| 1.114        | New Strategy Paper                                                                                            |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            |                            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
| 1.114.1      | Preparation, review and approval of New CSP (with a 3-Year Rolling Business Plan)                             | I                                                                      |   |                                    |                                               |                                                 | (i)                          | R <sup>4</sup>                              | C               | C                                  | R <sup>12</sup>                    |     | C <sup>8</sup>             |                            | R <sup>12</sup>                                       | (i)                |      | R <sup>14</sup> |        |      |  | A <sub>1</sub> <sup>14</sup>    | A <sub>2</sub> <sup>14</sup> |                              |
| 1.114.2      | Preparation, review and approval of New RISP (with a 3-Year Rolling Business Plan)                            |                                                                        | I |                                    |                                               |                                                 | (i)                          | R <sup>4</sup>                              | C               | (i)                                | R <sup>12</sup>                    | C   | C <sup>8</sup>             |                            | R <sup>12</sup>                                       | (i)                |      | R <sup>14</sup> |        |      |  | A <sub>1</sub> <sup>14</sup>    | A <sub>2</sub> <sup>14</sup> |                              |
| 1.115        | Other Programming Documents <sup>15</sup>                                                                     |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            |                            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
| 1.115.1      | Interim CSP/RISP; Country Brief; or JCAS                                                                      | Follow the respective process for new CSP/RISP (See 1.114.1 / 1.114.2) |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            |                            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
| 1.115.2      | Updated CSP/RISP ("Extension")                                                                                | Follow the respective process for new CSP/RISP (See 1.114.1 / 1.114.2) |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            |                            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
| 1.116        | Mid-Term Review                                                                                               |                                                                        |   |                                    |                                               |                                                 |                              |                                             |                 |                                    |                                    |     |                            |                            |                                                       |                    |      |                 |        |      |  |                                 |                              |                              |
| 1.116.1      | Preparation, review and approval of Mid-Term Review of CSP                                                    | I                                                                      |   |                                    |                                               |                                                 | (i)                          | R <sup>4</sup>                              | C               | C                                  | R <sup>12</sup>                    |     | C <sup>8</sup>             |                            | R <sup>12/16/17</sup>                                 | (i)                |      | R <sup>17</sup> |        |      |  | A <sub>1</sub> <sup>16/17</sup> | (i) <sup>16</sup>            | A <sub>2</sub> <sup>17</sup> |
| 1.116.2      | Preparation, review and approval of Mid-Term Review of RISP                                                   |                                                                        | I |                                    |                                               |                                                 | (i)                          | R <sup>4</sup>                              | C               | (i)                                | R <sup>12</sup>                    | C   | C <sup>8</sup>             |                            | R <sup>12/16/17</sup>                                 | (i)                |      | R <sup>17</sup> |        |      |  | A <sub>1</sub> <sup>16/17</sup> | (i) <sup>16</sup>            | A <sub>2</sub> <sup>17</sup> |

Figure 1.1: Example of a DAM page, showing the two stacked tables and the footnote band at the foot of the page.

There are five authority codes but seventeen distinct notations, because most codes take a level, and the letter alone is therefore not the answer. Take C: on its own it denotes Check and Verify, and C1 and C2 are its two levels:

- **C1** — examine the activity against a checklist of standards and preconditions, and allow it to proceed only if all of them are satisfied.
- **C2** — clear the activity as compliant with policy and regulation, after consulting the relevant departments.

Both are blocking obligations: the holder of a C1 or C2 code can halt the activity.

C3 and C4, however, are not levels of checking at all. They denote Consult, an unrelated obligation that happens to share the letter:

- **C3** — obtain the opinion of an independent function (IDEV, BCRM) or of a knowledgeable expert.
- **C4** — the Board of Directors or the Board of Governors must be consulted, while the President or Management still holds the approval.

These are advisory obligations: the holder of a C3 or C4 code contributes an opinion but cannot halt the activity on that basis alone. The full notation, reproduced in Figure 1.2, makes the extent of the ambiguity visible: a reader who resolves the letter without resolving its level can invert a blocking obligation into an advisory one, which is the single most consequential misreading the document permits.

### STANDARD AUTHORITY CODES

The following Standard Authority Codes are used in the 2023 DAM:

| Authority Level                   | What the authority level involves: Work to be done + Responsibilities + Accountability                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I</b> Initiate / Originate     | I <sub>1</sub> : To launch a process or activity; and/or<br>I <sub>2</sub> : To prepare a relevant, distinct output of that process or activity; and/or<br>I <sub>3</sub> : To participate in a multi-disciplinary / interdepartmental Task Team collectively responsible for a process or activity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>C</b> Check and Verify         | C <sub>1</sub> : To examine an activity against a formal checklist of quality standards, requirements, or preconditions and permit the activity to proceed to the next step only if all the set standards, requirements, or preconditions have been satisfied; or<br>C <sub>2</sub> : To clear the action or decision examined to proceed to the next step since it complies with applicable policies and regulations following appropriate consultation with relevant departments (e.g., PGCL, SNFI, etc.).                                                                                                                                                                                                                                                                          |
| <b>C</b> Consult                  | C <sub>3</sub> : In the case of the Bank's Independent Functions, notably Independent Development Evaluation, and Independent Compliance Review and Mediation, to tap the opinions, perceptions, or assessments of a responsible official, or knowledgeable, well-informed expert. (This is distinct from "Check and Verify", above).<br>C <sub>4</sub> : In the case there is a requirement to consult the Board of Directors or the Board of Governors, while the President or Management has the authority to approve.                                                                                                                                                                                                                                                             |
| <b>R</b> Review and Recommend     | R <sub>1</sub> : To examine a submitted activity or output critically and assess its validity, value, or contribution from a number of well-defined perspectives (including compliance with the letter and spirit of adopted policies and regulations, alignment with established strategies and priorities, etc.) and provide concrete suggestions for alternative approaches aimed at specific improvements, or to guide further steps; or<br>R <sub>2</sub> : To examine a submitted proposal critically and determine whether it meets essential quality standards and requirements and, if it does so, clear it to be submitted to a higher authority, accompanied by a statement advocating for the proposal and/or identifying specific issues that need guidance or decision. |
| <b>A</b> Endorse / Approve / Sign | A <sub>1</sub> : To validate an activity, or accept an output, or endorse a proposed decision submitted for consideration; or<br>A <sub>2</sub> : To provide the final authority to proceed to implement a decision—anchored on certitude of adequate due diligence having been carried out by responsible officials to ensure appropriate quality standards, adherence to policies, safeguards, laws and regulations, alignment to strategies and priorities, and good risk management; or<br>A <sub>3</sub> : To append one's signature on a document to signify knowledge, agreement, approval, or acceptance on behalf of the Bank Group.                                                                                                                                         |
| <b>(i)</b> To be Informed         | ▪ Having the right or privilege to be provided information on an activity or proposal that is under review; and/or<br>▪ Having the obligation to find and process information on an activity or proposal that is under review.<br>▪ No formal responsibility to act on the information in this activity but not excluded from exercising influence.                                                                                                                                                                                                                                                                                                                                                                                                                                   |

xxiv

Figure 1.2: Authority-code legend, in which five codes expand into seventeen level-qualified notations.

#### 1.3.1.2 Governance and Compliance Requirements

The DAM is not a manual of procedures. It tells who can do something, but not how to do it. Clearly erroneous approvals would be likely to be picked up in normal organization reviews, as the individual consulted would know that it was beyond their authority. However, the DAM structure creates situations where an erroneous approval appears to be fully justified: two almost identical activities may have totally different approvers, and the person who is requested to approve is still a valid approver for this activity category as a whole — they have no way of knowing, from their position, that this particular case is assigned to someone else. This section covers the requirements that the document places on persons who wish to. This section addresses conditions for the proper interpretation of the document without any consideration of hierarchy.

Approximation is allowed to a certain degree in most reference materials. The worst thing that can happen if an employee gets the meeting time wrong is that they miss the meeting and are able to catch up with the minutes later. This is how the DAM does not work. When a staff member misunderstands it, or misses an important role, in the course of carrying out a task, that task moves forward without the required authorisation. There is no immediate rejection or alert. The outcome is not that the reader is left without an answer, but that the reader proceeds from a self-assured but partial answer, with no hint of the opposite.

A response can be perfectly accurate and still be misunderstood. The different tables in the DAM answer questions like “who approves X”, but by only naming the approver it conceals the need to consult others, as approval and verification are often interdependent stages of the same task – something FR4 aims to avoid. This is not an uncommon situation. Of the 216 activities that have an identified approver, 81 have no documented check or validation step. The mismatch was discovered during a trial meeting with the client – while demonstrating the agent by asking who authorises X, Yasser noticed that the response also had no check/verify function. His reasoning was that an experienced employee can easily see the connection between an approver and the person who verifies it, but a new employee may not automatically see this relationship and could easily miss it. That query showed the discrepancy, and hence the system is now giving a documented check/verification step irrespective of the initial query asked.

Finally, the DAM file can be hard to understand for new staff without proper guidance. The staff member may miss crucial information while reading the document.

### 1.3.1.3 Challenges in Consulting the DAM in Practice

One of the few weaknesses of Consulting the DAM in Practice is that it takes time just to find the file in the local environment among all the other documents, and once it’s found, the user still needs to scroll all the way down to the specific task that he/she needs. In some cases, even after finishing it, the user can find some difficulties understanding the instruction provided by the DAM tables clearly. This causes staff to call someone more experienced to explain it.

Table 1.3 shows the problem in its sharpest form: two activities whose titles differ only in a monetary band, sitting on consecutive pages, carrying different approvers. A reader who matches on the wording alone and does not check the threshold will name the wrong approver while believing the match was exact.

Table 1.3: Two near-identical activities with different approvers.

| ID        | Page | Title                                   | Approver       |
|-----------|------|-----------------------------------------|----------------|
| 2.432.4.a | 43   | Individual Consultant: $\leq$ UA 50,000 | Sector Manager |
| 2.433.4.c | 44   | Individual Consultant: $>$ UA 250,000   | PRC            |

## 1.3.2 Study of Existing Systems

With the DAM's demands on readers now clear, the natural question is whether an existing tool already meets them. We can examine two categories of tools against these weaknesses to see whether any address what the DAM lacks. The first method involves consulting the DAM and performing a manual search. The second, however, is a tool that requires using a generic question-answering assistant designed to analyze the submitted file and answer the user's question. The study of both of these systems makes it possible to better understand the needs and expectations regarding the outcome required from the DAM AI Agent.

### 1.3.2.1 Manual Consultation and Document Search

The current process is manual consultation of the PDF, supported by the basic search the PDF Viewer tool provides (Acrobat, Foxit,...). The Keyword search finds a character string, which helps when the staff already knows the exact title of an activity or its identifier, and helps very little otherwise. Searching for the word "approve" will return either the explanatory authority code table on the first few pages or every table in the DAM. Searching for a partially remembered activity name returns nothing at all unless the remembered words match the printed ones exactly.

More importantly, text search returns locations not answers. It can tell the staff that a term exist in page X. It cannot tell them who approves the activity described there since that requires understanding that the pages holds a table and that the tables in the DAM have a complexe visual design and that the same action can change depending on the authority level and the footnote attached to it.

### 1.3.2.2 Generic Document Question-Answering Assistants

A second category of existing solution is the General purpose document assistant tools such as ChatPDF [17], NotebookLM [18] or Microsoft Copilot's file upload mode [19]. These tools typically function by splitting the document into passages, embed those passages into vectors, retrieve the passage similar to the question and then pass them to a language model that writes an answer. This is standard retrieval-augmented generation, or RAG [3].

This approach performs well on the conventional text heavy documents but breaks on this document for a reason that the DAM is structural rather than literary. When a page of the DAM is converted to plain text, the spatial relationship that carry its meaning are destroyed. The rotated header that identified a column becomes a stray string lost in the extracted corpus, disconnected from the footnotes beneath it. A retrieved passage may therefore contain the activity name and the authority code letter without preserving the information that the authority code belonged to a specific cell and has a specific footnote or authority level. in that case the model is left working from a text that has already lost the answer, and produces a confident answer either ways.

(a) “Who approves loans?”

who is responsible of initiating missions?



The responsibility for initiating missions varies depending on whether the operation is sovereign or non-sovereign:

- **For sovereign operations**, the task of initiating missions is typically carried out by the Task Manager or Program Officer, in coordination with the concerned departments and stakeholders, as outlined in PSO 2.120 (Page 725). The process involves the Task Manager or Program Officer initiating the mission and coordinating with the relevant sector or country teams 725.
- **For non-sovereign operations**, the initiation of missions is similarly the responsibility of the Task Manager or Origination Sector Manager, often based in HQ or regional offices, as described in NSO 3.110 (Page 759). The activity incorporates the initiation by Task Managers assigned to the operation 759.

#### In summary:

- **Initiation of missions is primarily carried out by Task Managers or Program Officers** in both sovereign and non-sovereign contexts, who are responsible for the origination, coordination, and initial planning of the missions.

#### References:

- **Section PSO 2.120 (Page 725)**: Details on the organization of Missions for sovereign operations.

(b) “Who is responsible for initiating missions?”

Figure 1.3: ChatPDF’s answers to two DAM authority questions, with page citations it generated automatically. The two exchanges are stacked rather than placed side by side so that the wording of each answer stays legible at normal page width.

To test this directly, two pages of the DAM were uploaded to ChatPDF and asked "who approves loans?" and "who is responsible for initiating missions?" (Figure 1.3). Both answers were fluent, well-structured, and included automatically generated page citations. Checked against this project's own extracted data, however, neither holds up at the level of detail the DAM actually records. The DAM lists the President as an approver in exactly one context — a staff-seniority escalation ladder for mission-related sign-off — not as a general final approver for loans. The specific page ChatPDF cited for loan approval in fact records a different and more complex set of roles, one of which could not even be confidently resolved to a name during this project's own extraction. The mission-initiation answer merges several distinct activities sharing a common process code into a single summary, silently dropping at least one case with a different initiator entirely. Both answers illustrate the mechanism described above: the tool produces something fluent and well-organised from a page whose spatial structure it has already discarded, and the fluency of the result gives no indication of where it has quietly generalised past what the actual row records.

### 1.3.2.3 Critical Analysis: Limitations of Current Approaches

Both approaches ultimately encounter a fundamental challenge: they lack an awareness of error. This is the failure mode surveyed across generative systems as hallucination[4], and it is particularly consequential here, because a confidently phrased wrong approver is indistinguishable to the reader from a correct one. When a search engine produces results or a language model generates text, neither system differentiates between a supported answer and one that lacks justification. This distinction is crucial for compliance purposes.

Table 1.4 assesses both categories against the needs established earlier in this section, alongside the approach taken in this work.

Table 1.4: Existing approaches assessed against the needs of this project.

| Requirement                           | Search | Assistants | This work |
|---------------------------------------|--------|------------|-----------|
| Preserves positional table meaning    | No     | No         | Yes       |
| Answers a question, not a location    | No     | Yes        | Yes       |
| Answer traceable to a source row      | No     | No         | Yes       |
| Refuses when it cannot answer         | N/A    | No         | Yes       |
| Surfaces mandatory obligations        | No     | No         | Yes       |
| Tolerates typos and loose phrasing    | No     | Yes        | Yes       |
| Document stays inside the institution | Yes    | No         | Yes       |
| No per-page or licence cost           | Yes    | Partly     | Yes       |
| Deployable within this project        | Yes    | Yes        | Yes       |

The table above presents a comparative analysis of the two existing systems for reading the DAM document. Three major differences between the systems are evident. First, the question-

answering assistant can tolerate typos within the document, whereas the manual search method does not account for such errors. Second, the question-answering assistant differs from the manual search method by its ability to answer user questions regarding the DAM document, a capability not available with the manual search method. Third and finally, the manual search method differs from the question-answering assistant in terms of security. Submitting an internal, confidential document of this magnitude to a generic question-answering assistant, one not created or licensed by the AFDB, constitutes a security breach. Consequently, a manual search within the document can be viewed as a less efficient but more secure solution, as it relies on no external tools that could compromise the security and confidentiality of the DAM document.

Thus, a conclusion can be drawn from this comparative analysis of the two previously used methods. The DAM AI Agent must, first, be capable of functioning even in the presence of typos. Next, it must accurately and precisely answer the various questions a user might ask. Finally, the DAM AI Agent must comply with AFDB security standards, and the document submitted to it must remain confidential. This last point will be explained further in the Authentication section of Chapter 5.

## 1.4 Functional Needs Analysis

The gaps identified above translate directly into requirements: each one traces back to a specific limitation of manual search or generic assistants documented in the preceding section.

### 1.4.1 Functional requirements

The functional requirements are categorised into two groups. Requirements FR1 to FR10 pertain to the fundamental question-answering capability and derive from current document consultation practices. Requirements FR11 to FR15 address the platform capabilities introduced in the project's second phase, which transform the system from a lookup tool into an application usable inside an institution. Table 1.5 describes each of them.

Every requirement listed satisfies two conditions. It corresponds to a need observed in the way the document is consulted, rather than to a capability that merely seemed attractive to build; and it is met by the delivered system. Requirements that failed either condition were removed rather than retained as statements of intent, since a requirements table describing intentions cannot be used to evaluate what was actually produced.

Table 1.5: Functional requirements.

| Ref. | Requirement              | Description                                                                                                                                 |
|------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| FR1  | Direct identifier lookup | Given a valid activity identifier, the system returns that activity's recorded authority information with full confidence and no ambiguity. |

| Ref. | Requirement                    | Description                                                                                                                                                                          |
|------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR2  | Natural-language lookup        | Given a question describing an activity by name rather than by identifier, the system resolves it to the correct activity and reports how confident that resolution is.              |
| FR3  | Authority-role queries         | For a resolved activity, the system answers who approves, reviews, checks, initiates, or must be informed, using the document's own notation.                                        |
| FR4  | Mandatory obligation surfacing | Whatever the question asked, the system always surfaces a recorded check-and-verify obligation and, where present, the informed parties.                                             |
| FR5  | Cross-reference resolution     | Where an activity's row is a pointer to another activity, the system follows the pointer automatically rather than reporting that no responsibilities exist.                         |
| FR6  | Typing and phrasing tolerance  | The system tolerates common misspellings and phrasing variation without requiring the document's exact vocabulary.                                                                   |
| FR7  | Honest refusal                 | For an identifier that does not exist, or a question outside the document's subject matter, the system says so explicitly instead of returning a best guess.                         |
| FR8  | Conversational continuity      | A follow-up question that does not restate its subject is answered against the activity discussed immediately before.                                                                |
| FR9  | Structural analytics           | The system exposes an overview of the document's own structure: node counts, authority-code distribution, roles most frequently involved, and per-chapter breakdowns.                |
| FR10 | Multilingual questions         | Questions asked in French, Spanish, Portuguese, or Arabic are answered in the language they were asked in, without translating role names or identifiers.                            |
| FR11 | Authenticated access           | Access to the application requires authentication, and analytical features are restricted to authorised roles.                                                                       |
| FR12 | Voice input                    | A user can dictate a question instead of typing it, in any of the supported interface languages.                                                                                     |
| FR13 | Document attachment            | A user can attach a supporting document to a conversation and ask questions about its content, with answers from that source clearly distinguished from answers grounded in the DAM. |

| Ref. | Requirement                      | Description                                                                                                                                                                                                                  |
|------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR14 | Localised conversation interface | The conversation interface, including its controls and its answer provenance labels, is available in each supported language, with right-to-left layout applied for Arabic.                                                  |
| FR15 | Conversation sharing             | A user can grant a colleague read-only access to a conversation, either by naming them directly or through an expiring link encoded as a QR code, with shared views reflecting new messages without requiring a page reload. |

### 1.4.2 Non-Functional Requirements

Where the functional requirements state what the system must do, the non-functional requirements state the qualities it must exhibit while doing it. They are not secondary. NFR1 in particular, which forbids any answer from naming a role or code the source document does not record for that activity, takes precedence over fluency, latency, and every other consideration in the table: a system of this kind that is pleasant to use and occasionally wrong is worse than no system at all, because it transfers confidence without transferring correctness. The remaining requirements span traceability, availability, reproducibility, portability, and the handling of credentials and uploaded files. Table 1.6 details each one.

Table 1.6: Non-functional requirements.

| Ref. | Requirement                            | Description                                                                                                                                                                                               |
|------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR1 | Factual fidelity                       | No answer may contain a role, code, or qualification that is not recorded in the source document for that activity. This requirement takes precedence over fluency, speed, and every other consideration. |
| NFR2 | Traceability                           | Every answer must be attributable to a specific activity record so that a user can verify it against the printed document.                                                                                |
| NFR3 | Availability without external services | Core lookup must work when no language model provider is reachable, degrading to template answers rather than failing.                                                                                    |
| NFR4 | Response time                          | Answers should return within a few seconds on the free hosting tier used for deployment, which rules out re-parsing the document per request.                                                             |

| Ref. | Requirement             | Description                                                                                                                                |
|------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| NFR5 | Reproducibility         | The same question must produce the same underlying facts on every run, independent of any model's sampling behaviour.                      |
| NFR6 | Portability             | The system must run identically on a developer machine and on the hosting platform, without environment-specific configuration.            |
| NFR7 | Usability on any device | The interface must remain usable on a phone, since consultation often happens away from a desk.                                            |
| NFR8 | Credential protection   | Authentication credentials must never be stored in recoverable form, and session tokens must expire.                                       |
| NFR9 | Upload safety           | Attached documents must be constrained by type and size, and must never be executed or interpreted as anything other than text to be read. |

### 1.4.3 Project Constraints

The conditions below were fixed before any design decision was taken, and each one closed off an approach that would otherwise have been reasonable. They are recorded here because several choices defended later in this report, particularly the preference for lexical retrieval over learned embeddings and the decision to cache the parsed knowledge base at build time, follow directly from them rather than from a free comparison of alternatives. Table 1.7 sets out each constraint and what it ruled out.

Table 1.7: Project constraints.

| Constraint                | Consequence for the project                                                                                                                                                      |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No labelled training data | No query-and-answer pairs and no annotated tables existed, which ruled out training a supervised model and pushed retrieval toward lexical methods that need no training corpus. |
| Document scope            | The full DAM exceeds 250 pages. Work was scoped to a 79-page subset covering three chapters, with the remainder left as future work.                                             |
| No infrastructure budget  | Deployment had to fit within free hosting tiers, which constrained memory use and forced the parsed knowledge base to be cached at build time rather than rebuilt on startup.    |

| Constraint                      | Consequence for the project                                                                                                                                                      |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Verification against the source | Because no ground-truth dataset existed, correctness had to be established by comparison against real pages of the document, which set the pace of the data understanding phase. |

## 1.5 Proposed Solution

The requirements above rule out both alternatives surveyed earlier: manual search cannot answer a question, and a generic assistant cannot be trusted to answer it correctly. The system described in the rest of this chapter is built to do both, starting from a general design principle and then the four-layer architecture that implements it.

### 1.5.1 General Presentation of the Solution

The proposed system delineates two distinct tasks that conventional assistants typically merge: ascertaining factual content and determining the appropriate manner of expression. The factual information is extracted from a structured knowledge base, created by analyzing the document once offline, utilizing rules derived from the document's own structure and subsequently validated against printed materials. In contrast, phrasing is managed by a language model, which rewords facts that have been previously retrieved.

In between these two layers, a crucial verification process occurs. Before any generated response is presented to the user, it undergoes a comparison with the retrieved facts. Any response that omits or modifies a role name is discarded in favor of a predetermined template. Consequently, users receive accurate answers, albeit delivered in a less conversational format. This verification mechanism ensures the language model's safe application within this context. It is described in detail in Chapter 4.

### 1.5.2 Global System Architecture

The system comprises four distinct layers. The offline extraction layer converts the source PDF into a validated set of nodes, which are subsequently cached and included with the application. The knowledge layer utilizes this node set as a directed graph, enabling efficient searches and lookups. The agent layer interprets user inquiries, translates them into actionable tasks, formulates definitive responses, and utilizes the language model along with a verification process. Finally, the presentation layer interfaces with the agent through a REST framework, facilitating conversation, document attachment, export, sharing, and analytics features.

Figure 1.4 shows how these four layers relate, and in particular where the boundary between offline and online work falls.

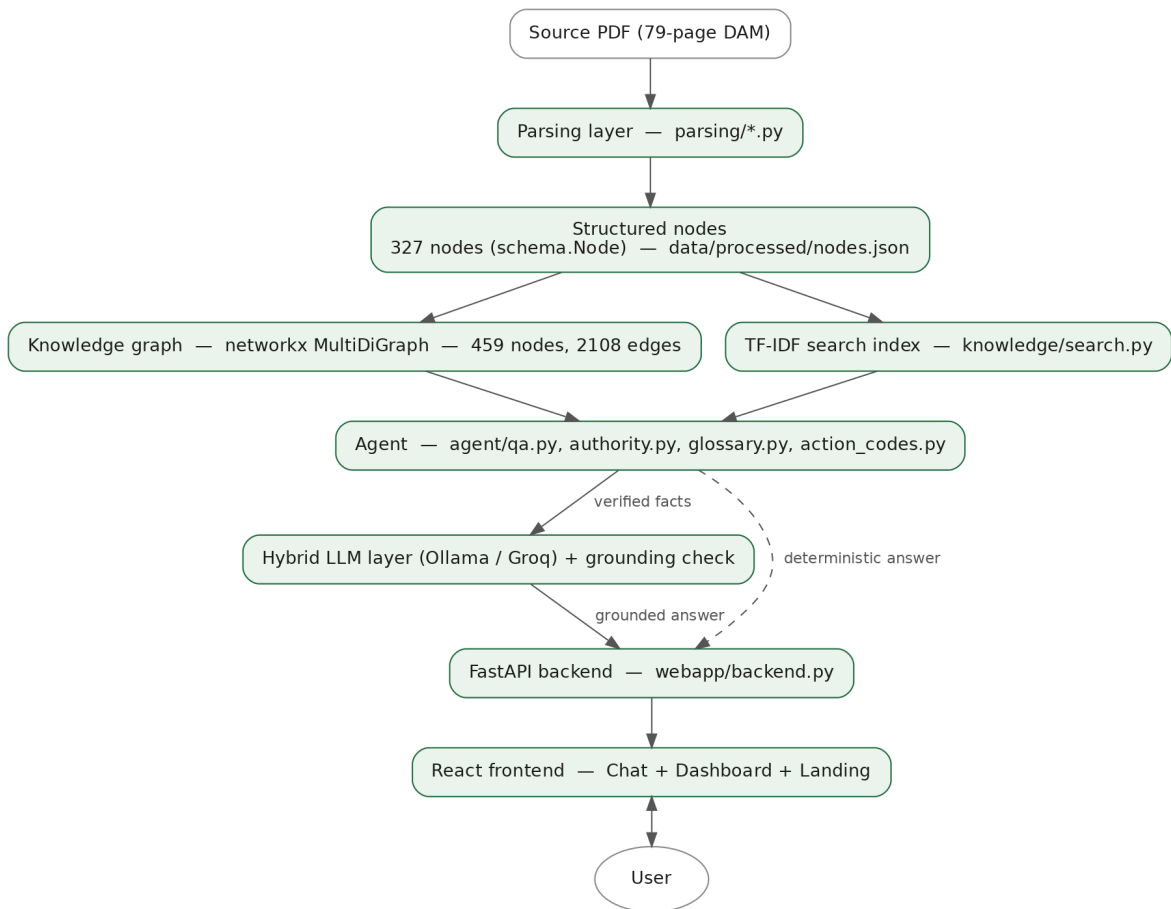


Figure 1.4: Global architecture of the proposed system, from source document to user-facing answer.

One notable consequence of this separation is that, because extraction occurs offline and its output is cached, the running application interacts exclusively with a prepared set of nodes stored in memory at startup, never directly accessing the PDF. This approach was adopted for deployment efficiency,

### 1.5.3 Selected Technologies

Several tools and technologies were selected to serve a specific purpose in the construction of the system, and each was chosen against the constraints recorded in Table 1.7 rather than on general merit. Table 1.8 lists each technology and the role it plays.

Table 1.8: Selected technologies and their role in the system.

| Technology                      | Role in the system                                                                                                                                             |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python                          | Language for extraction, knowledge base, agent, and back-end.                                                                                                  |
| pdfplumber[20]                  | Character- and word-level PDF extraction with positional coordinates, which is what makes geometric reasoning about rotated headers and superscripts possible. |
| Pydantic[21]                    | Schema definition and validation for the node model, so malformed records fail at construction rather than at query time.                                      |
| networkx[9]                     | Knowledge graph construction and traversal, using a directed multigraph so that a role can hold several distinct responsibilities over the same activity.      |
| scikit-learn[8]                 | TF-IDF vectorisation and cosine similarity for free-text activity resolution.                                                                                  |
| FastAPI[24]                     | REST backend, chosen for its typed request and response models and automatic API documentation.                                                                |
| React[25] and Vite[26]          | Web client, built as four independent per-page bundles rather than a single-page application, and served as pre-built static files by the backend.             |
| Groq[22] and Ollama[23]         | Language model providers for answer phrasing, cloud-hosted and local respectively, behind a common interface so either can be used or neither.                 |
| Whisper (hosted)[22]            | Server-side transcription of dictated questions, chosen over browser speech recognition for consistent behaviour across clients.                               |
| Web Speech API[27]              | Browser-native speech synthesis for reading answers aloud, which keeps audio playback entirely on the client.                                                  |
| python-jose[28] and passlib[29] | JSON Web Token issuing and password hashing for the authentication layer.                                                                                      |

| Technology | Role in the system                                                                      |
|------------|-----------------------------------------------------------------------------------------|
| Docker[32] | Containerisation, so the application runs identically in development and in deployment. |
| Render[33] | Hosting platform, used on its free tier.                                                |

## Conclusion

The Document Asset Management (DAM) system addresses a critical issue, yet its publication format significantly hinders accessibility. Manual consultation proves to be both slow and susceptible to error due to the document's meaning being intricately linked to its layout. Similarly, generic document assistants encounter challenges for the same reason, often resulting in a lack of fluency. The aforementioned requirements aim to address these issues, and the proposed architecture effectively separates factual retrieval from wording. This approach harnesses the potential of language models while mitigating their inherent unreliability. The subsequent chapter will delve into the source document, exploring the complexities involved in its extraction.

## CHAPTER

## 2

# DATA UNDERSTANDING: ANALYSIS OF THE DAM SOURCE DOCUMENT

### Introduction

Most data science projects begin with a structured dataset: rows, columns, typed fields, a distribution to describe and missing values to count. This one begins with a PDF. There is no dataset to explore until one has been built, and building it is only defensible if the document it comes from has first been understood on its own terms.

That is the purpose of this chapter. It does three things. It presents the source document, its provenance, its four-level hierarchy and the authority notation used in its tables. It then characterises the content quantitatively, establishing what the processed subset actually contains: how many activities, carrying how many responsibilities, distributed across which roles and which authority codes. Finally, it documents the ways in which automated extraction fails on this particular document, each failure mode identified by comparing extracted output against the printed page rather than inferred from theory.

The last of these carries the most weight for what follows. Every parsing rule presented in Chapter 3 exists to resolve a specific defect catalogued here. Reading the two chapters in sequence is intended to make the pipeline legible as a response to evidence rather than as a set of arbitrary engineering choices.

## 2.1 Source Document Presentation

Before any extraction rule can be justified, the document itself has to be described: where it comes from, how it is organised, and what notation it uses to record an authority decision. Every later chapter refers back to the vocabulary established here.

### 2.1.1 Source and Context

The dataset analyzed in this study comes from the African Development Bank Group’s Delegation of Authority Matrix, which is an internal governance document of more than two hundred and fifty pages distributed in the form of a PDF file. It is not a published research dataset and has no accompanying schema, documentation, or machine-readable export. The DAM has been specifically made to be read by the AFDB staff members. Subsequently, a specific data extraction method needed to be used in order to turn the DAM into a knowledge base. At the beginning of the project, work was scoped to a sixty-six pages subset, focusing only on two chapters, as per requested by the client. This initial export was later on replaced by a seventy-nine pages document for two main reasons: First, the larger file includes the glossary, the abbreviations list, and the authority-code legend, all of which the system needs as reference data rather than as content to be parsed. Second, It contains a full chapter excluded by the initial scope, that is relevant to the proper execution of this project. That chapter was brought into scope once its table structure was examined and found to be compatible with the same parsing approach, at the cost of one substantial new class of layout problem described in Section 2.3.5.

### 2.1.2 Hierarchical Structure of the Document

The DAM is structured according to a four-level hierarchy. Thus, the accurate reconstruction of this architectural framework represented a pivotal phase of the study. At the highest level, “chapters” categorize the AFDB’s activities into broad operational domains. Chapters are further subdivided into processes. These processes are identified by rounded numerical codes (e.g., 2.510) which group related activities. Processes then encompass individual tasks, designated by sequential numbers (e.g., 2.513) and certain tasks decompose further into child tasks, identified by a three-part numerical notation (e.g., 1.117.2). Finally, a small yet important subset of entries consists of threshold variants: sub-items keyed by monetary band rather than sequence number, delegating approval authority based on financial magnitude.

### 2.1.3 Table Layout and the Authority Notation

Content is presented as wide tables. The leftmost columns carry the activity identifier and its title. The remaining columns each match to an organisational role, and their headers are rotated vertically so that long role names fit within the available width. Each cell at the intersection of an activity and a role carries an authority code, or nothing at all if that role has no part in that

activity. Alongside the codes, superscript digits refer to footnotes printed at the bottom of the page, qualifying the responsibility with conditions such as delegation limits or specific circumstances. Two properties of this presentation create most of the difficulty. First, the codes are short, so distinguishing a code from a footnote marker relies on their positions in their respective cells rather than length. Second, the association between a code and a role is purely positional, established by horizontal alignment between a cell and a header printed sideways somewhere above it.

## 2.2 Exploratory Analysis of the Document

With the document's structure and notation established, the question becomes what the processed subset actually contains once parsed, from overall node counts down to the specific record types, roles, and edge cases that shape what the system can and cannot be asked.

### 2.2.1 Global Overview and Node Distribution

Once parsed and validated, the seventy-nine-page subset yields 327 nodes, distributed across the four hierarchy levels as shown in Table 2.1. These records carry 1,776 individual responsibility entries, each one an assertion that a named role holds a specific authority code over a specific activity.

Table 2.1: Distribution of node types across the processed subset.

| Node type         | Count      | Description                                                         |
|-------------------|------------|---------------------------------------------------------------------|
| Chapter           | 3          | Top-level domains of Bank activity covered by the processed subset. |
| Process           | 35         | Groups of related activities within a chapter.                      |
| Task              | 138        | Individual activities, the level most questions are about.          |
| Child task        | 102        | Sub-activities of a task.                                           |
| Threshold variant | 49         | Variants of a task keyed by a monetary threshold band.              |
| <b>Total</b>      | <b>327</b> |                                                                     |

Following the table above, we can see that even with only three chapters, we can see that they produce 35 processes, and therefore the processes have 240 tasks, of which 102 are child tasks and sub-activities. The proportion of threshold variants is higher than an outside reader might expect, and it reflects something about how the AFDB delegates authority. Approval rights are frequently banded by amount. Thus, a single named activity can carry three or four different approvers depending on the sum at stake. A system that treated each activity as having one answer would be wrong for roughly fifteen percent of the document.

### 2.2.2 Authority Code Distribution

Across the 1,776 responsibility entries, the codes are not evenly distributed. Informed markers and check-and-verify obligations are considerably more common than approvals which is unsurprising. As a matter of fact, a single approval is usually surrounded by several roles that are set to verify the work beforehand and numerous more that must be notified afterwards.

This statement can be illustrated by an example: a user happens to generate a query asking about who approves a certain activity and the system responds to the said query with strictly the approver. The response given by the system does not include the prerequisite steps and/or subsequent actions that need to be fulfilled in order to transfer approval. In other words, the system is considered to have partially answered the query and has omitted the mandatory obligations attached to it. This observation leads to understanding the need for requirement FR4, and the behaviour it specifies is further explained in Chapter 4.

### 2.2.3 Role and Actor Analysis

The document names 131 distinct roles across the processed subset, ranging from country office staff through sector managers, division managers, directors, and vice- presidents, all the way up to the Board. However, role names are long and heavily abbreviated. This issue had led to primarily considering the vertically rotated headers previously mentioned and the abbreviations are not always defined in the document's own glossary.

That last point produced a small yet visible feature of the finished system. In spite of the fact that some acronyms appear throughout the role names, they remain separately undefined in the abbreviations pages. As a result, the system is set to maintain a small set of curated aliases for the acronyms that are not defined, avoiding the burden for the user to look for it by themselves. Additionally, the system will clearly state when an expansion is drawn from that curated set instead of the glossary provided in the document. This feature provides a more transparent and easily understandable navigation through the system for the user.

Averaged across all 327 records, each activity carries an average of 5.48 responsibility entries. Just under thirty percent of records carry no direct responsibilities at all, which at first looks like an extraction failure and mostly is not. Chapters and processes are organizational containers rather than activities, so they legitimately have none, and a further group of tasks are pure cross-references, discussed below.

### 2.2.4 Threshold Variants and Monetary Bands

Threshold variants were not part of the initial understanding of the document's structure. Indeed, these threshold variants were detected during the data extraction process, when rows appeared in the shape of a task's sub-item while carrying a monetary band instead of a sequential number. In order to correctly identify the task, the threshold variants needed to be acknowledged as a

separate level of a task rather than forcing them into the child-task category. This distinction is also explained by the different natures of both categories : on one hand, a child task is a component of a task. On the other hand, a threshold variant is an alternative to its siblings.

### **2.2.5 Cross-References and Redirects**

A few entries in the DAM document record no responsibilities but instead have references to other parts of the document. These references come in two ways. One way involves a list of explicit references that are presented as “See DAM 16.100, 16.200,” and similar forms. The second way is by way of a range of references, which are said as “Refer to Activities 2.114 - 2.117 in Section 2.” This needs to be further elaborated into a list of all identifiers in the range and not just the two endpoints. Fourteen references like these were found in the analyzed set of data. When one sees this kind of statement, he or she will go through the pages referred to and continue analysis. The system is expected to follow the references because when someone asks for activity 1.117.2 and receives the answer that it has no responsibility listed, that answer is accurate but practically useless.

### **2.2.6 Consolidated View**

In combination, the exploratory analysis reveals some conditions for all further work done on the document. The document has a hierarchical structure; nevertheless, it cannot be derived from identifiers’ names. It is a positionally meaningful structure, and is coded by the association of cells with vertically turned headers. Quite a significant number of rows in this document represent not activities but pointers or threshold options. Furthermore, most responsibilities listed here are the ones that would be considered secondary by a naïve question answering system.

## **2.3 Extraction Quality Assessment**

This section highlights the inconsistencies found when the document is processed automatically. Every bug described below was identified through comparison of the output produced with the corresponding printed page it was taken from. Such a method takes more time than verification of correct code execution, but it is the only way to find such problems because any failure here results in totally believable output until it is compared with the source.

### **2.3.1 Text Layer Inspection**

PDF is a page description format. It contains the information about which glyph is drawn at each particular coordinate in a particular font and size. It does not contain any information about the semantic meaning of the group of these glyphs. Libraries like pdfplumber extract characters and words along with their positions. However, all structural properties have to be restored from the geometrical one because it is not done by the library. In the file "row" is not "row". It is the set of

words having close enough vertical positions. How close enough is not defined anywhere and can be determined empirically only.

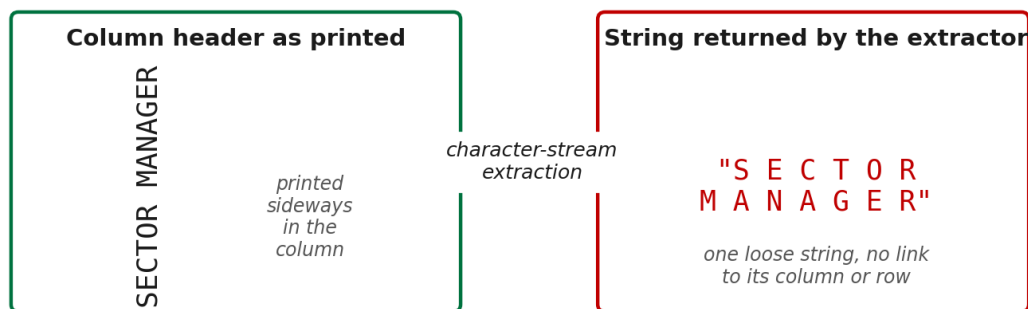


Figure 2.1: Column headers as printed, rotated through ninety degrees, beside the text an extractor returns for the same region. The header row is not recoverable as a table header.

### 2.3.2 Rotated Column Headers

The headers and the text an extractor returns for them are shown side by side in Figure 2.1.

Headers for roles are printed vertically. When the standard word extraction is used, they appear just as plain words with usual coordinates, which means that nothing is known about their rotation and the horizontal position of the word corresponds to the narrow strip of the rotated text rather than the whole column underneath it. Therefore, in order to attribute any code to some role, header words have to be clustered by their horizontal position and each cell should be associated with the nearest header cluster. The problem occurred somewhere in the middle of the document: some pages have headers only once and the table continues further without them. These pages have codes but no headers, so attribution fails unless the last seen headers are used instead.

### 2.3.3 Fused Footnote Digits and Authority Codes

Figure 2.3 shows a code and its footnote marker as printed, alongside the fused token that results when the size difference between them is disregarded.

The footnote markers are encoded as the superscripts adjacent to the authority code. After extraction, if the code A and the footnote 3 occur, the string “A3” is produced, which visually cannot be distinguished from the correct level-bearing code “A3.” The distinction can be made only on the basis of the vertical offset of the digits from the baseline and the difference in font sizes between the digit and the letter. Superscripts appear at higher offsets and in smaller fonts, and the rule was established experimentally based on real-life examples of characters, not on intuitive assumptions, since the offsets are small, and an intuitively guessed threshold would result in a significant number of misclassifications. Another flaw impacted the informed marker. Since the

marker consisted of the parenthesized character i, it could be detected by the string “(i)” in the text. However, on a real-life page, this marker consists of five separate characters, so nothing matched. Before this problem was discovered, of the 276 records in an early version of the extraction, 174 included the informed marker inside the printed document that had no impact on the extracted data. This flaw remained undetected until the output involved record comparison with the page. It can be explained by the fact that there is an additional Authority code when flattened produces the same string “I”.

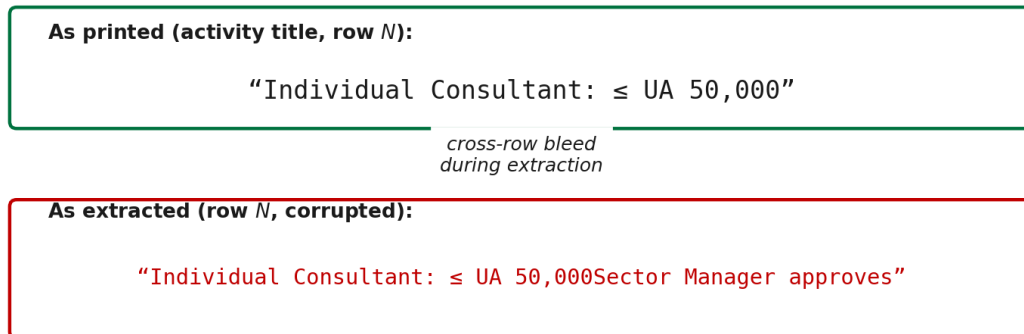


Figure 2.2: An activity title as printed, and the corrupted string produced when text from an adjacent row bleeds into it during extraction.

### 2.3.4 Cross-Row Text Bleeding and Title Corruption

Figure 2.2 pairs a printed title with the corrupted string extracted for the same activity.

The activity titles can consist of several lines, so a line of the title of one activity can be closer to the row of another activity in a dense table. In such situations, the title is associated with the wrong activity. Therefore, both activities are valid by themselves : one of them contains an overly short title, while the other is overly long, and there is nothing indicating that an error has occurred. At its worst, the problem occurred in roughly one-quarter of the records generated by a preliminary version of the parser. This was resolved through the use of a system where in title attachment depended on the parse state rather than just proximity, utilizing the following three criteria: (1) is there currently an open task, (2) does the open task’s text end in a period, and (3) does the line have authority codes when the open task already has them? After implementing these rules, the sole outstanding case occurred in the entire set of records for activity 2.222.1, a documented exception rather than an automatic acceptance.

### 2.3.5 Split Rows in the Non-Sovereign Operations Chapter

With the introduction of the Non-Sovereign Operations chapter, as well as the need to define scope after the initial parsing of the text, a layout exception was introduced wherein a single logical row was sometimes laid out as two separate rows, with the authority codes appearing a bit below

the activity text. The first attempt at correction consisted of simply merging any two vertically contiguous bands separated by less than some tolerance level. A check done before implementation showed 1,614 candidates mergers in the document, but many of them were incorrect and would have merged unrelated activities. This general solution was therefore dismissed, and a more limited one adopted whereby only a band containing nothing other than authority codes was merged, a strong indicator that it was part of the row above it.

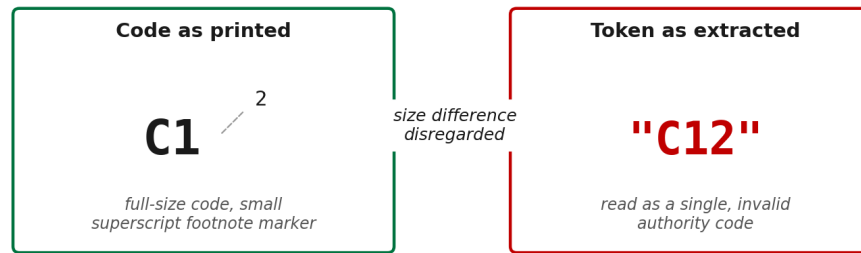


Figure 2.3: An authority code printed beside a superscript footnote marker, and the fused token produced when the size difference between them is disregarded.

### 2.3.6 Consolidated Data Issues Summary

Table 2.2 consolidates the defects described above, each paired with the stage of the pipeline that addresses it.

Table 2.2: Consolidated summary of extraction issues found in the source document.

| Issue                                 | Effect if unhandled                                               | Resolution                                                                                          |
|---------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Rotated column headers                | Codes cannot be attributed to roles.                              | Cluster header text by horizontal position; carry the header set forward across continuation pages. |
| Missing headers on continuation pages | Whole pages of responsibilities lost.                             | Reuse the last seen header set until a new one appears.                                             |
| Footnote digits fused with codes      | Codes misread; footnote qualifications lost.                      | Separate by measured vertical offset and font size of the character.                                |
| Informed marker not matched           | Informed parties absent from the majority of records.             | Match the marker as the separate characters it is actually rendered as.                             |
| Title bleeding across rows            | Titles attached to the wrong activity; free-text search degraded. | Conditional attachment based on parse state, terminating punctuation, and code presence.            |

| Issue                          | Effect if unhandled                                | Resolution                                                            |
|--------------------------------|----------------------------------------------------|-----------------------------------------------------------------------|
| Split rows in the NSO chapter  | Responsibilities detached from their activity.     | Merge a band only when it consists entirely of authority codes.       |
| Hierarchy from string prefixes | Two of four hierarchy levels silently wrong.       | Derive parentage from explicit parent fields and process arithmetic.  |
| Cross-reference rows           | Activities reported as having no responsibilities. | Extract references, expand ranges, and follow pointers at query time. |

## Conclusion

DAM can be seen as a well-written document for humans, and as an adversarial artifact for the parser, given that all the structural features that this artifact has can be conveyed using its structure. The problems presented in this chapter do not constitute edge cases. Rather, they together impact all the components in the answer that the parser should provide : the hierarchy, roles, titles, and informed parties. Furthermore, this chapter presents a methodology that can be applied in practice. None of these problems were detected using code inspection or by inferring what the parser was supposed to output. Rather, each problem was detected by collecting the output and comparing it against the corresponding page. This methodology continues in all subsequent parts of this thesis, as discussed in Chapter 3.

## CHAPTER

### 3

# DATA ENGINEERING PIPELINE AND KNOWLEDGE GRAPH CON- STRUCTION

## Introduction

This chapter details the transformation of the source document into queryable data. It discusses the selected processing environment, outlines the design of the extraction pipeline, and explicates the transformations implemented at each stage. Additionally, it presents the validated node set generated and the knowledge graph constructed from it, concluding with an evaluation of the pipeline's performance and its remaining limitations.

## 3.1 Processing Environment

Before describing the pipeline itself, it is worth justifying the kind of pipeline it is. Three choices were made before any extraction rule was written: a deterministic, rule-based approach over a learned one, a single-process program over a distributed framework, and a specific set of libraries chosen against the constraints of Table 1.7.

### 3.1.1 Motivation for a Deterministic, Rule-Based Pipeline

An alternative strategy for addressing this extraction challenge involves training a model by annotating a selection of pages to classify text regions and subsequently generalizing from this information. However, this approach was ultimately dismissed for practical reasons prior to any principled considerations. Specifically, the absence of annotated data posed significant hurdles, as the effort required to produce sufficient annotations for a single seventy-nine-page document would have outweighed the efforts involved in crafting a set of rules, all while complicating verifiability.

Verifiability serves as the foundational rationale for adopting a rule-based approach. Rules derived from measured character geometry can be readily verified against the originating page. In instances of failure, such discrepancies can be inspected closely, and the necessary corrections are localized. Conversely, a learned region classifier that misinterprets a single table offers no comparable insight into its decision-making process. In documents where errors are subtle and plausible, the ability to elucidate the reasoning behind a character's interpretation holds greater value than the enhanced flexibility that a learned approach might provide.

### 3.1.2 Why Not a Distributed Processing Framework

Distributed frameworks, such as Apache Spark, are typically employed when data processing demands exceed the capabilities of a single machine. However, this particular scenario is not one of those cases. The complete document is only a few megabytes, and the subset processed yields 327 records, with a full reconstruction of the knowledge base requiring approximately fifteen to twenty seconds on an average laptop.

Implementing a distributed framework would introduce unnecessary complexities, such as an execution model, cluster configuration, and serialization boundaries, which are irrelevant to this problem. The primary challenge lies in the interpretative aspect, specifically in discerning the meanings of characters rather than in the rapid processing of numerous characters. Consequently, the pipeline is structured as a single-pass, single-process program, which is best suited for this task and facilitates inspection of intermediate outputs during development.

### 3.1.3 Libraries Selected for Extraction, Validation and Modelling

Extraction is facilitated by pdfplumber[20], which provides detailed information about characters and words, including their coordinates, font names, and sizes. This character-level access is essential, as the superscript rule discussed in Chapter 2 relies on the vertical offset and size of individual characters, making word-level output insufficient.

The node model is established using Pydantic[21], ensuring that structural expectations are strictly enforced during construction. Consequently, any record that omits a required field or contains an authority code outside the recognized set will fail promptly, preventing issues from arising later in the graph.

## 3.2 Pipeline Design

With the environment settled, the pipeline itself has a shape worth laying out: how many stages it has, what each one hands to the next, and how the code implementing it is organised into modules.

### 3.2.1 Global Architecture

The process pipeline is made up of eight consecutive processes, where each process takes input from its previous process, and each process can be inspected independently. The raw characters get converted to physical rows, the physical rows are grouped into task blocks, the task blocks are cleaned and enriched with geometric and role-based data, and finally, they are arranged into validated records. Figure 3.1 shows the pipeline end to end, and it is worth reading it as a progressive narrowing of interpretation rather than as a sequence of transformations. At the left-hand edge the input is a stream of glyphs with coordinates, carrying no notion of a row, a cell, an activity or a role: the PDF format records where ink was placed, not what the ink means. At the right-hand edge the output is a set of validated records in which every responsibility is an explicit assertion that a named role holds a named obligation over a named activity. Every stage in between recovers exactly one layer of structure that the printed page expresses visually and the file itself does not encode.

The ordering is not arbitrary, and the stages are not independent. Each one consumes an abstraction the previous stage established and would be meaningless without it: codes cannot be separated from footnote markers before there are cells to inspect, cells cannot be attributed to roles before the rotated column headers have been clustered into columns, and no activity can be assembled before its block boundaries are known. This dependency is the reason the pipeline is deterministic and staged rather than a single extraction pass. A failure at any stage is therefore localised to that stage and inspectable in isolation, which is what made the defects catalogued in Chapter 2 diagnosable at all.

Table 3.1 names each stage, states what it does, and records why it exists at all, since several of them were added only after a specific defect made their absence visible.

Table 3.1: Stages of the data engineering pipeline, with the motivation for each.

| # | Name           | Function                                                                        | Why it exists                                                                                                                                 |
|---|----------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Raw extraction | Characters and words with co-ordinates, fonts, and sizes are read from the PDF. | Page-level text extraction discards the geometry on which every later stage depends, so extraction had to be character-level from the outset. |

| # | Name                         | Function                                                                                                             | Why it exists                                                                                                                                                           |
|---|------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | Row reconstruction           | Words are grouped into physical rows by vertical alignment, with split rows merged where the merge is provably safe. | Activities in the Non-Sovereign Operations chapter are printed across two physical rows, and without merging they were parsed as two unrelated fragments.               |
| 3 | Task block detection         | Rows are grouped into task blocks, and boundaries between consecutive activities are determined.                     | A title wrapping past its own identifier row was being claimed by the following activity, which silently attached responsibilities to the wrong task.                   |
| 4 | Title and reference cleaning | Titles are normalised, cross-references are extracted and expanded, footnote digits are stripped from title text.    | Roughly a quarter of titles carried embedded reference phrases or stray digits, which corrupted the search index and made free-text resolution unreliable.              |
| 5 | Geometric attribution        | Authority codes are separated from footnote markers by character geometry, and levels are attached.                  | A code and a footnote marker are both a letter beside a digit; only font size and vertical offset distinguish them, so fusing the two inverted the meaning of the cell. |
| 6 | Role attribution             | Codes are matched to roles by horizontal alignment with clustered, rotated column headers.                           | Column headers are printed rotated through ninety degrees and are not recoverable as table headers, so columns had to be reconstructed from header coordinates.         |
| 7 | Node assembly                | Enriched blocks are validated against the schema and merged with the chapter and process skeleton.                   | Enforcing the schema at construction makes a malformed record fail immediately, rather than surfacing later as a wrong answer to a user.                                |

| # | Name       | Function                                                                 | Why it exists                                                                                                                                          |
|---|------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8 | Validation | Whole-document consistency checks are run across the assembled node set. | Per-record validity does not imply document-level coherence: orphaned identifiers and unresolved references are only visible once the full set exists. |

### 3.2.2 Data Flow and Modular Organisation

The process is performed in each individual module with well-defined boundaries such that if an error occurs, it will be easier to pinpoint exactly where it occurred based on the output of one particular stage as opposed to following the execution throughout the whole program. This factor had great significance throughout the development process because whenever an incorrect output came out, the first thing to consider was which stage produced it.

Modules for parsing take care of rows, action blocks, metadata, geometry of action, column definitions, and text cleaning. Modules for modelling handle nodes, structural backbone, graph, and caching. The schema lives in a separate module, which is dependent on both but depends on none of the two.

## 3.3 Data Ingestion

The ingestion process deals with each page of the document individually. Characters and their locations and font metrics, as well as words with their bounding boxes are extracted from every single page. These two granularities are saved because the row construction needs word granularity, and the superscript rule is based on the character granularity.

References are extracted independently from other pages of the same document. The legend of authority codes is converted into a structured file that describes the full code, its hierarchy level and the meaning. This reference is checked against the schema and seventeen entries are loaded successfully. Abbreviations are extracted as a glossary. The glossary needed a special approach due to the facts that some entries break a line, some rows are section headings but not the definitions and the term and its definition are sometimes placed with a fractional difference in point. Therefore, row grouping is done with the same tolerance as elsewhere in the processing.

## 3.4 Document Processing and Normalisation

Stages two through six of Table 3.1 are covered here: the point where raw characters become rows, rows become task blocks, and blocks are cleaned and enriched enough for a schema to

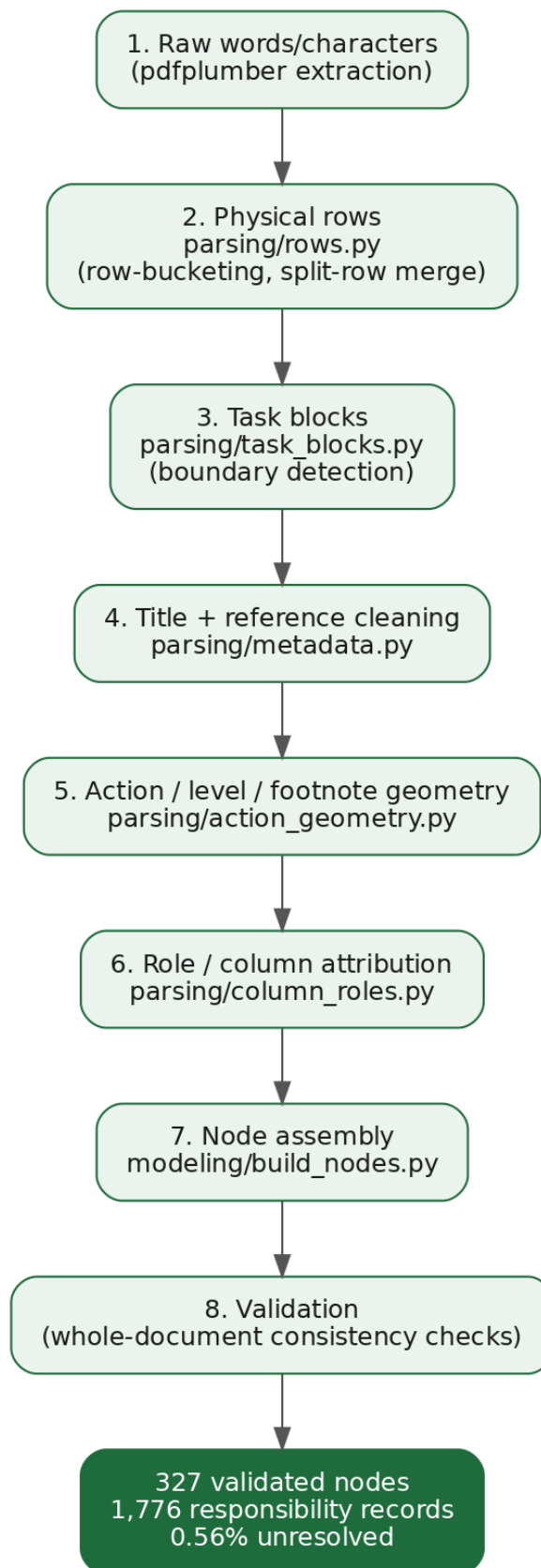


Figure 3.1: Eight-stage data engineering pipeline, from raw PDF extraction to a validated node set.

accept them. This is also where most of the defects catalogued in Chapter 2 are addressed directly, one subsection per family of defect.

### **3.4.1 Row Reconstruction**

Words are arranged into lines through grouping of their vertical coordinates. The reason for tolerance is that text that may appear on the same visual line is normally not placed on identical coordinates. Where the chapter Non-sovereign operations splits a logical line into two lines, these lines are combined into one only when the specific condition described in Chapter 2 applies. That is, the lower line consists of authority codes alone.

### **3.4.2 Task Block Detection**

Blocks then come out of rows in such a way that each block represents an activity. The boundary detection turned out to be the most iterative part of the process as the visual cue that denotes a boundary was unreliable. There were three different cues considered: (1) whether there is an open task, (2) whether the open task ends on a full stop, and (3) whether there is any new row with authority codes while there are codes for the current open task. Of these three cues, the third one appears to be the most reliable one.

### **3.4.3 Title and Reference Cleaning**

Normalization of titles is achieved by stripping footnote digits, compressing whitespace, and stripping references that would have been incorporated as noise in the title. Extraction of references is concentrated in one function used by all pathways that deal with titles or notes. The current scheme contrasts with the original design that had two different but conflicting implementations, one of which could handle the reference pattern while the other could not.

### **3.4.4 Geometric Attribution of Authority Codes**

Codes of authority are found in each row and are distinguished from footnote indicators according to the character-level rule stated in Chapter 2. Level digits belonging to the codes remain intact as part of the code, but superscripted digits are entered independently as footnote numbers for that responsibility.

### **3.4.5 Role Attribution**

The enriched blocks are put together in such a way that the records follow the required schema and they get coupled with a structural skeleton that contains the chapter nodes and process nodes, since these nodes are containers for the records and not emitted as activities by the table parser.

The parentage is defined explicitly and not by the structure of the identifiers. There is a pointer to the parent node inside each record and the hierarchy is built using these pointers. In an older

version of the system, the children of a node were defined based on the structure of the identifier and it worked well only for two of the four levels but not for the other two levels. This problem was detected only when the threshold variants started appearing in the identifier space and the grandchildren were clashing with the children under the same prefix.

The indicators in Table 3.2 are the measured state of the node set after construction, and they are the figures against which the pipeline’s adequacy is judged.

Table 3.2: Data quality indicators after node construction.

| Indicator                                           | Value                          |
|-----------------------------------------------------|--------------------------------|
| Structured records produced                         | 327                            |
| Responsibility entries extracted                    | 1,776                          |
| Distinct roles identified                           | 131                            |
| Unresolved role entries                             | 10 (0.56%)                     |
| Average responsibilities per record                 | 5.48                           |
| Records with no direct responsibilities             | 29.63%                         |
| Cross-references captured                           | 14                             |
| Responsibility entries carrying footnote references | 113 (6.36%), across 44 records |
| Residual title corruption                           | 1 record (2.222.1)             |

### 3.4.6 Geometric Tolerances

Several pipeline stages reason about position and size rather than about text, because the distinctions they draw are not present in the character stream at all. Those stages are governed by the tolerances in Table 3.3, stated in PDF user-space units, where one unit is  $1/72$  of an inch. The values are recorded here because without them the extraction is not reproducible: the same source document processed with different tolerances yields a different node set.

Table 3.3: Geometric tolerances governing extraction, in PDF user-space units.

| Tolerance                   | Value | What it separates                                                                                                         |
|-----------------------------|-------|---------------------------------------------------------------------------------------------------------------------------|
| Body text minimum size      | 8.0   | Characters at or above this size are body text. Below it, a character is a candidate footnote marker rather than content. |
| Footnote digit maximum size | 4.0   | A trailing digit at or below this size is treated as a footnote reference and stripped from the cell.                     |
| Row band padding            | 1.0   | Vertical tolerance when collecting the characters and words belonging to one printed row.                                 |
| Row merge maximum gap       | 5.0   | Two vertically adjacent row bands closer than this may be merged into a single logical row.                               |

| Tolerance                    | Value | What it separates                                                                                        |
|------------------------------|-------|----------------------------------------------------------------------------------------------------------|
| Column alignment tolerance   | 2.0   | Horizontal distance within which a cell is considered aligned to a column centre.                        |
| Tight cluster gap            | 3.0   | Characters separated by less than this belong to the same header cluster.                                |
| Column merge gap             | 14.0  | Two header clusters closer than this are treated as one column rather than two.                          |
| Subgroup tolerance           | 0.6   | Vertical agreement required for header fragments to form one subgroup.                                   |
| Subgroup minimum gap         | 1.5   | Separation below which two candidate subgroups are not distinguished.                                    |
| Decoration gap maximum       | 8.0   | Distance beyond which an adjacent glyph is ruled out as decoration attached to a code.                   |
| Spaced-word gap maximum      | 6.0   | Gap below which adjacent single letters are read as letter-spaced prose rather than as standalone codes. |
| Glossary title minimum size  | 20    | Font size at or above which a glossary line is a heading rather than an entry.                           |
| Glossary top merge tolerance | 3.0   | Vertical tolerance when joining a wrapped glossary definition to its term.                               |

Two of these tolerances resolve ambiguities that no single test settles on its own.

The footnote-digit rule is size-based, and size alone produces false positives. Department codes such as PGCL.1, FIFC.4 and FITR.2 also end in a digit, and that digit belongs to the code. The rule therefore carries a second, independent condition: the character preceding the digit must not be a period. The two tests are needed together because the same code was observed rendering its digit at normal size on some pages and at the reduced footnote size on others, so size is not a reliable discriminator by itself.

The spaced-word tolerance separates a genuine authority code from letter-spaced prose. A single capital letter standing alone in a row may be an authority code, or it may be one glyph of a title that the renderer has spaced out character by character, as in A p p r a i s a l. A real code is followed either by the gap to the next table column or by nothing at all, so adjacency below 6.0 units to another single-letter word identifies the second case. Authority codes themselves are recognised by the pattern `^[ICRA]\d*$`, one of the four letters with an optional level digit.

Figure 3.2 shows these quantities on a real page region.

## 3.5 Knowledge Graph Modelling

The pipeline's final output is a validated set of nodes, but a set on its own cannot answer a relational question such as who approves an activity. That is why the nodes are loaded into a graph rather than

Alignment band of one column

Rotated header clusters

Full-size code beside a reduced-size footnote digit

CSP-RISP 1.100: PROGRAMMATIC ENGAGEMENT WITH COUNTRIES & RECS

|              |                                                           |                                    |                 |                                               |
|--------------|-----------------------------------------------------------|------------------------------------|-----------------|-----------------------------------------------|
| DAM Code No. | CSP-RISP 1.100                                            | SNVP/RDVP/AHVP/PEVP/PIVP/FIVP/ECVP | Authority Codes |                                               |
| Process:     | PROGRAMMING FOR COUNTRY & REGIONAL INTEGRATION OPERATIONS |                                    | <b>I</b>        | Initiate (or Launch) / Originate (or Prepare) |
| Date:        | December 2022                                             |                                    | <b>C</b>        | Check and Verify / Check and Clear            |
|              |                                                           |                                    | <b>R</b>        | Review and Recommend / Review and Clear       |
|              |                                                           |                                    | <b>A</b>        | Endorse (A1) / Approve (A2)                   |
|              |                                                           |                                    | <b>(I)</b>      | To Be Informed                                |

| Activity No. | Country Strategy Papers (CSP), Regional Integration Strategy Papers (RISP) and other programming documents <sup>a</sup> | Country Economist <sup>1</sup>                                         | Regional Integration Coordinator <sup>2</sup> | Country / Regional Programs Officer <sup>3</sup> | C / RISP Team <sup>4</sup> | Regional Portfolio & Implementation Managers <sup>5</sup> | Team Economist (ECCE) <sup>6</sup> | Country Manager / DDC <sup>7</sup> | RDG <sup>8</sup> | Director RDH <sup>9</sup> | Director ECCE <sup>10</sup> | TRAC <sup>11</sup> | PRC <sup>12</sup> | RVVP <sup>13</sup> | RVVP <sup>14</sup> | CS-COM <sup>15</sup> | PST <sup>16</sup> | CODE <sup>17</sup> | RD <sup>18</sup> |
|--------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|----------------------------|-----------------------------------------------------------|------------------------------------|------------------------------------|------------------|---------------------------|-----------------------------|--------------------|-------------------|--------------------|--------------------|----------------------|-------------------|--------------------|------------------|
| L.110        | Country Strategy Papers (CSP), Regional Integration Strategy Papers (RISP) and other programming documents <sup>a</sup> |                                                                        |                                               |                                                  |                            |                                                           |                                    |                                    |                  |                           |                             |                    |                   |                    |                    |                      |                   |                    |                  |
| L.114        | New Strategy Paper                                                                                                      |                                                                        |                                               |                                                  |                            |                                                           |                                    |                                    |                  |                           |                             |                    |                   |                    |                    |                      |                   |                    |                  |
| L.114.1      | Preparation, review and approval of New CSP (with a 3-Year Rolling Business Plan)                                       | I                                                                      |                                               |                                                  | (I)                        | R <sup>1</sup>                                            | C                                  | C                                  | R <sup>12</sup>  | C <sup>4</sup>            |                             |                    | R <sup>12</sup>   | (I)                | R <sup>14</sup>    |                      | A <sup>16</sup>   | A <sup>17</sup>    |                  |
| L.114.2      | Preparation, review and approval of New RISP (with a 3-Year Rolling Business Plan)                                      | I                                                                      |                                               |                                                  | (I)                        | R <sup>1</sup>                                            | C                                  | (I)                                | R <sup>12</sup>  | C                         | C <sup>4</sup>              |                    |                   | R <sup>12</sup>    | (I)                | R <sup>14</sup>      |                   | A <sup>16</sup>    | A <sup>17</sup>  |
| L.115        | Other Programming Documents <sup>15</sup>                                                                               |                                                                        |                                               |                                                  |                            |                                                           |                                    |                                    |                  |                           |                             |                    |                   |                    |                    |                      |                   |                    |                  |
| L.115.1      | Interim CSP/RISP, Country Brief, or JCAS                                                                                | Follow the respective process for new CSP/RISP (See L.114.1 / L.114.2) |                                               |                                                  |                            |                                                           |                                    |                                    |                  |                           |                             |                    |                   |                    |                    |                      |                   |                    |                  |
| L.115.2      | Updated CSP/RISP ('Extension')                                                                                          | Follow the respective process for new CSP/RISP (See L.114.1 / L.114.2) |                                               |                                                  |                            |                                                           |                                    |                                    |                  |                           |                             |                    |                   |                    |                    |                      |                   |                    |                  |
| L.116        | Mid-Term Review                                                                                                         |                                                                        |                                               |                                                  |                            |                                                           |                                    |                                    |                  |                           |                             |                    |                   |                    |                    |                      |                   |                    |                  |
| L.116.1      | Preparation, review and approval of Mid-Term Review of CSP                                                              | I                                                                      |                                               |                                                  | (I)                        | R <sup>1</sup>                                            | C                                  | C                                  | R <sup>12</sup>  | C <sup>4</sup>            |                             |                    | R <sup>12</sup>   | (I)                | R <sup>14</sup>    |                      | A <sup>16</sup>   | (I) <sup>18</sup>  | A <sup>17</sup>  |
| L.116.2      | Preparation, review and approval of Mid-Term Review of RISP                                                             | I                                                                      |                                               |                                                  | (I)                        | R <sup>1</sup>                                            | C                                  | (I)                                | R <sup>12</sup>  | C                         | C <sup>4</sup>              |                    |                   | R <sup>12</sup>    | (I)                | R <sup>14</sup>      |                   | A <sup>16</sup>    | A <sup>17</sup>  |

Figure 3.2: A DAM table region annotated with the geometric quantities used during extraction: rotated header clusters, the horizontal alignment band of one column, and a full-size authority code beside a reduced-size footnote digit.

kept flat, with the graph's nodes and edges defined next, along with what the assembled structure looks like in aggregate.

### 3.5.1 Why a Graph

The valid set of nodes could have been modeled as either relational tables or as a flat set of documents[5]. The graph was chosen since the queries posed to the system are relational in nature. For instance, the query "Who approves this activity" translates into traveling in the graph from the activity node via edges of a certain kind, while "which activities does this role approve" amounts to traveling the opposite way in the graph. In both instances, the operation performed is a single one on a graph, unlike join or scan on a flat structure.

A directed multigraph is used instead of a simple graph since one role can hold multiple responsibilities for the same activity. That is why two nodes will be linked with multiple edges, each carrying its unique authorization code and references to the footnote.

### 3.5.2 Node and Edge Schema

The graph contains two kinds of node. Activity nodes correspond one-to-one with the 327 structured records. Role nodes correspond to the 131 distinct roles. Edges carry the relationships between them.

Table 3.4 sets out the edge types the graph carries and what each one asserts.

Table 3.4: Edge types in the knowledge graph.

| Edge type       | From → To           | Meaning                                                                                                 |
|-----------------|---------------------|---------------------------------------------------------------------------------------------------------|
| contains        | Activity → Activity | Hierarchical containment: chapter to process, process to task, task to child task or threshold variant. |
| responsible_for | Role → Activity     | A recorded responsibility, carrying the authority code, its level, and any footnote references.         |
| references      | Activity → Activity | A cross-reference extracted from the activity's own text, including ranges expanded to their members.   |

Figure 3.3 shows a representative excerpt, centred on activity 3.111, in which the containment edge from the parent process and the individual responsibility edges to each role are both visible.

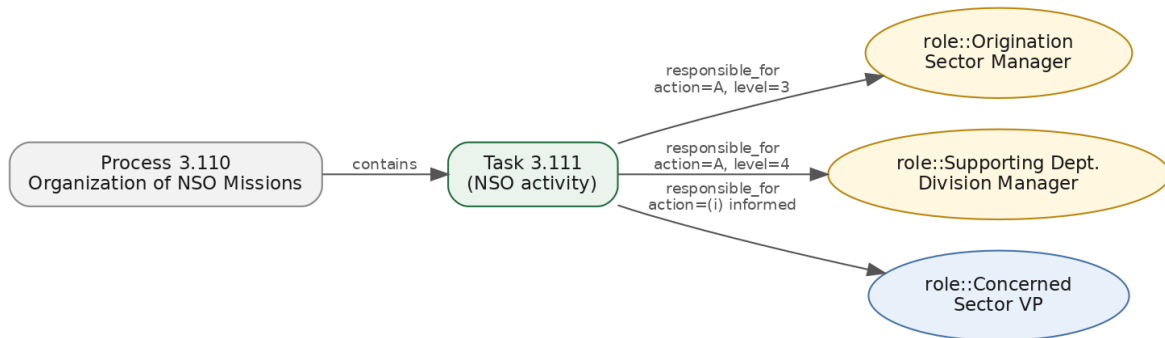


Figure 3.3: Excerpt of the knowledge graph around activity 3.111, showing the containment edge from its parent process and the responsibility edges connecting it to its approving and informed roles.

### 3.5.3 Graph Statistics

The assembled graph holds 459 nodes and 2,108 edges. The node count is the sum of the 327 activities and the 131 roles, plus one aggregate node. The edge count is dominated by responsibility edges, with containment edges accounting for the hierarchy and a small number of reference edges added from cross-references.

## 3.6 Pipeline Evaluation

A pipeline that runs is not the same as one that can be trusted. Both the extraction step's runtime behaviour and the data-quality checks applied to its output are covered below, since output is not allowed anywhere near a user's question until it passes them.

### 3.6.1 Execution and Caching

It takes around fifteen to twenty seconds to reconstruct the document from the PDF. This time frame is fine in an offline scenario but unacceptable when requested; hence, the node set that is being built is written into a cache file and later read during application initialization. The service running will never access the PDF while working.

The above arrangement deviates from the original one; the explanation for this change is provided in Chapter 5. The original implementation started by parsing the document when the application was initialized and was halted by the hosting system because of memory limit issues. Caching solved this problem, and in addition, this change prevented a more hidden problem, which is relying on the runtime environment to parse the document.

### 3.6.2 Data Quality Outcomes

The role that is unresolved For the subset that was processed, there are missing titles in record 2.516 and record 2.522, and one record (2.222.1) remains partially corrupted. Rather than being accepted and unstated, all three are known and documented. There was a candidate fix for the last case that was applied and tested but found to be not a fix for the target case but a regression in another case, so it was reverted. This was thought to be a better result than to leave a documented defect where a change could be made to make the overall extraction more desirable.

The second restriction is further discussed in the general conclusion and is due to the footnotes. Footnote reference numbers are captured and remain unchanged in respect of responsibilities and the bottom of the page text will not yet be parsed so that the system can say that a qualification is applicable if the text says so. During testing two similar extraction defects were found: a column header was confused as a role name, while two adjacent footnote digits were joined into one number in the same records. The planned remediation is also documented along with these two in Chapter 5.

But only three chapters have been exercised against the pipeline. The remainder of the entire document has the same table conventions as far as the present document is concerned, but for purposes of this document, at least, and certainly as part of a continuation of the work, such tables would be the first step in processing them. ved at a rate of 0.56 percent is the key quality measure of the pipeline and deserves careful consideration. This rate is the ratio of the number of responsibility items extracted, for which there was no matching column header. This is not related to checking whether the right code has been extracted for each cell, and this is an assessment that can only be done manually because of the lack of a ground truth dataset. This issue was examined by sampling and comparing particular activities against the printed page.

### 3.6.3 Observed Limitations

For the subset that was processed, there are missing titles in record 2.516 and record 2.522, and one record (2.222.1) remains partially corrupted. Rather than being accepted and unstated, all three are

known and documented. There was a candidate fix for the last case, that was applied, tested, but found to be not a fix for the target case but a regression in another case so it was reverted. This was thought to be a better result than to leave a documented defect where a change could be made to make the overall extraction more desirable.

The second restriction is further discussed in the general conclusion and is due to the footnotes. Footnote reference numbers are captured, and remain unchanged in respect of responsibilities and the bottom of the page text will not yet be parsed so that the system can say that a qualification is applicable if the text says so. During testing two similar extraction defects were found: a column header was confused as a role name while two adjacent footnote digits were joined into one number in the same records. The planned remediation is also documented along with these two in Chapter 5.

But only three chapters have been exercised against the pipeline. The remainder of the entire document has the same table conventions as far as the present document is concerned but for purposes of this document, at least, and certainly as part of a continuation of the work, such tables would be the first step in processing them.

## **Conclusion**

The pipeline transforms a low-level document that is only graphically organized into 327 records that are validated and have 1776 responsibility entries, with less than 1 percent of those entries unassigned, and it converts the output into a graph of 459 nodes and 2108 edges that can answer questions about relations by traversal. Each stage is deterministic and inspectable, which enabled the defects as described in Chapter 2 to be detected, corrected, and staved off. The next chapter adds the agent that queries this graph.

## CHAPTER

# 4

# RETRIEVAL, GROUNDED GENERATION AND AGENT DESIGN

## Introduction

The facts are stored in the knowledge graph. This chapter explains the layer that converts a typed and/or spoken query into an answer from it. These are three problems that need to be solved in order: figuring out what the user is talking about, figuring out what he wants to know about it, and saying it back to him in a way that doesn't go against the grain of the situation.

The third is the place the majority of design work went, which sets this system apart from the typical retrieval-augmented assistant[3]. This language model is not relied on to make any decisions. It is fed facts it has already retrieved and can only be asked to restate them; no results are presented until it confirms what it has said with the facts.

## 4.1 Retrieval Methodology

Resolving what a question is about comes before anything can be said about it, and the system takes two different routes depending on how the question is phrased: a fast, unambiguous path for a question naming an identifier directly, and a ranked free-text path for everything else. Both routes are described below, along with the numeric gates that decide what a free-text score is allowed to become.

### 4.1.1 Identifier Fast Path

If a question has something in it that looks like an activity identifier, it first attempts a direct look-up. Identifiers are unique, so there is no ambiguity and a match is precise. This route is the low cost route, and it's the one that is used by those users who already know what they're looking for.

If an identifier is not found in the node set, it is considered as a different result and not as a failure. This distinction is not as insignificant as it may seem. Previously, an unrecognised identifier would be matched with free text search over the entire question, including the bogus identifier itself. The other words of the question (such as “task”, “who”, and “approves”) have enough common vocabulary with real activity titles to score positive, but would have been confused with the actual activity a user was asking about. The behaviour was not an edge case situation, but a natural case.

### 4.1.2 Identifier Grammar and the Invalid-Identifier Branch

An identifier is recognised by the pattern  $\backslash d+(\backslash . \backslash d+)^{\{1,3\}}(\backslash . [a-z])?$ , which admits between two and four dotted numeric components and an optional trailing letter. It therefore matches 2.126, 1.117.2, 2.432.4.a and rejects a bare integer, so an ordinary monetary figure in a question is not mistaken for an activity reference.

Recognition and existence are separate tests, and the separation produces three outcomes rather than two. A matched string that is a key in the node set resolves immediately at confidence 1.0. A matched string that is not a key routes to a distinct `invalid_id` outcome. A query containing no identifier-shaped substring at all falls through to the free-text path.

The middle outcome exists because merging it into free-text search produces a specific failure. A question such as “who approves 9.999.999” still contains the words *who* and *approves*, and those words alone are sufficient to score against some unrelated real activity. The system would then answer confidently about an activity the user never named. Routing the case separately means the response states that the identifier does not exist.

Suggestions accompanying that refusal are computed by longest shared dotted prefix. For an invalid identifier  $x$ , every indexed node whose leading component equals that of  $x$  is ranked by

$$\text{depth}(x, n) = \max\{j : x_{1..j} = n_{1..j}\},$$

the number of leading dotted components the two share, and the three highest are returned. If the leading component itself matches no node, the identifier is stripped from the query and the remaining words are put through the text index instead, so a malformed reference accompanied by a usable description still yields candidates.

Figure 4.1 shows the complete path, including the threshold gate described in Section 4.2.1.

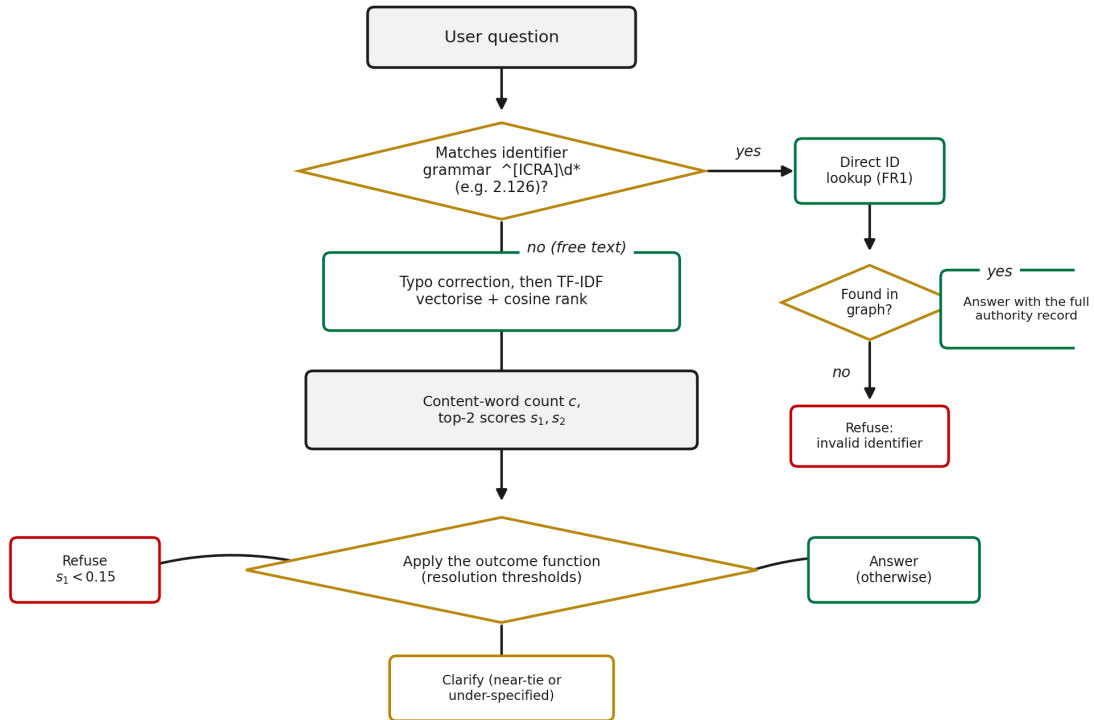


Figure 4.1: Resolution decision flow, from raw query to one of four outcomes: identifier resolution, invalid-identifier refusal, clarification, or a scored free-text match.

### 4.1.3 Free-Text Resolution

If the question mentions an activity but not any specific activity, lexical similarity is used to resolve the question. Each activity is represented in a vector using TF-IDF[1, 2] and a comparison is performed to the question by cosine similarity, returning the vector with the highest score and its value.

Tasks and child tasks, as well as process-level titles, are included in the index. Initially they were excluded because they didn't ask about activities, they asked about grouping organisations. The user was asking about the organisation of missions in the non-sovereign chapter, and so he was seeking to match nothing useful, since the only node with a title that had that phrasing was a process, and processes were not indexed.

### 4.1.4 Index Construction

The free-text path is served by a TF-IDF index[1, 2] fitted once at startup and held in memory for the lifetime of the process. Table 4.1 gives its configuration.

Table 4.1: Configuration of the TF-IDF retrieval index.

| Parameter           | Value                                        | Effect                                                                                                                              |
|---------------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Corpus unit         | Activity title                               | One document per indexed node; the title is the only field vectorised.                                                              |
| Indexed node types  | process, task, child_task, threshold_variant | Chapter nodes are excluded, since chapter titles are empty in the extracted set.                                                    |
| Empty-title filter  | applied                                      | A node whose title is blank after normalisation contributes no row.                                                                 |
| stop_words          | "english"                                    | Removes closed-class English words before weighting, so shared function words do not raise a score.                                 |
| ngram_range         | (1,2)                                        | Indexes unigrams and bigrams. A two-word phrase such as “mission programme” contributes a term of its own in addition to its parts. |
| Similarity          | Cosine                                       | Scores lie in $[0, 1]$ ; 0 means the query and title share no weighted term.                                                        |
| Candidates returned | $k = 5$                                      | The ranked list the resolution layer receives. Zero-scoring rows are discarded before return.                                       |
| Mutability          | Read-only after fit                          | The matrix is never refitted at runtime, so an identical query returns an identical ranking for the life of the deployment.         |

Including bigrams matters for this corpus because DAM titles are short noun phrases in which word order carries meaning. “Report to the Board” and “Board report” share every unigram and no bigram, and a unigram-only index scores them identically.

Given a query  $q$  and the set of indexed titles  $D$ , the resolved candidate set is

$$R(q) = \operatorname{argtop}_{d \in D}^k \frac{\mathbf{v}(q) \cdot \mathbf{v}(d)}{\|\mathbf{v}(q)\| \|\mathbf{v}(d)\|}, \quad k = 5,$$

where  $\mathbf{v}(\cdot)$  is the TF-IDF transform described above. The score is lexical overlap. A score of 0 states that the query shares no weighted term with the title, which is not the same claim as the activity being unrelated to the question.

#### 4.1.5 Why Lexical Retrieval Rather Than Embeddings

The more popular solution is semantic retrieval using learned embeddings – this is the one that would apply to a large and diverse body of ordinary language text. The volume of this corpus

is not large and it is not representative. It's a handful of hundred short papers that have been expressed in a specific institutional jargon, such that the words the user will type are the ones most likely used by the document.

Lexical matching is also explainable: this is a property that is important for this project. The shared terms can be carried as a score, a mismatch can be identified by analyzing the vocabulary, and the entire layer runs without downloading a model, without an inference cost, and without relying on an external service. As retrieval accuracy was the key for everything else, it was more valuable to be able to reason about why a given node was returned than to get the additional recall that an embedding model might have provided.

#### 4.1.6 Typo Tolerance

People misspell things, particularly long institutional phrases. Rather than relying on a language model to interpret intent, the system applies a deterministic correction step: each alphabetic word in the question is compared against the vocabulary the document itself uses, and replaced with the closest match when the similarity exceeds a threshold and the word is not already an exact match.

The threshold was calibrated against a real false positive rather than chosen by intuition. Set too low, ordinary words get rewritten into document vocabulary they merely resemble, which corrupts questions that were correct to begin with. Because the step is deterministic, it behaves identically whether or not a language model provider is reachable.

#### 4.1.7 Correction Algorithm and Parameters

Correction runs before vectorisation and operates on the query alone; the index is never altered. Each alphabetic token is compared against the vocabulary of single words the vectoriser already holds, so the correction target is the document's own wording rather than a general English dictionary. A token identical to a vocabulary entry under case folding is left alone. Otherwise the closest vocabulary entry is retrieved by Ratcliff–Obershelp sequence matching, and the token is replaced only when that similarity reaches the threshold in Table 4.2. A token with no sufficiently close match is passed through exactly as typed.

Table 4.2: Parameters governing deterministic typo correction.

| Parameter            | Value                    | Effect                                                                                              |
|----------------------|--------------------------|-----------------------------------------------------------------------------------------------------|
| Similarity measure   | Ratcliff–Obershelp ratio | Length-normalised, in $[0, 1]$ . Computed against the index vocabulary, not an external dictionary. |
| Acceptance threshold | 0.88                     | A token is replaced only at or above this similarity. Below it, the token is left as typed.         |
| Minimum token length | 4 characters             | Tokens of three characters or fewer are skipped entirely.                                           |

| Parameter             | Value      | Effect                                                                                |
|-----------------------|------------|---------------------------------------------------------------------------------------|
| All-capital tokens    | Skipped    | A fully capitalised token is treated as an institutional acronym and never corrected. |
| Candidates considered | 1          | Only the single closest vocabulary entry is eligible.                                 |
| Capitalisation        | Preserved  | A corrected token keeps the leading case of the token it replaces.                    |
| Scope                 | Query only | The index, the stored titles, and the answer text are untouched.                      |

Two of these guards address distinct failure modes. The length floor exists because similarity is close to uninformative on very short tokens: at one to three characters a large fraction of the vocabulary sits above any usable threshold, so short words would be rewritten arbitrarily. The all-capitals rule exists because the DAM’s role and authority vocabulary is dense with acronyms such as DDG, RDG and PGCL, and fuzzy-matching an acronym against a vocabulary of English title words can convert a correct token into an unrelated common word.

The effect on retrieval is direct. TF-IDF has no inherent tolerance for misspelling: a misspelled token is simply out of vocabulary and contributes nothing to the score, so a query carrying the right words with the wrong spelling scores as though those words were absent. Correcting before vectorisation restores the contribution rather than compensating for its loss afterwards.

## 4.2 Evaluation Protocol and Confidence Semantics

There is no held-out test set here, because there is no labelled corpus to hold out. Correctness is established instead by comparison against the source document, and every resolution carries a method label and a confidence value so that the basis of an answer is visible to the user rather than implied.

Every answer carries the method by which its activity was resolved, and that label is shown to the user rather than kept internal. Table 4.3 states what each method means and how much weight an answer resolved that way should be given.

Table 4.3: Resolution methods and their interpretation.

| Method      | Interpretation                                                                                      |
|-------------|-----------------------------------------------------------------------------------------------------|
| id          | Exact identifier match. No ambiguity.                                                               |
| text_search | Resolved from the activity’s name. The score is shown, and low scores are flagged in the interface. |
| invalid_id  | An identifier was recognised in the question but does not exist in the document.                    |

| Method              | Interpretation                                                |
|---------------------|---------------------------------------------------------------|
| needs_clarification | The question is too vague to resolve to a single activity.    |
| out_of_scope        | The question is not about the subject matter of the document. |
| glossary            | The question asked for the expansion of an abbreviation.      |
| action_code_legend  | The question asked what an authority code means.              |
| smalltalk           | Conversational input with no lookup component.                |

Exposing the method alongside the answer is a small decision with a large effect on trust. A user who can see that an answer came from an exact identifier match treats it differently from one resolved by name at a middling similarity score, and that difference in treatment is appropriate.

### 4.2.1 Resolution Thresholds

A ranked candidate list is not yet an answer. Four numeric gates sit between the ranking and the response, and together they determine whether the system answers, asks for clarification, or refuses. Table 4.4 states them.

Table 4.4: Thresholds governing the outcome of a free-text resolution.

| Gate                  | Value | Behaviour                                                                                                                                                        |
|-----------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Answer floor          | 0.15  | A top score below this value is not answered. The query is treated as unresolved rather than matched to its nearest title.                                       |
| Clarification ceiling | 0.60  | Above this score a top match is accepted even when the query is under-specified. At or below it, an under-specified query is sent to clarification.              |
| Separation minimum    | 0.15  | The gap between the first and second candidate. A gap narrower than this, with a top score under 0.85, routes to clarification rather than selecting the leader. |
| Carry-over ceiling    | 0.50  | A follow-up question scoring above this resolves on its own words; at or below it, the activity carried over from the previous exchange is used instead.         |

Two of these gates encode conditions that a single score cannot express on its own.

The separation minimum addresses near-ties. A top score of 0.42 against a runner-up of 0.39 indicates that two activities match the query almost equally well, and returning the leader presents

a coin toss as a resolution. The condition is therefore evaluated on the difference rather than the level, and it is suppressed above 0.85, where the leader is strong enough in absolute terms that a close second does not undermine it.

The clarification ceiling is paired with a lexical test rather than applied alone. The system counts content words in the query, defined as tokens of at least three characters that appear in neither the English stop-word list, nor the intent vocabulary that identifies which authority type is being asked about, nor a small set of domain-generic terms (task, activity, process, dam, please, tell). A query contributing one or fewer content words is under-specified: in “what about this task” every token is either a stop word or domain-generic, so the ranking that results is driven by nothing the user actually specified. Such a query is answered only when the top score clears 0.60 on its own.

The resulting outcome for a free-text query, writing  $s_1$  and  $s_2$  for the top two scores and  $c$  for the content-word count, is

$$\text{outcome} = \begin{cases} \text{refuse,} & s_1 < 0.15, \\ \text{clarify,} & (c \leq 1 \wedge s_1 < 0.60) \vee (s_1 - s_2 < 0.15 \wedge s_1 < 0.85), \\ \text{answer,} & \text{otherwise.} \end{cases}$$

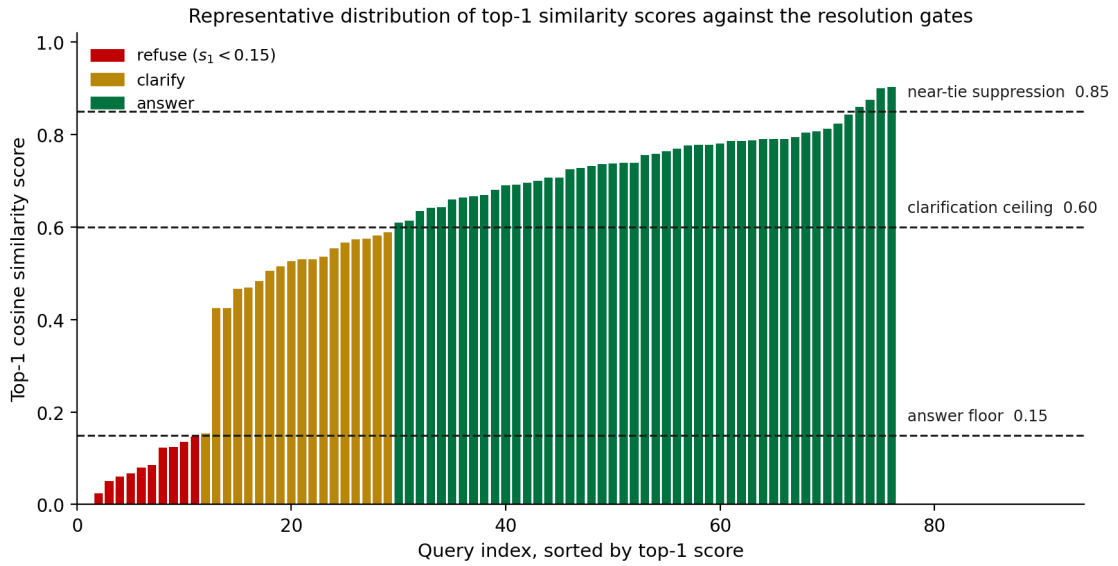


Figure 4.2: Top-ranked similarity score across a batch of free-text queries, with the answer floor, clarification ceiling, and near-tie suppression level marked.

Figure 4.2 places observed scores against these gates.

## 4.3 Deterministic Answer Construction

Once an activity has been resolved, the system still has to decide what the question was actually asking about it, what else the document requires to be said regardless of the question, and what to

do when there is no honest answer to give.

### **4.3.1 Intent Detection**

When the activity has been classified, the system then knows what kind of question is being asked about it. Questions are mapped to authority codes in the document, and the mapping needs to take into account how people ask these questions, with “who signs off on this?” meaning an approval and “who has to check this?” meaning check and verify.

This mapping cannot be naive when it comes to the letter C. According to Chapter 1, C1 and C2 are check and verify whereas C3 and C4 are consult, and mapping “check” to the letter C would be mixing up two different duties. This is why intent detection is against full codes and not their initial letter.

### **4.3.2 Mandatory Obligation Surfacing**

Where an answer provides the response to the question, it also includes that which the document requires to be included. In the case of an action that has a check-and-verify entry in the DAM, that entry will appear even where the question pertained to the approval entry. The names of any informed individuals on record will appear.

This is because this is not an addition of contextual information. It is simply a required entry in the DAM, and it is someone new to the company asking the very specific question who will need the other three answers. This note is hidden only if it would repeat the answer provided.

### **4.3.3 Cross-Reference Following**

Where a resolved activity has no responsibilities of its own but does carry a cross-reference, the system follows the reference and reports the target’s responsibilities, stating that it has done so. This mirrors what a human reader does when a row tells them to look elsewhere, and it converts a dead end into an answer.

### **4.3.4 Honest Refusal**

Three situations lead to a non-answer. In the case of an identifier that doesn’t exist, the fact that it doesn’t exist is stated, and a request is made to validate the identifier or describe what is happening. When a question is too vague to answer, the user is asked to provide additional information. When a question is off-topic, it is considered out of scope.

This is what should not happen in any of these scenarios. This is what requirement FR7 asks for and what makes positive answers valuable: a machine that knows all does not tell us anything about the reliability of its answers.

## 4.4 Auxiliary Query Types

But not all queries are authority queries. There are three more query types that get answers directly.

Glossary queries ask for the definition of an abbreviation and are answered using the abbreviations found in the document along with a small list of aliases for the acronyms found in role descriptions and not explicitly expanded. If the answer comes from the list of aliases, then the source of the definition is reported.

Authority-code queries ask for the definition of a code and are answered from the legend of the transcription. Authority codes detection is conservative on naked codes to avoid collision of 'I' with the pronoun. A naked code becomes a real mention only if it appears in the same sentence with some other code or is unambiguously used, like the parenthesized informed status marker or an alphanumeric combination not forming an English word.

Input queries are matched to anchored patterns and answered directly, bypassing the lookup pipeline. Patterns match anchored to the whole query string to avoid matching of 'hi' in the phrase 'history of the DAM'.

## 4.5 Hybrid Language Model Layer

Answer phrasing is delegated to a language model through a provider interface with two implementations and four operating modes.

Table 4.5: Language model providers and operating modes.

| Mode  | Provider                                    | Behaviour                                                                                    |
|-------|---------------------------------------------|----------------------------------------------------------------------------------------------|
| Auto  | Cloud, falling back to local, then template | Default. Uses whichever provider is available, and the deterministic template if neither is. |
| Cloud | Groq[22]                                    | Hosted inference. Fast, requires an API key and network access.                              |
| Local | Ollama[23]                                  | Runs on the same machine or in a companion container. No external dependency.                |
| Off   | None                                        | Template answers only. Every lookup capability remains available.                            |

It has been intentionally designed such that the agent and the providers interface is very small, and this has served to be of great benefit in the course of the project where a hosted model was deprecated by the provider and started throwing errors in the deployment environment. The solution was a mere modification of a constant behind the interface, without any impact on the agent and its tests.

### 4.5.1 Model Configuration

Table 4.6 records the models each mode resolves to and the task each performs. Every identifier is read from configuration rather than compiled in, so a deprecated model is replaced without a code change.

Table 4.6: Models used by task, with the mode that selects each.

| Task                    | Model                  | Notes                                                                                                                               |
|-------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Answer phrasing, hosted | openai/gpt-oss-120b    | Default for the hosted mode. Reached over an OpenAI-compatible chat completions interface.                                          |
| Answer phrasing, local  | llama3.1               | Default for the local mode, served on localhost:11434. Selected when no hosted credential is present and the local service answers. |
| Transcription           | whisper-large-v3-turbo | Converts a dictated audio clip to text. Hosted only; the local mode has no transcription path.                                      |
| Image understanding     | qwen/qwen3.8-27b       | Used for image attachments. Requested through the same endpoint with an image content part carrying a base64 data URL.              |
| Translation and tone    | As answer phrasing     | Both reuse whichever provider the active mode resolved to; neither has a model of its own.                                          |

Vision support is a property of the provider rather than of the mode. The provider interface exposes a capability flag that is false by default and true only for the hosted provider, and the attachment path tests that flag before constructing a request. An image sent while the local mode is active is refused with a statement of the limitation rather than dispatched to a model that would ignore it.

### 4.5.2 Language Detection

Translation is expensive relative to retrieval, so a deterministic pre-filter decides whether the translation path runs at all. The filter uses three tests and returns true on the first that matches: presence of any character in the Arabic block U+0600–U+06FF; presence of an accented Latin character from the set âãäêëëîïôöûüçñãð; or a whole-word match against per-language function-word lists holding French, Spanish and Portuguese closed-class terms.

A single function-word hit is sufficient. The threshold is set there because a realistic short question contains very few of them once domain vocabulary is excluded: the French phrasing of

“who approves 3.111” contributes exactly one, namely *qui*. Requiring two hits fails on that query.

Setting the bar that low is only safe because of what a false positive costs. A query wrongly flagged as non-English is sent through translation, and the model reports the language as English and returns the text unchanged, so the cost is one additional round trip and never an altered answer. The word lists were also screened against ordinary English DAM vocabulary, and ambiguous single-letter entries were removed for the same reason.

Retrieval itself is never multilingual. A non-English question is translated into English before it reaches the index, and the answer is translated back afterwards, so the index, the correction vocabulary, and the thresholds in Table 4.4 operate on English in every case and behave identically regardless of the language the user typed.

## 4.6 Grounded Generation and the Verification Check

The facts are settled by this point; what remains is turning them into readable prose without letting the model that does the wording quietly change what was said.

### 4.6.1 Contract Given to the Model

This model has no access to the knowledge graph and does not make decisions about what to retrieve. It gets a question, some set of previously retrieved facts, and a system prompt asking it to express these facts naturally without fabricating anything, omitting anything, and using any formatting. In case when the answer needs to be formulated in another language, there is an additional rule for the model that instructs it to translate the sentence without changing role names, identifiers, and footnotes.

Figure 4.3 traces the complete flow, and the position of the verification step in it is the point of the diagram: the model sits inside the loop, not at its end, and nothing it produces reaches the user without being checked back against the facts it was given.

### 4.6.2 Verification Step

The instruction is a prompt, but not a guarantee. The verification converts the instruction into a property of the system.

Before the answer is generated, it is verified whether any role mentioned in the list of facts that back up the answer is found in the generated answer. In case of the absence of any of them, the generated answer is dropped, and a deterministic template answer is provided instead. The verification process relies on the containment test and not on the similarity test. Therefore, the paraphrasing of a role would not pass verification. The instruction that renamed, combined two roles into one or omitted the third role out of four informed people failed to satisfy the verification criterion regardless of fluency.

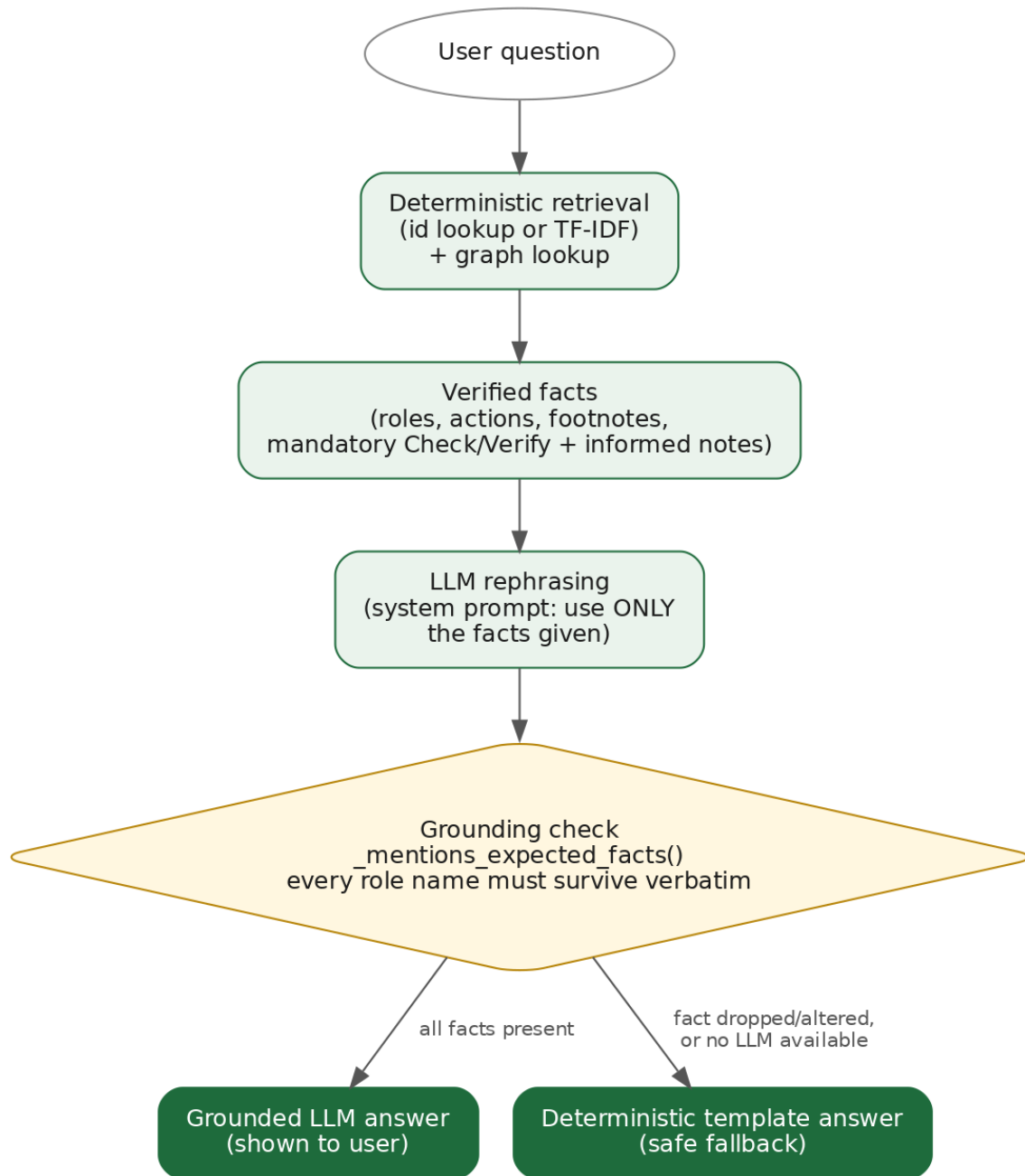


Figure 4.3: Grounded generation flow. The model may only rephrase facts that have already been verified, and any rephrasing that drops or alters a fact is discarded in favour of the deterministic template answer.

For both sides of comparison normalization of whitespace and case was applied. The reason for this tolerance was the fact that the previous version of verification used to fail some otherwise correct answers owing to mere formatting differences. The adjustment made the process less sensitive to formatting without reducing the correctness of verification, as all the same words in the same order should be present anyway.

### 4.6.3 Answer Length and the Fallback Rate

At first, the verification process happened more often than expected. The reason for that turned out to be a conflict between two instructions rather than the lack of reliability of the model. The instruction included in the system prompt dictated the answer containing from one to three sentences. It is impossible to articulate all duties of some activities in three sentences; hence, the model did not include some facts in accordance with the brevity instruction, and the verification was right.

The correction involved loosening the brevity condition rather than the accuracy condition. In case the number of facts backing up an answer exceeded a certain threshold, the brevity instruction was replaced by another instructing to use as many sentences as needed to cover the facts, while other constraints should stay in effect.

## 4.7 Conversational Capabilities

A single-turn question-answer exchange covers most of what earlier sections describe, but a conversation is rarely that tidy in practice. The capabilities layered on top of single-turn resolution include carrying the subject across a follow-up, adapting tone, handling multilingual phrasing, and working from a worked example rather than a bare identifier.

### 4.7.1 Context Carry-Over

A user who has just asked about an activity and then asks “and who has to be informed?” expects the second question to be understood against the first. The client passes the identifier resolved by the previous exchange back to the server as `previous_node_id` on each subsequent request, and where a follow-up contains no resolvable subject of its own, the retrieval layer answers it against that identifier rather than returning a failure. The interface then states explicitly that the subject was carried over, so that a user is never left to infer which activity an answer refers to.

Carry-over is state held by the client rather than by the server. The server remains stateless with respect to conversational context: it is told the previous identifier on each call instead of remembering it, which keeps a conversation reproducible from its transcript alone and avoids any session affinity in deployment.

The mechanism required an explicit suppression rule. Carry-over was initially applied whenever a question failed to resolve a subject, which silently captured question types that have nothing

to do with the previous activity. Asking what an authority code means while an activity was in context produced an answer about that activity rather than about the code, because the code question resolved no subject of its own and was therefore treated as a follow-up. Auxiliary query types, glossary lookups, authority-code explanations and small talk, are now classified before carry-over is considered, and are excluded from it.

### 4.7.2 Tone Adaptation

Questions are classified as neutral, frustrated, or confused, and a short empathetic opener is prepended where the tone is not neutral. The classification is gated behind a deterministic pre-filter: a lexical check for markers of frustration or confusion decides whether a model call is worth making at all, so a neutral question, which is the overwhelming majority of traffic, incurs no additional latency or cost. Tone is assessed against the text the user actually typed rather than against any internally translated version of it, since translation normalises register and would erase precisely the signal being measured.

Tone affects only the opener. It cannot alter the facts that follow it, which are assembled before tone is ever considered.

### 4.7.3 Multilingual Questions

Questions in French, Spanish, Portuguese, or Arabic are detected, translated into English for retrieval, and answered in the language in which they were asked. Detection uses a deterministic pre-filter over character ranges and function-word lists rather than a model call, so an English question, again the common case, is never charged for a capability it does not use.

Retrieval itself remains English-only and entirely unchanged. This is deliberate: the index, the typo-correction dictionary and the identifier grammar are all built from the document's own English vocabulary, and translating the question before retrieval rather than localising the index means that the retrieval behaviour tested in this chapter is exactly the behaviour a non-English user receives. Two distinct language concerns are kept separate in the implementation, the language a question must be translated *from* for retrieval, and the language the answer is phrased *in*, so that an English question can be answered in French when the interface language has been set explicitly.

The verification check described in Section 4.6.2 is not translated. It continues to search for the literal English role names inside the generated answer whatever language the surrounding sentence is written in. A French answer that has dropped a role therefore fails exactly as an English one would, and the guarantee does not weaken as language support widens.

## 4.8 Worked Examples

The following exchanges were run against the delivered build and are reproduced as the system produced them.

Asking “*who approves 2.126*” resolves by identifier with confidence 1.0 to the quarterly mission programme activity, and names the Regional Director General and the Director of the Nigeria office as approvers, with the abbreviation expanded inline. The answer also states that the task must be checked and verified by the Sector Manager and the Supporting Department Division Manager, and that the concerned Sector Vice-President, the Regional Vice-President, and the Task Manager or Project Team must be informed. None of that was asked for. All of it is mandatory, and surfacing it is the behaviour required by FR4.

Asking “*who needs to be informed for quarterly mission program*” resolves the same activity from its name alone, through the lexical path rather than the identifier path, with the answer now centred on the informed parties.

Asking “*who approves the qaurterly mision program*”, with three deliberate misspellings, resolves identically. The correction is deterministic, so the behaviour is identical with or without a model provider available.

Asking “*who initiates 1.117.2*” exercises the redirect path. That activity’s own row records no responsibilities and points to another activity, so the system follows the pointer and reports the target’s responsibilities instead of dead-ending on an empty record.

Asking “*who approves of Communication with Co-Financiers of projects*” followed by “*and who are the informed parties?*” exercises carry-over: the second question, which names no subject, stays on the activity resolved by the first.

Asking “*what happens with 9.999.999*” produces a refusal, stating that the identifier does not exist and suggesting that the user check it or describe the activity in words. This is the behaviour the design exists to produce, and it is worth reading as a result rather than as a limitation: the alternative, a plausible answer about a neighbouring activity, is the failure mode this entire architecture was built to prevent.

## Conclusion

This chapter has described how a question becomes an answer. Resolution proceeds through an identifier fast path and a lexical search layer, both deterministic, and answers are constructed from records held in the knowledge graph rather than generated. A language model participates only in how an answer reads, never in what it asserts, and a verification step compares the generated text against the retrieved facts and discards any rephrasing that has lost one of them.

The value of that separation is that it makes a guarantee possible. Retrieval may fail, and when it does the system says so; but an answer that is returned cannot contain a role or an obligation the document does not record, because no component downstream of retrieval is permitted to introduce one. Around that core sit the behaviours that make the system usable rather than merely correct: typo tolerance, redirect following, mandatory obligation surfacing, context carry-over, tone adaptation, multilingual handling, and explicit refusal.

What this chapter has not addressed is everything surrounding the agent: who is allowed to use it, how conversations persist, how answers reach a user who is not at a keyboard, and what

the accumulated structure of the matrix itself reveals when it is examined in aggregate. The next chapter presents the platform built to deliver those.

## CHAPTER

# 5

# IMPLEMENTATION RESULTS AND DELIVERED PLATFORM

## Introduction

The preceding chapter closed with an agent that resolves a question and constructs a verified answer. That agent is a library: it has no users, no memory, and no way of reaching anyone who is not running Python on the same machine. This chapter presents the platform built around it, and the order of presentation follows the order in which a capability had to exist before the next one became possible.

The chapter opens with the architecture and the boundary that separates the immutable knowledge base from mutable application state, since that boundary is what allowed every subsequent feature to be added without touching the components responsible for correctness. Access control follows, because an unauthenticated system cannot own a conversation, and without ownership neither persistence nor sharing has any meaning. The conversation interface and its input modes come next, then conversation persistence and the sharing mechanism built on top of it. The dashboard is treated separately and at length: it is the one part of the platform that reports on the matrix itself rather than answering questions about it, and it turns out to expose properties of the document that no page-by-page reading makes visible. The chapter closes with the REST surface, the test suite, and deployment.

## 5.1 Application Architecture

Two structural decisions carry the rest of this chapter: how the client and server are split, and how the two very different kinds of data the system holds are kept apart from each other. Both were made early and never revisited, because later features were designed to fit them rather than the other way round.

### 5.1.1 Client–Server Structure

The web client is a multi-page application rather than a single-page one. It is built with React[25] and bundled by Vite[26] into four independent entry points, one each for the landing page, the sign-in page, the conversation page and the dashboard, and the backend exposes a route for each that returns the corresponding pre-built HTML document. The compiled JavaScript and stylesheet bundles are served from two static mounts alongside them.

The distinction between what is written and what is served is worth making explicit, since the two are not the same artefacts. The interface is authored as React components in JSX, a syntax no browser can execute directly. The build step compiles that JSX into plain JavaScript, bundles it, and emits for each entry point a minimal HTML document whose only content is an empty mount element and a reference to the compiled bundle. What the backend serves is therefore the compiled output, not the source: no JSX file exists in the deployed artefact, and the interface a user sees is constructed in the browser by React once that bundle has loaded and mounted into the empty element. Nothing is rendered or templated on the server: the backend returns files that were produced at build time, and every dynamic element of a page is populated by the client calling the API after it loads.

This was a deliberate choice rather than a simplification, and it trades a small amount of navigation smoothness for two properties that matter more here. A single-page application owns its own routing, which obliges the server to answer every unrecognised path with the application shell so that the client-side router can interpret it; get that fallback wrong and a hard refresh on an interior route returns a 404. With one real route per page that failure mode does not exist, because every URL a user can reach is a URL the server genuinely serves. The second property is that authentication boundaries fall on page loads, where the server can act on them, rather than on client-side transitions it never observes.

Serving the client from the same process that exposes the API keeps the deployment to a single service and removes cross-origin configuration entirely, which matters on the free hosting tier described later in this chapter, where a second service would be a second cold start. The backend itself is built on FastAPI[24].

At startup the backend loads the cached node set, constructs the knowledge graph in memory using networkx[9], and fits the TF-IDF index over activity titles. All three structures are held in process memory for the lifetime of the service and are read-only thereafter. A question is therefore answered from memory, with no database round trip and no parsing, which is what makes the response-time requirement achievable on a free hosting tier.

The web layer contains no domain logic. Every decision about the content of an answer is taken inside the agent; the backend authenticates the caller, hands the question to the agent, and serialises what comes back. This separation was maintained deliberately, and its value is visible in the development history: each platform capability described in this chapter was added without modifying the retrieval or generation components, so no feature in this chapter can have altered the correctness guarantee established in Chapter 4.

### 5.1.2 Two Stores, Deliberately Separated

Two kinds of data exist in the system, and almost every design decision in this section follows from refusing to mix them.

The knowledge base is immutable. It is a serialised artefact produced by the pipeline of Chapter 3, shipped with the application, loaded into memory at startup, and never written to at runtime. It lives in no database at all. The consequence worth stating is that the knowledge base cannot be corrupted by application traffic: there exists no code path by which a user action modifies what the system believes the document says. A user can create an account, hold a hundred conversations and share every one of them, and the answer to “who approves 2.126” is bit-for-bit what it was before they arrived.

Application state is the opposite in every respect. Accounts, conversations, and sharing grants are mutable by definition, they are written on ordinary user actions, and they must survive a process restart. They therefore live in a relational database.

Keeping the two apart is what makes the correctness argument of Chapter 4 survive contact with a multi-user application. If the extracted matrix lived in the same database as the conversations, then every feature added to the platform would become a potential vector for altering the facts, and each one would need to be argued against that possibility. Because it does not, the argument is made once, structurally, and does not need revisiting.

### 5.1.3 Choosing the Persistence Layer

The database is a managed PostgreSQL instance hosted on Neon, reached through SQLAlchemy as an object-relational mapper. Two aspects of that choice were forced rather than preferred, and both are worth recording because the reasoning generalises.

The first is why a hosted database at all, rather than a file alongside the application. The application is deployed on a platform whose free tier provides no persistent disk. A file-based database written next to the running service would therefore be destroyed on every redeployment, and the failure would be silent: the service would restart cleanly, the schema would be recreated empty, and every registered account would simply be gone with no error anywhere to indicate it. That is the same class of defect as the memory failure described in Section 5.8, an assumption about the persistence of local state that the platform does not honour, and it was avoided in advance only because the earlier failure had already taught the lesson.

The second aspect is the local-development fallback. Requiring a hosted database to run the test suite would make the tests slow, network-dependent, and impossible to run offline, so the connection string defaults to a local file-based SQLite database when no environment variable is supplied. Production sets that variable and gets PostgreSQL. This yields a genuine engineering problem, since the two engines do not offer the same column types, and the way it is resolved is described next.

### 5.1.4 Identifiers and Dialect Portability

Every table uses a randomly generated UUID as its primary key rather than an auto-incrementing integer. The reason is not aesthetic. Conversation identifiers appear in URLs, and a sequential integer key would make those URLs enumerable: a user holding conversation 41 could try 40 and 42 and learn, from the difference between a refusal and a not-found, how many conversations exist and roughly when theirs was created. Authorisation is still checked on every request, so enumeration would not grant access, but a random 128-bit identifier removes the information leak entirely rather than relying on the check to contain it. Identifiers are generated in the application rather than by the database, which also means an object's identity is known before it is persisted.

The portability problem is that PostgreSQL has a native UUID column type and SQLite does not. Rather than abandoning the native type in production, or maintaining two model definitions, the schema declares the column once with a dialect variant: the type resolves to a native UUID when the engine is PostgreSQL and to a 36-character string when it is SQLite. One model definition drives both, so the objects exercised by the 375 tests on SQLite are the same objects that run against PostgreSQL in production, and a schema change cannot be applied to one and forgotten in the other.

### 5.1.5 Accounts Table

The users table is where the three sign-in paths converge, and its most important property is a column that is allowed to be empty. Table 5.1 gives its definition.

Table 5.1: Accounts table users.

| Column | Type   | Purpose and constraints                                                                                              |
|--------|--------|----------------------------------------------------------------------------------------------------------------------|
| id     | UUID   | Primary key, generated by the application.                                                                           |
| email  | string | Unique and indexed, never null. Normalised to lower case on write so that address casing cannot create two accounts. |
| name   | string | Display name. Nullable, since a provider may not release it.                                                         |

| Column                                                                                                                                                  | Type      | Purpose and constraints                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------|
| hashed_password                                                                                                                                         | string    | <b>Nullable.</b> The bcrypt digest for accounts that have a password. Null for accounts created through a federated provider. |
| oauth_provider                                                                                                                                          | string    | Nullable. Which provider authenticated this account, where one did.                                                           |
| oauth_sub                                                                                                                                               | string    | Nullable. The provider's own immutable subject identifier for this person.                                                    |
| role                                                                                                                                                    | enum      | analyst or administrator. Never null.                                                                                         |
| created_at                                                                                                                                              | timestamp | Account creation, in UTC.                                                                                                     |
| <b>Constraint</b> uq_oauth_identity: the pair (oauth_provider, oauth_sub) is unique, so one provider identity cannot be attached to two local accounts. |           |                                                                                                                               |

A single table serves all three paths, and a row is distinguished only by which columns carry values. A password account populates `hashed_password` and leaves the two federated columns null. A federated account does the reverse. The schema permits both at once, which is what allows a person who signed up with a password to later authenticate through Google and arrive at the same account rather than a second one.

The nullable password column is a security property rather than a convenience. An account created through Google has no password anywhere in this system, so there is no digest to extract from a database disclosure, no credential to reuse against another service, and no password to be phished. The login endpoint enforces the corresponding rule explicitly: it refuses when `hashed_password` is null, so a federated account cannot be attacked through the password form at all. Figure 5.1 shows the table as deployed, with the bcrypt digests visible and the federated columns null for the three accounts registered at the time of writing.

### 5.1.6 Conversations Table

A conversation is stored as one row carrying its entire message sequence in a single JSON column, rather than as a parent row with one child row per message. This is a deliberate denormalisation and it deserves the scrutiny it usually attracts.

The normalised design is the textbook answer and would be correct if messages were ever addressed individually. They are not. Every read in this application fetches a whole conversation to display it, and every write appends to the end of one. There is no query anywhere in the system that asks for a single message, no pagination within a conversation, and no aggregate computed across messages. A child table would therefore contribute a join on every read and an ordering column to maintain, in exchange for a capability nothing uses.

The second reason is that the provenance metadata attached to each answer, the resolved identifier, the resolution method, the confidence score, the model that phrased it and the language it was answered in, is a nested structure whose shape changed repeatedly during development. In a

| id                        | email                   | name        | hashed_password              | oauth_provider | oauth_sub |
|---------------------------|-------------------------|-------------|------------------------------|----------------|-----------|
| 2d1f0737-ccdc-44de-a67... | alaaalamloum@gmail.com  | Troll       | \$2b\$12\$0ceM/FY00rUc/s4... | NULL           | NULL      |
| 49240d7e-03a4-433a-9f6... | adem.mrabet99@gmail.com | adem mrabet | \$2b\$12\$/5lwUcb4DtnMbwZ... | NULL           | NULL      |
| dee7d920-3062-4605-972... | nadinesliti9@gmail.com  | Nadine      | \$2b\$12\$Uf3wh7eFjrMCsnc... | NULL           | NULL      |

Figure 5.1: Accounts table in the deployed instance. Each account carries a bcrypt digest beginning \$2b\$12\$, and the federated identity columns are null for accounts registered with a password.

column-per-field design each of those changes is a schema migration. In a JSON column the client and the agent agree on the shape and the database stores what it is given. Given that this metadata is only ever read back and rendered, never filtered or joined on, the flexibility was worth more than the constraint.

The honest counterpart is stated plainly: this choice forecloses future queries such as “how many answers fell back to the template” without either a scan over the JSON documents or a migration to a normalised form. That is an acceptable trade only because no such query exists today, and it would need revisiting if the usage analytics discussed in Section 5.5.8 were implemented.

Table 5.2 gives the definition, and Figure 5.2 shows it populated.

| id                        | owner_id                  | title                      | title_is_default | messages                                                    |
|---------------------------|---------------------------|----------------------------|------------------|-------------------------------------------------------------|
| 19547d3c-1bdf-47b1-997... | 2d1f0737-ccdc-44de-a67... | Bonjour, je veux savoir... | FALSE            | [{"role": "user", "message": "Bonjour, je veux savoir..."}] |
| 50ce3510-291c-4c03-8f9... | dee7d920-3062-4605-972... | who approves 2.126         | FALSE            | [{"role": "user", "message": "who approves 2.126"}]         |
| 69205f7c-aac4-4121-aea... | dee7d920-3062-4605-972... | who approves 2.126         | FALSE            | [{"role": "user", "message": "who approves 2.126"}]         |
| 73f9db4f-ea8a-4468-bfb... | 49240d7e-03a4-433a-9f6... | good morning               | FALSE            | [{"role": "user", "message": "good morning"}]               |
| 875b34d1-2b2e-4fcf-983... | 49240d7e-03a4-433a-9f6... | Hello, hi my name is A...  | FALSE            | [{"role": "user", "message": "Hello, hi my name is A..."}]  |

Figure 5.2: Conversations table in the deployed instance. Each row carries a whole thread: the owning account, the title derived from the first question, and the JSON message column holding every exchange with its provenance metadata.

Table 5.2: Conversations table conversations.

| Column           | Type      | Purpose and constraints                                                                                       |
|------------------|-----------|---------------------------------------------------------------------------------------------------------------|
| id               | UUID      | Primary key. Appears in share URLs, hence random rather than sequential.                                      |
| owner_id         | UUID      | Foreign key to users, indexed. The account that created the conversation and the only one that may extend it. |
| title            | string    | Display title, defaulting to “New chat”.                                                                      |
| title_is_default | boolean   | Whether the title is still the placeholder.                                                                   |
| messages         | JSON      | The ordered message list, each entry carrying role, text, and provenance metadata.                            |
| created_at       | timestamp | Creation time, UTC.                                                                                           |
| updated_at       | timestamp | Maintained automatically on every write; drives sidebar ordering.                                             |

The `title_is_default` flag solves a small problem neatly enough to be worth a sentence. A conversation should take its title from the first question asked, so that the sidebar is navigable, but a user who renames a conversation must not have that title overwritten by the next message. Storing the title alone cannot distinguish a placeholder from a deliberate choice that happens to resemble one. The boolean records intent rather than inferring it: the first message replaces the title only while the flag is set, and any explicit rename clears it permanently.

### 5.1.7 Sharing Tables

Sharing is carried by two tables that do different jobs, and separating them is what keeps authorisation coherent.

`conversation_shares` records grants that already exist: one row per pair of conversation and recipient. It is the single authority on who may view a conversation, and every read path consults it. The pair is constrained unique, which makes re-sharing with the same colleague idempotent rather than an error or a duplicate row. Two separate foreign keys point at users, one for the recipient and one for the grantor, so the table also records who extended the access rather than only that it exists.

`conversation_share_links` records an invitation that has not yet been accepted: a random token, the conversation it refers to, who issued it, and when it expires, defaulting to seven days. Crucially, a link is not itself a grant. Claiming one creates an ordinary row in `conversation_shares`, which means the QR-code path and the by-name path converge on the same authorisation record, and the question of who may view a conversation continues to have exactly one answer in the code. Had the link been treated as a credential in its own right, every read path would have needed to check two conditions instead of one, and the two would have drifted.

One active link is maintained per conversation. A second request for a QR code returns the existing token rather than minting a new one, so a code already printed or already scanned does not quietly stop working.

Table 5.3 gives both definitions, and Figure 5.3 shows a live grant.

Table 5.3: Two sharing tables.

| Table                    | Column                                                                                                                    | Purpose and constraints                                          |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| conversation_shares      | id                                                                                                                        | Primary key.                                                     |
|                          | conversation_id                                                                                                           | Foreign key, indexed.                                            |
|                          | shared_with_user_id                                                                                                       | Foreign key, indexed. The account granted read access.           |
|                          | shared_by_user_id                                                                                                         | Foreign key. The account that granted it.                        |
|                          | created_at                                                                                                                | When access was granted.                                         |
|                          | <b>Constraint</b> <code>uq_conversation_share: (conversation, recipient)</code> unique, making a repeat share idempotent. |                                                                  |
| conversation_share_links | id                                                                                                                        | Primary key.                                                     |
|                          | conversation_id                                                                                                           | Foreign key, indexed.                                            |
|                          | token                                                                                                                     | Unique and indexed. The random secret encoded into the QR image. |
|                          | created_by_user_id                                                                                                        | Foreign key. Who issued the link.                                |
|                          | created_at                                                                                                                | Issue time.                                                      |
|                          | expires_at                                                                                                                | Expiry, defaulting to seven days after issue.                    |

The screenshot shows the Neon database management interface. On the left is a sidebar with navigation options like Overview, Integrations, Settings, Monitoring, Postgres database, Tables, SQL Editor, Backup & Restore, Computes, and Data API. The main panel displays the 'Tables' section for the 'production' database. Under the 'public' schema, the 'conversation\_shares' table is selected. The table structure is shown with columns: id, conversation\_id, shared\_with\_user\_id, shared\_by\_user\_id, and created\_at. A single record is listed with the following values: id (86072975-55ae-4290-ba7...), conversation\_id (875b34d1-2b2e-4fcf-983...), shared\_with\_user\_id (dee7d920-3062-4605-972...), shared\_by\_user\_id (49240d7e-03a4-433a-9f6...), and created\_at (2026-09-20 19:15:10.8ms).

Figure 5.3: Sharing grants held in `conversation_shares`, showing one live grant. The row names the conversation, the account granted access, and the account that granted it.

### 5.1.8 Referential Integrity and Deletion

Both sharing tables are declared as dependants of the conversation with delete-orphan cascade, so removing a conversation removes its grants and its outstanding links in the same operation. This

matters more than ordinary tidiness. Without the cascade, deleting a conversation would leave grant rows pointing at an identifier that no longer resolves, and an outstanding share link would remain valid in the sense that its token still existed and had not expired. Access-control records that outlive the thing they protect are precisely the kind of residue that later produces a security incident, so the lifetime of a grant is bound to the lifetime of its conversation at the schema level rather than in application code that might be bypassed.

One implementation detail is worth recording because it caused a real error. The share table holds two foreign keys into users, and the mapper cannot infer which one a given relationship traverses when more than one candidate exists; both relationships therefore declare their join column explicitly. The error surfaces at mapper configuration time rather than at query time, which is the preferable failure, but it is opaque enough on first encounter to be worth naming here.

## 5.2 Access Control

Ownership is the concept this whole section serves: nothing about persistence or sharing means anything until the system can say whose conversation it is looking at. Figure 5.4 shows the sign-in screen the decisions below sit behind.

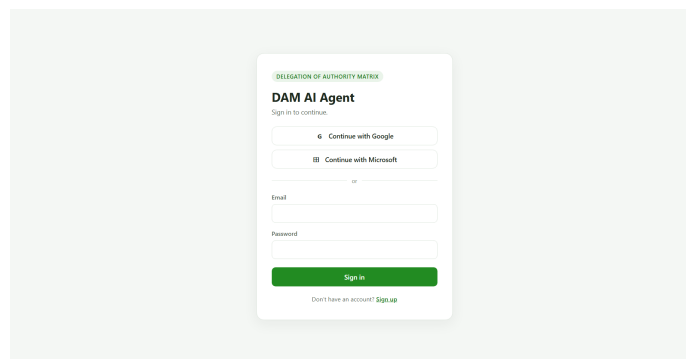


Figure 5.4: Sign-in screen, offering federated sign-in through Google or Microsoft alongside the email-and-password form.

### 5.2.1 Why Authentication Was Necessary

The first version of this system had no accounts. Anyone reaching the service could ask questions, and conversations lived in the browser's own storage. That was adequate while the system did one thing, and it stopped being adequate the moment conversations needed to be persisted and shared, for a reason that is structural rather than incidental: sharing requires ownership, ownership requires identity, and identity requires authentication. There is no way to grant a colleague access to something that belongs to nobody.

A second reason is specific to this subject matter. The analytics layer aggregates the delegation structure of the institution, which is organisationally sensitive in a way that a single lookup is not, and restricting it requires being able to tell who is asking.

### 5.2.2 Password Storage

Passwords are never stored in a form from which the original can be recovered. Each account holds a bcrypt digest, produced through the passlib library, and verification re-derives a digest from the submitted password and compares the two. Bcrypt was chosen over a general-purpose cryptographic hash for reasons worth setting out, since the distinction is frequently elided.

A hash such as SHA-256 is designed to be fast, which is the correct property for verifying file integrity and precisely the wrong one for storing passwords: speed is exactly what an attacker holding a stolen database wants, since it lets them test billions of candidate passwords per second on commodity hardware. Bcrypt is deliberately slow and, more importantly, adjustably slow. Its cost parameter sets the number of key-expansion rounds as a power of two, and raising it by one doubles the work required both for a legitimate verification and for every guess an attacker makes. The digests in this system carry a cost of twelve, meaning  $2^{12}$  rounds, which is visible in the `$2b$12$` prefix of every stored value in Figure 5.1. The parameter is stored alongside the digest rather than fixed in code, so the cost can be raised as hardware improves and existing accounts continue to verify against their original setting until their next successful login.

Bcrypt also salts each password individually and stores the salt in the same field. The consequence is that two accounts choosing the same password hold entirely different digests, which defeats both precomputed rainbow tables and the simpler attack of noticing that two rows share a value and therefore share a password.

The interaction between password storage and federated accounts is the property described earlier: an account authenticated through a provider carries no digest at all, and the login endpoint refuses it before any comparison is attempted.

### 5.2.3 Token Design

Authentication is stateless. A successful sign-in returns a signed JSON Web Token[11], and the client presents it in the Authorization header of every subsequent request. Table 5.4 lists what the token asserts.

Table 5.4: Claims carried by the access token.

| Claim | Value                       | Why it is present                                                       |
|-------|-----------------------------|-------------------------------------------------------------------------|
| sub   | Account UUID                | Identifies the caller. Every ownership check resolves from this.        |
| email | Address                     | Lets the interface show the signed-in account without an extra request. |
| role  | analyst or administrator    | Carries the authorisation decision, avoiding a lookup on every request. |
| iat   | Issue time                  | Establishes token age.                                                  |
| exp   | Issue time plus eight hours | Bounds the damage a leaked token can do.                                |

The token is signed HS256[12], a keyed signature using a secret held only by the server. Signing establishes integrity, not confidentiality: the claims are encoded, not encrypted, and anyone holding the token can read them. Nothing secret is therefore placed in it. What signing guarantees is that a client cannot alter a claim, and the claim that matters is role: without a signature, elevating oneself to administrator would be a matter of editing a string.

The signing secret is read from the environment with no fallback default, so a deployment that has not configured one fails immediately at startup rather than silently signing tokens with a value an attacker could guess from the source. Failing loudly at boot is the correct behaviour for a misconfiguration of this class.

Expiry is eight hours, chosen to span a working day. The trade-off is explicit: a shorter window would force repeated sign-ins during ordinary use, and a longer one would leave a usable credential in a browser long after the person had finished. Because verification is stateless, a token cannot be revoked before it expires, which is the cost of not consulting the database on every request. For this application that cost is acceptable; a system holding higher-value operations would need a revocation list and the lookup that comes with it.

On the client the token is held in session storage rather than local storage. Both are readable by any script executing on the page, so the distinction is not about resisting script access; it is about lifetime. Session storage is cleared when the tab closes, whereas local storage persists indefinitely, leaving a valid credential on a shared or unattended machine long after the user believed they had finished. The cost is that closing the tab requires signing in again, which was judged the right trade for a system holding governance information.

### 5.2.4 Federated Sign-In

Two federated paths exist alongside the password form, through Google and through Microsoft. The motivation is institutional rather than technical: staff at an organisation of this kind already hold a corporate directory account, and requiring a separate password adds a credential to be managed, forgotten, reused, or written down, while removing the ability of the institution to revoke access centrally when someone leaves.

Both providers are integrated through OpenID Connect rather than by hand-coded calls to provider endpoints. The client is configured with a discovery document URL, and the endpoints, signing keys and supported parameters are retrieved from the provider at runtime. This matters for durability: a provider rotating its signing keys or relocating an endpoint does not break the integration, because nothing about those details is hardcoded. The requested scope is the minimum the application needs, namely the identity token plus the address and display name. The Microsoft integration additionally accepts a tenant identifier, defaulting to the multi-tenant endpoint, so that the same build can be pointed at a single institutional directory by configuration alone.

Both providers are registered only when their credentials are present in the environment. An instance configured with neither still starts and still serves password sign-in, which keeps local

development and the test suite free of any dependency on external identity services.

### 5.2.5 Resolving a Federated Identity to an Account

The step where a provider's response becomes a local account is the most security-sensitive part of the flow, and the order of the lookups is the substance of it.

The account is located first by the pair of provider and subject identifier, the provider's own immutable handle for that person, and never by email address alone. This ordering is deliberate. Email addresses are not stable identifiers: an organisation may reassign a departed employee's address to a new hire, and a person may change their own address at the provider. A system matching on email alone would hand the new holder of a recycled address the previous holder's account and every conversation in it. Matching on the subject identifier makes that impossible, because the identifier is never reissued.

Only when no account carries that provider identity does the resolution fall back to email, and then only to link the provider identity onto an existing account rather than to authenticate against it. This is what lets a user who registered with a password later sign in through Google and arrive at the same account rather than a duplicate. The account is created fresh only when both lookups fail.

The uniqueness constraint on the provider-and-subject pair enforces the invariant at the database level, so a defect in this logic cannot produce two local accounts claiming the same provider identity.

Figure 5.5 lays out the same decision as a flowchart, since the order in which the three checks run is the part of this logic most likely to be got wrong by a future change.

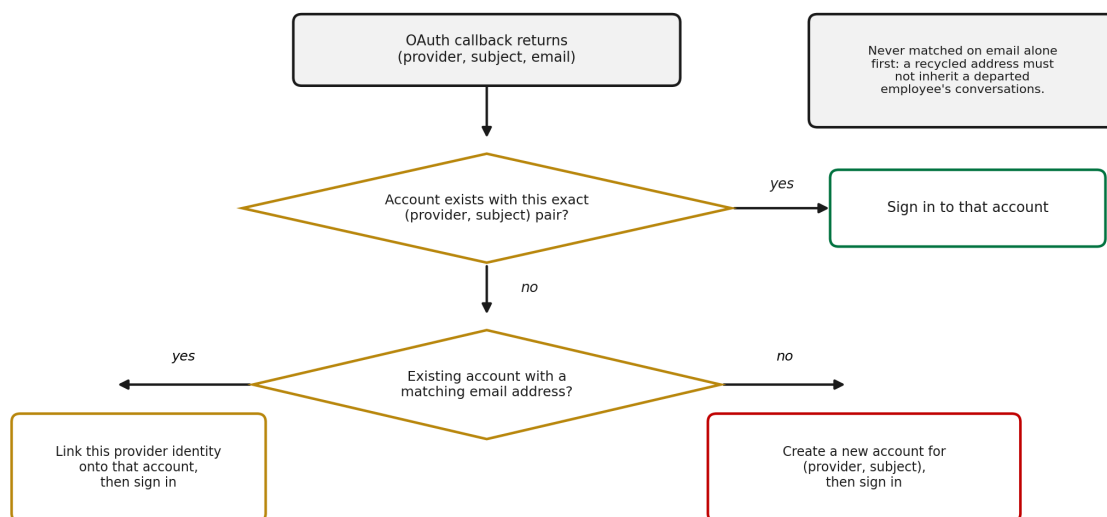


Figure 5.5: Resolving a federated identity to a local account. The provider-and-subject pair is checked first and email is consulted only as a fallback for linking, never for authentication.

### 5.2.6 Roles and Authorisation

Two roles are defined. An **analyst** may ask questions, attach documents, and own and share conversations. An **administrator** may additionally open the analytics dashboard. The distinction reflects the sensitivity difference described earlier: an individual conversation reveals what one person asked, whereas the dashboard aggregates the institution's delegation structure.

Authorisation is enforced server-side as a dependency on the protected routes rather than by hiding controls in the interface. An analyst who constructs the request by hand is refused exactly as one who clicks, which is the only enforcement that means anything; a hidden button is a usability affordance, not a control.

The first account created on an empty database becomes an administrator, which solves the bootstrap problem of needing an administrator before one can be appointed. This is recorded as a deliberate convenience rather than a complete design: promoting a second administrator still requires a direct database edit, and a production deployment would want an administrative interface for it.

### 5.2.7 Hardening the Sign-In Path

Three further protections apply specifically to authentication, and Table 5.5 sets out what each defends against.

Failed attempts are rate-limited by source address within a rolling five-minute window, which raises the cost of credential stuffing from free to impractical without locking out a legitimate user who has mistyped a password twice.

A failed sign-in returns one generic message regardless of whether the account exists. The temptation to be helpful here, by saying that no account was found, produces an oracle that converts a list of institutional email addresses into a list of confirmed users. The implementation is careful on a related point: the same generic refusal is returned when an account exists but has no password because it is federated, so the response does not distinguish that case either.

A minimum password length of eight characters is enforced at registration. This is a weak control on its own and is stated as such; it raises the floor without pretending to establish password quality.

Table 5.5: Authentication threats and the corresponding mitigation.

| Threat                                     | Mitigation in this system                                                                                |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Database disclosure exposing passwords     | Only salted bcrypt digests at cost twelve are stored; federated accounts hold no digest at all.          |
| Offline brute-force against stolen digests | Bcrypt's cost parameter makes each guess expensive, and the parameter can be raised without a migration. |
| Rainbow-table attack                       | Per-password salts make precomputation useless and hide the fact that two accounts share a password.     |

| Threat                                                 | Mitigation in this system                                                                                 |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Account enumeration                                    | One generic refusal for unknown account, wrong password, and federated account alike.                     |
| Credential stuffing                                    | Rate limiting per source address over a rolling window.                                                   |
| Privilege escalation by editing a token                | The role claim is covered by the HS256 signature; any alteration invalidates the token.                   |
| Credential left on a shared machine                    | Tokens held in session storage, discarded when the tab closes, and expiring after eight hours regardless. |
| Conversation URL enumeration                           | Random UUID identifiers rather than sequential integers, with the ownership check retained as well.       |
| Recycled email address inheriting an account           | Federated identities resolved by immutable provider subject identifier before email is ever consulted.    |
| Stale access after a conversation is deleted           | Grants and outstanding links removed by database cascade with the conversation itself.                    |
| Misconfigured deployment signing with a default secret | The signing secret has no default; a deployment without one fails at startup.                             |

## 5.3 Conversation Interface

With sign-in in place, what a signed-in user actually does becomes the subject: ask a question, dictate it instead of typing, attach a document to be questioned separately from the DAM, or read the interface in their own language. Each input mode runs through the same underlying answer pipeline from Chapter 4, since none of them change what an answer is, only how the question reaches it.

### 5.3.1 Asking a Question

The chat page is the primary interface. A question is typed or dictated, and the answer returns accompanied by the metadata that makes it checkable: the resolved activity identifier, the resolution method, the confidence score where one applies, whether a language model phrased the response, and the language the answer is written in. Figure 5.6 shows an exchange in which the agent first declines to guess at an under-specified question and offers candidate activities, then answers the follow-up by identifier at full confidence.

Header controls select the language model mode, as set out in Table 4.5, including switching it off entirely, and select the answer language. Beneath any answer a model has phrased, a control reveals the deterministic template answer underneath it, so a user can compare the readable sentence against the unembellished statement of what the matrix records. This control is the interface expression of the architectural separation argued for in Chapter 4: the two layers remain visibly distinct at the point of use, rather than being merged into a single authoritative-sounding paragraph.

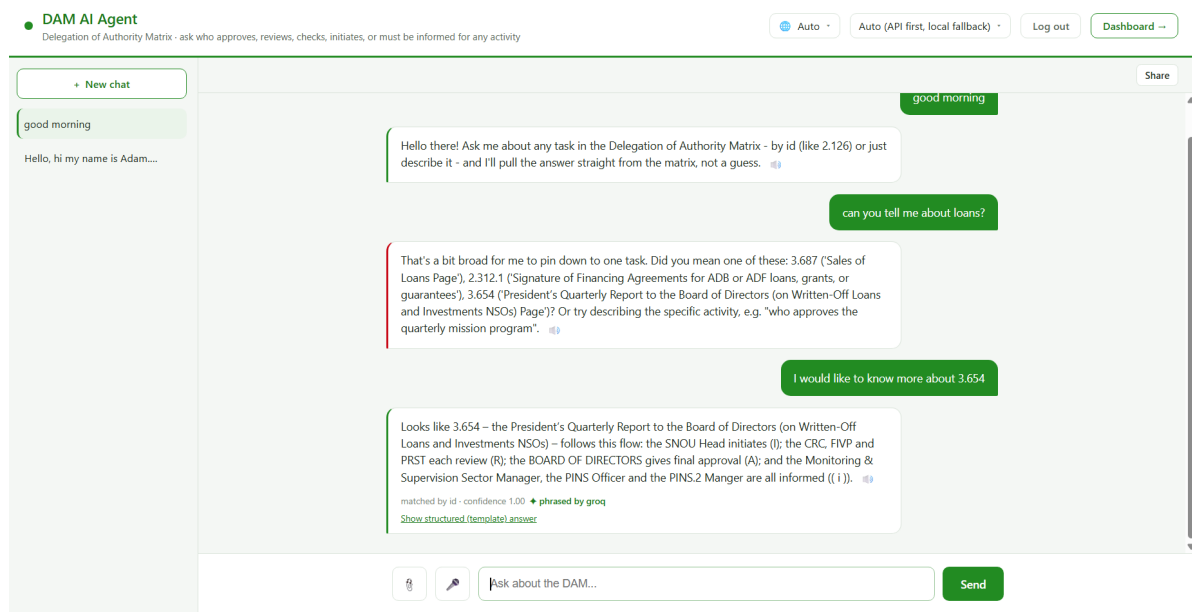


Figure 5.6: Conversation interface. The first answer refuses an under-specified question and offers candidates rather than guessing; the second resolves by identifier at confidence 1.0 and carries its provenance beneath the answer.

The landing page, shown in Figure 5.7, states the same distinction before a user has asked anything, since the difference between an answer retrieved from a structured source and an answer generated from a document is precisely the expectation a new user arrives with.

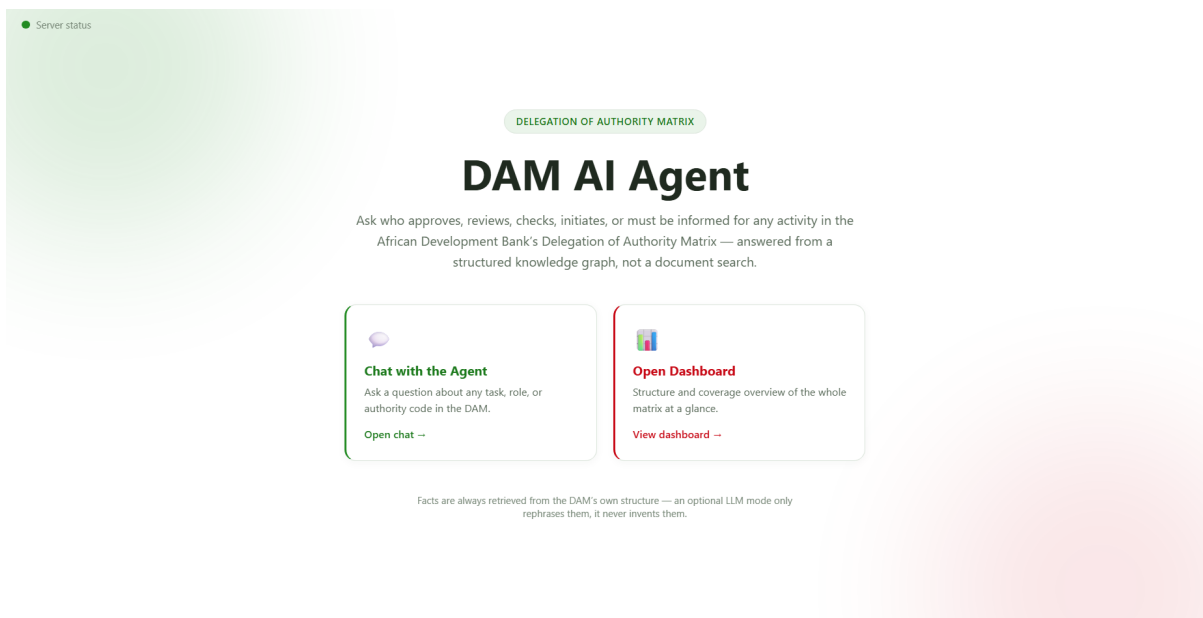


Figure 5.7: Landing page, which states the grounding guarantee before any question is asked.

### 5.3.2 Voice Input

A question may be dictated instead of typed. The composer exposes a microphone control which captures audio in the browser through the `MediaRecorder` interface and posts the resulting clip

to a transcription endpoint, where it is transcribed by a hosted Whisper model and returned as text. Transcription is therefore server-mediated rather than performed in the browser, which keeps behaviour identical across browsers that implement on-device speech recognition inconsistently or not at all.

One design decision is deliberate and easy to miss. The returned transcript is placed into the input field for review rather than submitted automatically. Speech recognition performs poorly on institutional acronyms and on numeric activity identifiers, which is exactly what these questions contain; a user who never sees the transcript cannot correct an identifier that was misheard. Routing the transcript through the input field also means it passes through the deterministic typo correction described in Chapter 4 before retrieval, so a near-miss transcription is frequently recovered without the user intervening at all.

Answers can also be read aloud, and this direction genuinely is browser-local: playback uses the browser's own speech synthesis interface, selecting a voice matching the answer language, with no audio leaving the machine.

### 5.3.3 Attachment Question Answering

A document may be attached to a conversation and questioned directly. The intended use is a source that is not the DAM: a draft policy, a procedure note, or a revised extract of the matrix that has not yet been published.

Uploads are constrained by an allowlist rather than a denylist, so an unanticipated file type is refused rather than silently accepted, and by a fifteen-megabyte ceiling enforced before any byte reaches the storage layer. Accepted files are stored in an S3-compatible object bucket under a key namespaced by the uploading account, and are retrieved for reading through short-lived presigned URLs rather than public links, since a conversation may be reopened long after the attachment was posted and there is no point at which minting a permanent URL would be correct.

Answering differs by document type. Text, CSV, PDF and Word documents are reduced to plain text, PDFs through the same character-level extractor used by the pipeline and Word documents through a dedicated reader, and the extracted text is supplied to the language model with an instruction to answer only from that text and to say plainly when the answer is not present in it. Images are handled through a vision-capable model instead, and the system establishes that the selected provider supports vision before attempting the request rather than discovering mid-request that it does not. Formats that cannot be read reliably, legacy binary Word files and spreadsheets among them, are refused with a message naming the limitation.

The important property of this feature is not the capability but the boundary it draws. An answer derived from an attachment is labelled as such in the interface and carries no activity identifier, because there is none: it did not come from the graph. The guarantee defended throughout this report is that an answer about the DAM is assembled from a verified extraction of a verified document. An uploaded file has been through no such process. It may be a draft, it may be superseded, and the system says so by refusing to present the two kinds of answer in the same visual

language.

### **5.3.4 Interface Localisation**

The agent answers in five languages. The conversation interface follows: its controls, its placeholder text, its empty state, and the provenance labels beneath each answer are held in a message catalogue with one entry set per supported language, and a missing entry falls back to English rather than rendering a raw key. For Arabic the document direction is set to right-to-left so that the conversation layout mirrors correctly.

The scope is the conversation interface, and it is worth being precise rather than generous about that: the landing page, the sign-in form, and the dashboard remain in English. Extending the catalogue to those surfaces is mechanical rather than difficult, and is recorded here as incomplete rather than described as done.

One category of text is deliberately never translated. Role names, activity titles, authority codes and identifiers are reproduced exactly as the source document records them, in every language. They are content rather than interface text, and translating them would break both the traceability requirement and the verification mechanism of Chapter 4, which matches generated text against the literal role names held in the graph.

## **5.4 Conversation Persistence and Sharing**

Once a conversation belongs to someone, two further questions follow naturally: can it survive between sessions, and can it be shown to someone else without handing them the underlying account. The second question is where the real-time mechanism that keeps a shared view from silently going stale comes in.

### **5.4.1 Conversations as Owned Objects**

Every conversation is persisted server-side against the account that created it, which is what makes the rest of this section possible: a conversation that exists only in a browser tab can be neither resumed nor shared. Each exchange is stored with its question, its answer, and the provenance metadata that accompanied the answer, so a conversation reopened weeks later carries the same resolution method and confidence score it displayed originally.

### **5.4.2 Granting Access to a Colleague**

A conversation can be shared with a named colleague, which records a grant against that account and causes the conversation to appear in their own list. Access is read-only: a recipient sees the questions, the answers and the provenance, and can add nothing.

### 5.4.3 Link and QR-Code Sharing

Naming a colleague requires knowing which account they hold, which is an obstacle when the conversation is being shown to someone standing in the room. The system therefore also issues a share link: a random token with a seven-day default expiry, rendered by the client as a QR code so that it can be scanned from a phone without anything being retyped. Figure 5.8 shows the share dialogue with a generated code.

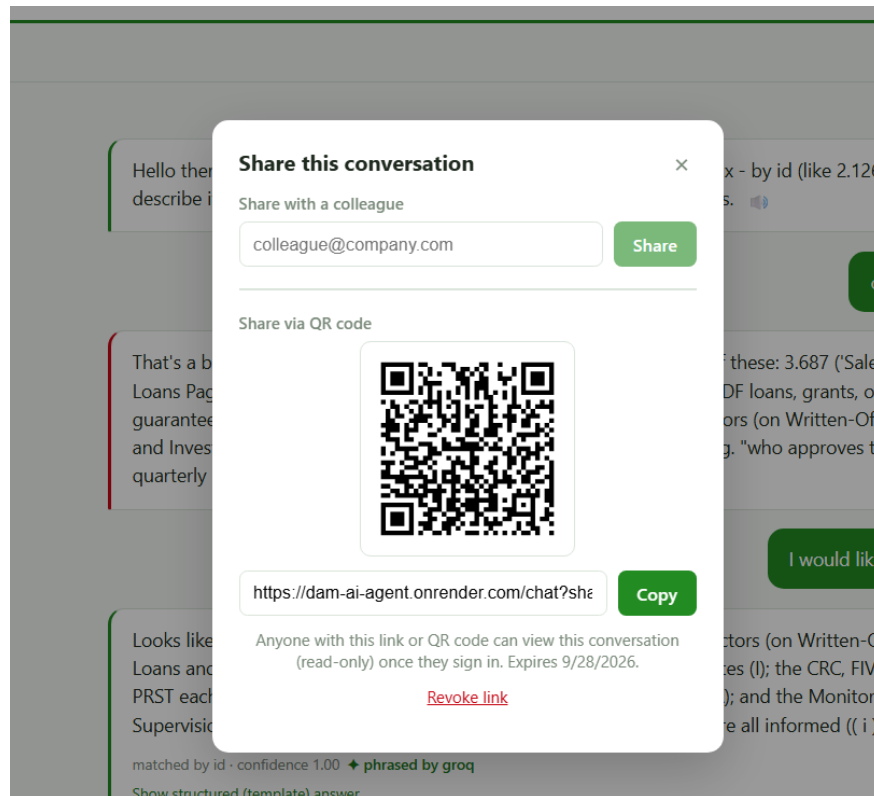


Figure 5.8: Sharing dialogue. A conversation may be shared with a named colleague or through an expiring link rendered as a QR code, with the expiry date stated and revocation available.

Visiting a share link does not itself grant access. The recipient must be signed in, and claiming the link invokes the same grant used by direct sharing, so the question of who may view a conversation has exactly one answer in the code rather than two that must be kept consistent. A link that has expired or been revoked is refused; the originating user may revoke at any time before expiry.

### 5.4.4 Real-Time Propagation

A shared conversation is a live view rather than a copy. Messages added after sharing appear to the recipient, which is the behaviour a colleague consulted mid-conversation expects, and which a snapshot cannot provide.

Propagation uses two mechanisms together. The server maintains a per-account publish and subscribe registry and pushes events to connected clients over a Server-Sent Events stream, so a new message typically reaches a shared viewer within about a second. A thirty-second poll runs alongside it. The redundancy is intentional: the stream is a long-lived connection and free hosting

tiers are not reliable custodians of those, so the poll bounds how stale a view can become if the stream is dropped silently. One implementation detail proved necessary in practice, namely that a background refresh is never permitted to shorten a conversation already on screen, which otherwise allowed a poll completing between a message being sent and its response arriving to momentarily erase it from the sender's own view.

The registry is per-process, which is correct for the single-instance deployment described later in this chapter and would require a shared broker to survive horizontal scaling. This is recorded as a known boundary rather than left to be discovered.

Figure 5.9 traces one message through both channels, and the rule at the bottom of the diagram is the one implementation detail mentioned above: a background refresh may add messages to a shared view but may never remove or shorten what is already displayed.

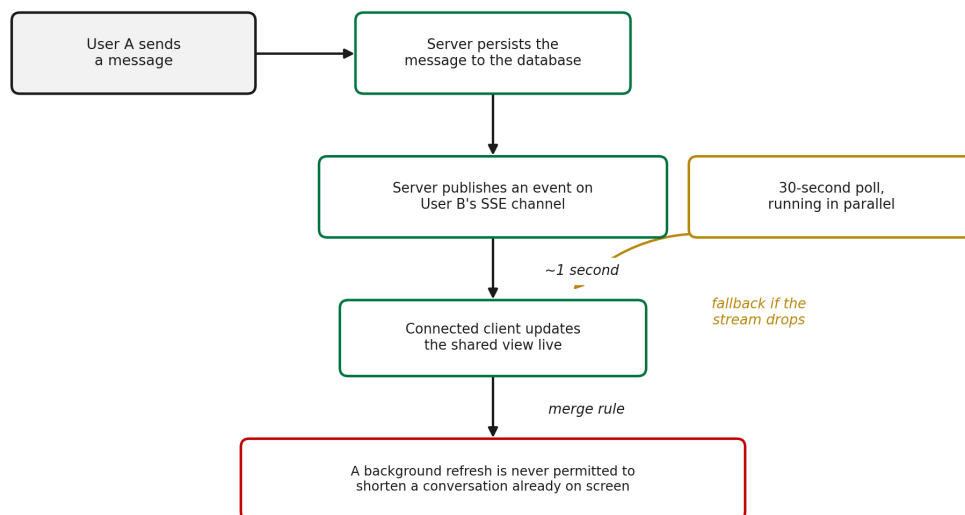


Figure 5.9: Real-time propagation of a shared conversation. The Server-Sent Events stream delivers a new message within about a second; the thirty-second poll is a fallback that bounds staleness if the stream drops.

## 5.5 Analytics Dashboard

Everything up to this point has been about answering a question against one activity at a time. The dashboard is the platform's one departure from that pattern: why it exists, how its numbers are computed, which indicators earned a place on it, and which governance indicators were specified but held back from the delivered release.

### 5.5.1 Purpose

Every other part of the platform answers questions about individual activities. The dashboard does something different: it reports on the matrix as an object, and in doing so exposes properties that no page-by-page reading reveals. A reader consulting the document sees one activity at a time by construction. Questions such as how many roles hold approval authority at all, how concentrated that authority is, or what proportion of the document carries no responsibilities of its own, cannot be answered by reading. They can be answered immediately once the same document is a graph.

Figure 5.10 shows the delivered dashboard.

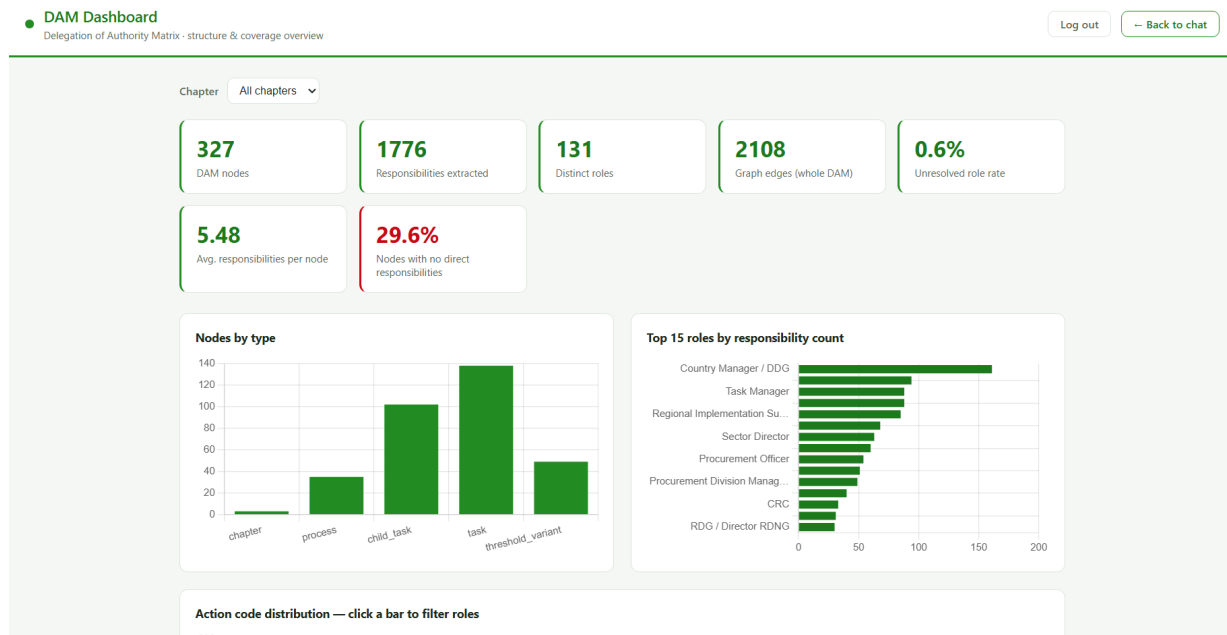


Figure 5.10: Analytics dashboard, showing headline indicators, the distribution of node types, and the roles ranked by the number of responsibilities they carry.

### 5.5.2 Computation Model

No statistic on the dashboard is stored. Every figure is computed at request time from the node set and the graph already held in memory, which has one consequence worth stating explicitly: the dashboard cannot drift out of agreement with the knowledge base, because there is no second copy of the numbers that could become stale. A parser change that alters the extraction is visible on the dashboard at the next request without any recomputation step, and the cost of that guarantee is an aggregation over a few hundred nodes, which is negligible.

The same computation is applied twice at different scopes. It runs once over every node in the matrix, and once per chapter over that chapter's nodes, producing an identical structure in both cases so that the client can render the whole-document view and a single-chapter view through the same code path.

One quantity is deliberately not scoped by chapter. The graph edge total remains whole-document, because responsibility and cross-reference edges routinely cross chapter boundaries:

the same role appears in several chapters, and a reference in one chapter frequently points into another. A per-chapter edge count would require a defensible rule for whether an edge leaving the chapter belongs to it, and any such rule would produce a number more misleading than useful.

### 5.5.3 How Indicators Were Chosen

An indicator earns its place on a dashboard by changing what somebody does. That criterion is easy to state and routinely ignored, and the usual result is a wall of counts that is read once out of curiosity and never again. Each figure reported here was therefore required to answer a specific question that somebody actually has, and the indicators fall into three groups according to whose question it is.

The first group answers *is the extraction still correct*, and its audience is whoever maintains the pipeline. These are regression instruments: they exist because a parser change can break a rule silently, producing a node set that is smaller or emptier or differently distributed while every individual record still looks plausible. A sampled manual check does not catch that. A number that moves does.

The second group answers *what is in this document*, and its audience is a reader trying to understand the matrix as an object rather than to look one activity up. A person consulting the DAM sees one activity at a time by construction, which makes the shape of the whole invisible to them however carefully they read.

The third group answers *what does this document imply about the institution*, and its audience is governance and audit. These are the indicators of Section 5.5.8, and they are specified rather than delivered.

The rest of this section takes the delivered indicators in turn and states, for each, what it measures, why it was chosen, and what it turned out to show. Table 5.6 summarises them first.

Table 5.6: Indicators computed by the delivered analytics dashboard.

| Indicator                  | Value | Answers       | Why it is reported                                                                                    |
|----------------------------|-------|---------------|-------------------------------------------------------------------------------------------------------|
| DAM nodes                  | 327   | What is in it | Establishes the size of the extracted corpus and moves immediately if a parsing change drops records. |
| Responsibilities extracted | 1,776 | What is in it | The real size of the matrix. Activities are containers; obligations are the content.                  |
| Distinct roles             | 131   | What is in it | Bounds how many organisational actors the delegation structure involves at all.                       |
| Graph edges                | 2,108 | What is in it | Confirms the graph is connected rather than a set of isolated records.                                |

| Indicator                             | Value | Answers       | Why it is reported                                                                       |
|---------------------------------------|-------|---------------|------------------------------------------------------------------------------------------|
| Unresolved role rate                  | 0.6%  | Is it correct | The single most sensitive regression detector in the project.                            |
| Average responsibilities per node     | 5.48  | What is in it | Procedural weight per activity, and a second regression signal.                          |
| Nodes with no direct responsibilities | 29.6% | What is in it | Quantifies the redirect structure, and justifies a correctness requirement of the agent. |
| Node type distribution                | —     | What is in it | Separates structural scaffolding from actionable activity.                               |
| Authority code distribution           | —     | What is in it | Shows which obligations the institution actually uses, against the seventeen available.  |
| Top roles by responsibility count     | —     | What is in it | Ranks actors by load and exposes concentration.                                          |

### 5.5.4 Correctness Indicators

**Unresolved role rate.** This measures the proportion of responsibilities whose authority code was located in a cell but whose owning column could not be determined. It exists because of how role attribution works: column headers in the source are printed rotated through ninety degrees, are not recoverable as table headers, and must be reconstructed by clustering header coordinates and matching each code to a column by horizontal alignment. That reconstruction is the most fragile step in the pipeline, and when it degrades it does so quietly, attaching obligations to no role at all rather than raising an error.

It was added as a development instrument rather than as an analytical one. During the geometric attribution work it was the number watched after every change, because a modification that improved one page's clustering while breaking another's is invisible in a sampled check and unmistakable here. Its present value of 0.6% is the figure against which the pipeline's adequacy is argued in Chapter 3, and the ten or so responsibilities behind that percentage are the residue of genuinely ambiguous layout rather than a systematic fault.

**Average responsibilities per node.** At 5.48, this serves a similar purpose from the opposite direction. The unresolved rate catches codes that lost their role; this catches codes that were never found. A parsing change that stops recognising a code in some layout leaves every remaining record perfectly valid, so nothing fails, and the only symptom is that activities become quietly thinner. Watching a mean rather than a total matters here, because a total also moves when the processed scope changes, and a ratio does not.

### 5.5.5 Structural Indicators

**Node and responsibility counts.** The corpus contains 327 records carrying 1,776 individual responsibilities. Reporting both is deliberate, because the second is the real measure of the matrix and the first is routinely mistaken for it. An activity is a container; the obligations attached to it are the content, and a document with few activities and many obligations each is a very different thing from the reverse.

**Node type distribution.** Records are counted by type, which separates the document into structural scaffolding and actionable content. Chapters and processes exist to organise; tasks, child tasks and threshold variants are what a user actually asks about. The distribution is what makes it possible to say honestly how much of a 79-page document is answerable material rather than hierarchy, and it also exposes the threshold-variant population, which is the mechanism behind the near-identical-activities problem that motivated the whole project in Chapter 1.

**Authority code distribution.** Seventeen level-qualified notations are available in the legend. This indicator shows which of them the institution actually uses and how often, and the interesting result is the imbalance: the delegation structure leans heavily on a small subset, while several codes appear rarely enough that their meaning is unlikely to be familiar to most staff. That is an argument for the glossary expansion built into the agent, grounded in a measurement rather than an assumption.

**Distinct roles and the role ranking.** There are 131 distinct roles, and the top fifteen by obligation count are ranked. The count bounds the size of the delegation structure; the ranking exposes its shape. A long flat ranking would indicate authority spread evenly, and a steep one indicates concentration in a handful of offices. The delivered ranking is visibly steep, which is the observation that motivates the authority-concentration indicator specified in Section 5.5.8.

**Graph edges.** The 2,108 edges confirm that the extraction produced a connected structure rather than a pile of independent records. It is reported because the difference between a list and a graph is the entire premise of the approach: if containment and reference edges were absent, the cross-reference following of Chapter 4 would have nothing to traverse.

### 5.5.6 An Indicator That Changed the Design

**Records with no direct responsibilities.** At 29.6%, this is the figure with the largest consequence in the chapter, and it was not anticipated.

It counts responsibility-bearing records that carry no obligation of their own. Encountering it for the first time, the natural reading is that something is broken: nearly a third of the extracted activities appear empty. Checking those records against the printed pages showed the extraction

was correct and the document is simply built that way. Roughly three activities in ten record nothing themselves and point to another activity instead.

That reframed a feature as a requirement. Cross-reference following had been treated as a refinement, worth having for completeness. This measurement showed that a system without it dead-ends on close to a third of the matrix, which is not a gap in polish but a failure on routine questions. The redirect path described in Chapter 4 exists in the form it does because this number was measured before the agent was finished.

It is also the clearest demonstration of why the dashboard was worth building. The fact was always present in the document and always invisible to a reader, because a reader encountering one empty activity concludes that this activity is unusual. Only the aggregate reveals that it is the norm.

### 5.5.7 Interactive Cross-Filtering

The charts are linked rather than independent. Selecting an authority code in the distribution chart filters the role ranking to the roles holding that specific authority, which turns a static display into a means of asking a different class of question: not which roles are busiest overall, but which roles hold approval authority specifically, or which hold check-and-verify obligations. The two rankings are not the same, and the difference between them is exactly the segregation-of-duties structure of the institution.

The cross-filter is computed server-side rather than derived in the browser, so that the counting rules, in particular the exclusion of unresolved roles, are applied in one place and cannot diverge between the ranking and the filtered ranking.

### 5.5.8 Governance Indicators Derivable from the Graph

The indicators above describe the document. A further family describes what the document implies about the institution, and the graph supports all of them without requiring any data source beyond the matrix itself. Table 5.7 specifies them.

These are specified rather than delivered. The dashboard as built reports the structural indicators of Table 5.6; the governance family below is the designed extension, defined here precisely enough to be implemented directly, and is not claimed as a delivered capability.

Table 5.7: Governance indicators derivable from the knowledge graph, specified as the designed extension to the dashboard.

| Indicator               | Definition and interpretation                                                                                                                                  |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authority concentration | Share of activities for which exactly one role holds approval authority. A high value identifies single points of failure, where one absence blocks a process. |

| Indicator                  | Definition and interpretation                                                                                                                                   |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Approval span per role     | Number of activities each role approves. Ranks roles by the breadth of authority they carry and surfaces disproportionate concentration.                        |
| Segregation of duties      | Share of activities where the initiating role differs from the approving role. A standard internal-control measure[13], computed here directly from the matrix. |
| Verification coverage      | Share of activities carrying at least one recorded check-and-verify obligation. Measures how much of the Bank's activity has a mandatory second pair of eyes.   |
| Notification breadth       | Mean number of informed parties per activity, with distribution. Indicates how widely decisions are communicated.                                               |
| Threshold banding rate     | Share of activities whose approver depends on a monetary band. Quantifies how far delegation is value-sensitive rather than flat.                               |
| Authority chain depth      | Mean number of distinct roles involved in an activity across all authority types. A proxy for procedural weight.                                                |
| Documentation completeness | Share of activities with no recorded responsibility of any kind, excluding structural containers. Flags gaps in the matrix itself.                              |
| Redirect dependency        | Share of activities whose responsibilities are reachable only by following a cross-reference. Measures how much of the document is indirect.                    |
| Role redundancy            | Share of activities with two or more roles able to approve. Read together with authority concentration, it separates resilience from ambiguity.                 |

Two of these must be read together rather than separately. Authority concentration and role redundancy are complementary, and either one alone supports the wrong conclusion: a single approver is efficient but fragile, while multiple approvers are resilient but can obscure who is actually accountable. Reported per chapter, the pair shows which pattern each part of the institution has adopted.

Segregation of duties is the indicator most likely to interest an audit function. It is a control principle the matrix already implements implicitly, in that the initiating and approving roles for an activity are recorded separately, and which is currently measured nowhere. Computing it requires no new data, only the graph that already exists, and it is the strongest argument in this project for treating a governance document as structured data rather than as a document.

## 5.6 REST Interface

Table 5.8 lists the endpoints the backend exposes. The surface is deliberately small, and one property of it is worth noting: the question endpoint is the only route that reaches the agent, so every path through which an answer can be obtained passes through the same verification.

Table 5.8: REST API endpoints exposed by the backend.

| Endpoint                                  | Method      | Purpose                                                                                                                                                                                                                                                                 |
|-------------------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /api/auth/signup                          | POST        | Creates an account and returns a signed token.                                                                                                                                                                                                                          |
| /api/auth/login                           | POST        | Exchanges credentials for a signed token. Rate-limited.                                                                                                                                                                                                                 |
| /api/auth/me                              | GET         | Returns the authenticated account and its role.                                                                                                                                                                                                                         |
| /api/auth/google/login, /callback         | GET         | Federated sign-in through Google.                                                                                                                                                                                                                                       |
| /api/auth/microsoft/login, /callback      | GET         | Federated sign-in through Microsoft.                                                                                                                                                                                                                                    |
| /api/ask                                  | POST        | Core endpoint. Accepts a question, model mode, optional carried-over activity, target language, and optional attachment reference. Returns the answer, the deterministic answer, the resolved identifier, resolution method and score, and language and model metadata. |
| /api/uploads                              | POST        | Stores an attachment against the calling account after type and size validation.                                                                                                                                                                                        |
| /api/transcribe                           | POST        | Transcribes a dictated audio clip to text.                                                                                                                                                                                                                              |
| /api/conversations                        | GET, POST   | Lists owned and shared conversations; creates a conversation.                                                                                                                                                                                                           |
| /api/conversations/{id}                   | GET, DELETE | Retrieves or deletes a conversation.                                                                                                                                                                                                                                    |
| /api/conversations/{id}/messages          | POST        | Appends an exchange with its provenance metadata.                                                                                                                                                                                                                       |
| /api/conversations/{id}/attachments/{key} | GET         | Returns a short-lived presigned URL, subject to conversation-scoped permission.                                                                                                                                                                                         |
| /api/conversations/{id}/share             | POST        | Grants a named account read-only access.                                                                                                                                                                                                                                |

| Endpoint                                | Method       | Purpose                                                                                |
|-----------------------------------------|--------------|----------------------------------------------------------------------------------------|
| /api/conversations/{id}/share/{user}    | DELETE       | Withdraws a previously granted access.                                                 |
| /api/conversations/{id}/share-link      | POST, DELETE | Issues or revokes an expiring share link and its QR encoding.                          |
| /api/conversations/shared/{token}/claim | POST         | Claims a share link for the signed-in account.                                         |
| /api/events                             | GET          | Server-Sent Events stream carrying conversation updates for the authenticated account. |
| /api/dashboard/summary                  | GET          | Aggregate statistics, per-chapter breakdowns, and graph totals. Administrator only.    |
| /api/llm/config                         | GET          | Reports the model each mode would currently use.                                       |
| /api/health                             | GET          | Liveness check used by the platform and the status indicator. Reports nodes loaded.    |

## 5.7 Testing and Validation

The system carries 375 automated tests across 33 files. The count matters less than their provenance: these are not tests written to reach a coverage target. Almost every one exists because a specific defect was found, and the test was written to pin the corrected behaviour so that it could not silently return. Table 5.9 gives the distribution.

Table 5.9: Breakdown of the automated test suite.

| Area                          | Tests | Coverage                                                                                                                                 |
|-------------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| Extraction and parsing        | 42    | Task boundary detection, split-row merging, rotated header extraction, title recovery, reference and footnote handling, glossary parsing |
| Node and graph construction   | 16    | Node assembly, hierarchy correctness, graph edges, lossless caching                                                                      |
| Retrieval and agent behaviour | 63    | Identifier fast path, free-text resolution, typo tolerance, small talk, authority codes, glossary lookup, answer orchestration           |

| Area                        | Tests      | Coverage                                                                                                     |
|-----------------------------|------------|--------------------------------------------------------------------------------------------------------------|
| Language model layer        | 59         | Provider mocking and failure handling, verification-step correctness, translation, tone classification       |
| Attachments and storage     | 37         | Text extraction per format, vision routing, object storage, upload validation, attachment question answering |
| Authentication and accounts | 21         | Password hashing, token issuance and expiry, rate limiting, federated sign-in                                |
| Conversations and sharing   | 33         | Persistence, direct shares, share links, claim, expiry, revocation                                           |
| Real-time events            | 6          | Publish and subscribe registry, fan-out, deduplication                                                       |
| API and error handling      | 42         | End-to-end route behaviour, attachment branch authorisation, unhandled-error contract                        |
| Dashboard and client        | 31         | Dashboard metrics, chapter scoping, structural checks on the web client source                               |
| <b>Total</b>                | <b>375</b> | <b>across 33 files</b>                                                                                       |

Automated testing was not the only form of verification. Individual records were checked against the printed pages they were extracted from, which is the only way to detect an answer that is plausible and wrong, and the running system was exercised with real questions before each significant change.

Two defects found during a late verification pass are reported here rather than removed. In one pair of records a column header was attributed as though it were a role, producing a responsibility held by a role name that does not exist in the organisation. In the same two records, two adjacent footnote digits were fused into a single number, yielding a footnote reference that matches no actual footnote. Both are localised to those records, both are visible in a live answer, and both are scheduled for correction alongside the footnote extraction work described in the general conclusion. They are stated because a report that describes only the defects that were fixed gives a misleading account of the state of the system.

## 5.8 Deployment

The application is containerised[32] through a multi-stage build that compiles the web client and packages the result with the Python service, so the image that runs in production is the image that was built and tested rather than an environment assembled at deploy time. A companion composition adds a local model container for offline operation; the hosted provider needs no container, being reached over the network.

Deployment targets a free hosting tier[33] with automatic redeployment on every change to the main branch. The first attempt failed, and the failure is instructive. The service parsed the source PDF at startup, exceeded the platform's memory ceiling, and was terminated before it ever served a request. The resolution was to move parsing to build time and ship the serialised node set, reducing startup to a file load. This brought both memory use and startup time within the platform's limits, and it is the direct reason the knowledge base is an immutable build artefact rather than something constructed at runtime, as described at the start of this chapter. The constraint produced the architecture.

## Conclusion

This chapter has presented the platform delivered around the agent. Authentication establishes ownership; ownership makes persistence meaningful; persistence makes sharing possible; and sharing, once conversations are live objects rather than snapshots, required a real-time channel. Each capability rests on the one before it, and none of them required modifying the retrieval or generation components, which is the practical payoff of the architectural separation argued for in Chapter 4.

The dashboard is the part of the system that does something the original document cannot. Reading the matrix answers questions about one activity; querying the graph answers questions about the institution. The structural indicators delivered here are modest in number, but two of them, the unresolved role rate and the proportion of records with no direct responsibilities, turned out to serve as both engineering instruments and analytical findings, the second quantifying a redirect structure that directly justifies a correctness requirement of the agent.

The system is deployed, reachable, and covered by 375 automated tests. It is also incomplete in ways this chapter has stated rather than concealed: localisation stops at the conversation interface, the governance indicators are specified but not implemented, and two extraction defects remain live. The general conclusion which follows sets these against what the project set out to do.

# GENERAL CONCLUSION

This work started with a document that has a problem with how it answers an important question: it's not a problem with the question itself, it's a problem in its form, and that form makes it difficult to retrieve. The DAM says who is authorized to approve, check, review, initiate, and be informed about all of the Bank's activities, and it is written within wide tables, with sideways headers, and a compact notation which cannot be easily looked up by hand or processed by computer.

What is constructed in response is a system which looks at the question of what facts are, and the question of how to say them. It was successfully parsed into 327 valid records with 1,776 responsibility entries, and converted to a graph of 459 nodes and 2,108 edges, with less than one percent of the responsibility entries not yet resolved. The answers are built from records in that graph; the answers are rewritten, to natural sentences, by a language model under a verification step which does not accept any rewriting that omits or changes a fact; questions are resolved to activities by an exact identifier path and a lexical search layer. Around that core is an authenticated platform with voice input, an explicit trust boundary for document attachment, conversations that can be shared read-only with a colleague and stay live as they are extended, a conversation interface localised into five languages, and an analytics dashboard that reports the structure and extraction quality of the matrix itself.

The following three points seem to be conclusions that can be drawn from this work.

The first is related to the extraction of documents. All of the significant problems that arose during this project yielded seemingly plausible output that would not have been detected by simply verifying that the program ran without error. They were discovered by cross-referencing the extracted information with the pages in which it was found. If the document is visual, then, that comparison is not a nice-to-have for quality assurance, but it is the only way.

The second relates to language models in factual contexts. Asking a model not to make up facts is not a given. It had to be made a property of the system, which involved a mechanical check of the model's output, against the facts it was provided with. It is easy to check, inexpensive, and

it is why there is any use for a generative component in a compliance context. It's the same idea as what's been described in Chapter 5 about logging how often that check fires: a safety property that's not measured in production is an assumption; it is not a guarantee.

The third is about refusal. The third is about refusal. The ability of the system to state that an identifier does not exist, or that a question falls outside the document it holds, rather than returning the nearest plausible match, is what makes the rest of its answers worth trusting.

## Limitations

The DAM was processed from a document of more than two hundred and fifty pages, and each chapter of the DAM is processed. The other chapters follow the same format, but that is yet to be proven.

Footnote handling is inadequate, and the most significant inadequacy. The responsibility numbers are extracted and linked to the footnote, and there are 113 such occurrences of responsibility numbers out of 1,776 of responsibility entries, but the text at the foot of the page is not parsed. The system can thus say that a qualification applies to a responsibility but fail to say what that qualification is. Because some footnotes in this document have conditions attached to them, such as delegation ceilings and prior approval requirements, an answer that names an approver but doesn't state the conditions attached to the approval is accurate but might be misleading. The two related extraction defects 1.116.1 and 1.116.2, the column header labeled as a role, and two footnote digits that are combined are all in the same part of the pipeline.

This was intentionally not started before a question, which the schema itself raises: is it the beginning of each chapter or of each process that footnote numbering starts again? The analysis of the extracted references shows that each reference appears at chapter level, as eleven different footnote numbers are repeated in two or more chapters, and there is no restart of numbers in each chapter. It is still not known for certain from two printed pages, however, which requires a parser to be built upon, something the rest of this project did.

Retrieval is lexical—that is, if the question is strictly in the terminology of the document, it will not be a successful retrieval. The system represents one version of one document and there is no way to determine whether the underlying DAM has been updated, which is more of an operational than theoretical concern in an institutional environment.

## Perspectives

The most straight-forward continuation is to finish the footnote extraction (which fixes the two known footnote extraction flaws at the same time), and then to process the rest of the chapters, testing that the parsing rules hold up as expected.

In addition, it would be desirable to have a hybrid retrieval layer that integrates the existing lexical matching with a semantic component to better resolve queries that use out-of-vocabulary

terms, while maintaining the existing verification step unchanged. If a versioning mechanism can compare a new issue of the DAM with the existing knowledge base, and report on any changes, then a static reference becomes a maintainable one.

The direction of analytics is the most open. The governance indicators in Chapter 5 are indicators of properties of the matrix that no one is currently measuring, and implementing them requires no data beyond the graph that already exists. Recording consultation activity would open a second direction, since a log of which activities are actually asked about would turn unanswered questions into a prioritised work list for extracting the remaining chapters. If the analysis of the graph could be extended to questions that the printed document cannot readily answer, such as where authority is located, in which roles there are unexpectedly many people in approval paths, or where one point of failure, such as an approval path, has more than one person in a chain of obligations, then it would be a governance instrument, and not merely a lookup gadget.

# BIBLIOGRAPHY

- [1] G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.
- [2] C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, U.K., 2008.
- [3] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [4] Z. Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):article 248, March 2023.
- [5] A. Hogan et al. Knowledge graphs. *ACM Computing Surveys*, 54(4):article 71, July 2021.
- [6] N. Provos and D. Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, 1999.
- [7] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [8] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [9] A. Hagberg, D. Schult, and P. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15, 2008.
- [10] S. Seabold and J. Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference (SciPy)*, pages 92–96, 2010.

- [11] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). IETF RFC 7519, May 2015.
- [12] M. Jones, J. Bradley, and N. Sakimura. JSON Web Signature (JWS). IETF RFC 7515, May 2015.
- [13] Committee of Sponsoring Organizations of the Treadway Commission (COSO). *Internal Control — Integrated Framework*. 2013.
- [14] African Development Bank. *Delegation of Authority Matrix*. Internal institutional document; primary data source of this study.

# WEBOGRAPHY

- [15] Polytech Intl — École Polytechnique Internationale Privée de Tunis. <https://pi.tn/en/> Accessed February 2026.
- [16] African Development Bank Group. About the African Development Bank Group. <https://www.afdb.org> Accessed February 2026.
- [17] ChatPDF. <https://www.chatpdf.com> Accessed March 2026.
- [18] Google. NotebookLM. <https://notebooklm.google> Accessed March 2026.
- [19] Microsoft. Microsoft 365 Copilot. <https://www.microsoft.com/microsoft-365/copilot> Accessed March 2026.
- [20] J. Singer-Vine. pdfplumber [software]. <https://github.com/jsvine/pdfplumber> Accessed March 2026.
- [21] Pydantic developers. Pydantic documentation. <https://docs.pydantic.dev> Accessed April 2026.
- [22] Groq Inc. Groq API documentation. <https://groq.com> Accessed May 2026.
- [23] Ollama. Ollama documentation. <https://ollama.com> Accessed May 2026.
- [24] S. Ramírez. FastAPI documentation. <https://fastapi.tiangolo.com> Accessed June 2026.
- [25] Meta Open Source. React documentation. <https://react.dev> Accessed June 2026.
- [26] Vite contributors. Vite documentation. <https://vitejs.dev> Accessed June 2026.

- [27] W3C Community Group. Web Speech API. <https://wicg.github.io/speech-api/> Accessed July 2026.
- [28] python-jose contributors. python-jose documentation. <https://github.com/mpdavis/python-jose> Accessed July 2026.
- [29] passlib contributors. Passlib documentation. <https://passlib.readthedocs.io> Accessed July 2026.
- [30] OWASP Foundation. JSON Web Token and Session Management cheat sheets. OWASP Cheat Sheet Series, <https://cheatsheetseries.owasp.org> Accessed August 2026.
- [31] ReportLab. ReportLab PDF library documentation. <https://www.reportlab.com> Accessed August 2026.
- [32] Docker Inc. Docker documentation. <https://docs.docker.com> Accessed August 2026.
- [33] Render Services, Inc. Render documentation. <https://render.com/docs> Accessed September 2026.

